Data Architecture Reference

A measured reference for data architecture and state design

Edition 3 preview 7

Built from commit `repo/book-plan-appendix-v1-5-gc54b8f62` (`c54b8f627478d5589acbe7109e94305e86a0de09`) — working tree false.
Typst 0.15.1 (image `sha256:032e292249bcd378480cc7c142cfa324b63ef8aadeb88d7e7230320c4c9c422f`).
Evidence registry `book/evidence/v6/claims.json`, digest `fd23f6f2784c576714265da25839f8fa9876f46d0badf38a48e3249d03492891`.

Edition and provenance

This is a generated artefact: Typst source plus the evidence registry, compiled in a pinned Podman image. **Every claim in this book must resolve to a claim in the active registry**, and every number must trace back to something actually measured in a report this book cites. Nothing here is hand-entered at build time; the fields below are injected from the build script.

Source commit	`c54b8f627478d5589acbe7109e94305e86a0de09`
Source describe	`repo/book-plan-appendix-v1-5-gc54b8f62`
Working tree at build	false
Built at (UTC)	`2026-10-06T00:49:32Z`
Typst version	0.15.1
Typst image digest	`sha256:032e292249bcd378480cc7c142cfa324b63ef8aadeb88d7e7230320c4c9c422f`
Evidence registry	`book/evidence/v6/claims.json`
Evidence digest	`fd23f6f2784c576714265da25839f8fa9876f46d0badf38a48e3249d03492891`
Source tree hash	`e499873d31cdea45f7cd6852fe29692eb351e01dee4f77cddaaacdf6cdf01ce7`

The book is assembled with Typst inside a pinned Podman image; the evidence index at the end is rendered from the registry file at build time.

Coverage gap

Environment and provenance limits travel with every number: all results come from one host (`host-zenbook-ux5406sa`) with shared cores and no real network between "cluster" nodes. Run tags mark the **producing code state**, not a tree containing the results. A reader reproducing a run must check out the run tag to get the code, then read the committed results directory.

How to read a number — the badge legend, the gap kinds and the rules that hold everywhere — is in **How to use this book**, on the page after this one.

How to use this book

Two reading paths

Learn a strategy: Theory → topic → strategy.
Choose for a situation: Study cases → business case → tested variation.

Part	What you will find
Theory	The mechanism, a picture, fit/failure conditions, costs and a case link.
Study cases	The business, variations, thrive/die situations and compact measured summaries.
Appendix	How to judge a result, reproduce it and inspect its full evidence card.

Reading a thrive-or-die example

- **Observed:** a named comparison in the cited evidence showed this effect.
- **Inferred:** the mechanism suggests this boundary, but that workload was not measured.
- **Not measured:** there is no result to resolve the choice.

Reading a number

A badge links a summary to its full claim card. Read the workload and the limit with the number. Results come from one shared-core host with no real network between cluster containers.

Badge	Meaning
Replicated	Observed across whole runs; the card still names spread and limits.
Repeated — no prior prediction	Observed across whole runs, but no direction was recorded before them.
Single run	Direction in one run, not a stable universal magnitude.
Mechanism	An implemented mechanism; no measured rate is implied.
Analogy	Transferred intuition, not evidence for the new workload.
Gap	Absent coverage, a missing instrument, an unsupported schema or unstable measurement.

Remember

- Learn the mechanism in Theory; check its tested boundary in the matching case.
- A read gain may carry a write, storage or correctness cost.
- The full evidence cards are reference material, not extra teaching chapters.

Contents

How to use this book . . . 3
Reading a thrive-or-die example . . . 3
Reading a number . . . 3
Part I · Theory . . . 6
Database schemas . . . 6
Normalized rows . . . 7
Indexes . . . 9
Rolldown . . . 11
Rollup . . . 13
Embedding . . . 15
Database SELECTs . . . 17
The answer contract . . . 18
Joins and filter placement . . . 20
CTE folding and materialization . . . 23
Membership and absence . . . 25
Aggregation and per-parent top-N . . . 27
Pagination and access paths . . . 29
Concurrent writes and time . . . 31
Hot rows . . . 32
Optimistic version checks . . . 33
Pessimistic locking . . . 34
History and current state . . . 35
Expiry and clock authority . . . 37
Caches . . . 39
Cache-aside and freshness . . . 40
Invalidation and stale fills . . . 42
Leases and fencing . . . 44
Queues . . . 46
Asynchronous handoff . . . 47
Idempotency and duplicate delivery . . . 48
Cluster placement . . . 50
Colocation and distributed access . . . 51
Part II · Study cases . . . 53
Study-case coverage map . . . 53
Charity donations . . . 54
Variation: find donations without copying everything . . . 54
Variation: answer the dashboard from stored totals . . . 55
Variation: put donations inside the donor . . . 55
Variation: ask who donated last in a period . . . 55
Variation: join donations and filter them . . . 56
Variation: reuse one expensive CTE . . . 56
Variation: who has, or has not, donated . . . 57
Variation: donor totals and each donor’s top donation . . . 57
Variation: page through donations, or match two ranges . . . 59
Variation: place the joined donations on three nodes . . . 59
Choosing a SELECT template . . . 60

General-admission ticket sales ... 61
Variation: spread the hot drop across seat rows ... 61
Variation: retain the sale after a refund ... 61
Variation: pay for history under contention ... 62
Reserved seats and checkout promises ... 63
Variation: check the promise inside the write ... 63
Variation: read a section as one document ... 64
Variation: choose the hold window from history ... 64
Configuration publication ... 65
Variation: store entries as rows or as a document ... 65
Variation: choose who maintains the overview ... 65
Variation: many writers edit one hot installation ... 65
A cached donor portal ... 67
Variation: cache a warm read ... 67
Variation: coordinate copies across application instances ... 67
Variation: require strict freshness ... 68
Variation: keep the cache correct through churn ... 68
Variation: count cache resources and locality honestly ... 68
Appendix · Evidence and reproduction ... 70
How to read and reproduce a result ... 70
Facts, derived state and history ... 72
Detailed coordination reference ... 73
Reproduce the book ... 74
Reproduce a measured run ... 74
Query plans behind the SELECT claims ... 75
Join spellings and CTE fences ... 75
Membership and absence ... 80
Totals and the top row per parent ... 83
Predicates and pagination ... 87
Placement on three nodes ... 90
Glossary ... 93
Evidence registry (active v6 claims) ... 96
normalized facts ... 96
rolldown / duplicated data ... 107
rollups / materialized aggregates ... 108
embedded documents ... 109
append-only history / snapshots ... 111
external read copies / caches ... 114
Retired predecessor claims ... 117

Part I · Theory

Theory explains the choice. Each topic groups strategies; each strategy shows its mechanism, fit, cost and an experiment that could challenge the expectation.

Database schemas

Choose where facts live before choosing how quickly to retrieve them. These strategies trade read work for storage, maintenance and coordination.

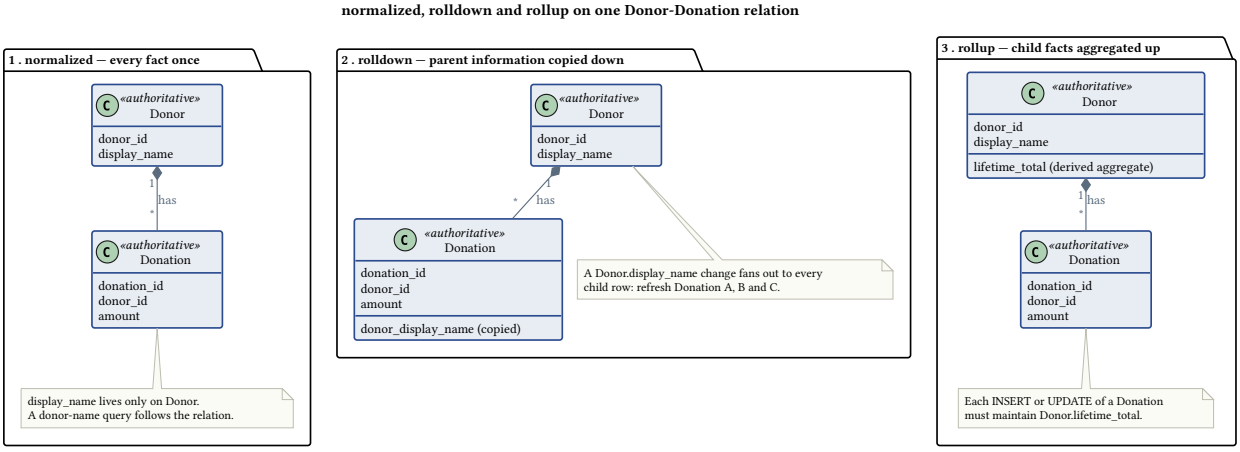


Figure 1: **Database schemas: the choices at a glance.**

Form: structure · Evidence: conceptual illustration

> **Text equivalent.** The picture maps the mechanisms. Layout and arrows do not represent measured throughput.

Strategies: Normalized rows · Indexes · Rolldown · Rollup · Embedding.

Normalized rows

STRATEGY · NORMALIZED ROWS

Key idea

Keep each fact in its owning row; combine rows when an answer needs several owners.

The problem

A donor belongs to a charity and has many donations. Repeating the same charity details on every donation creates extra update paths.

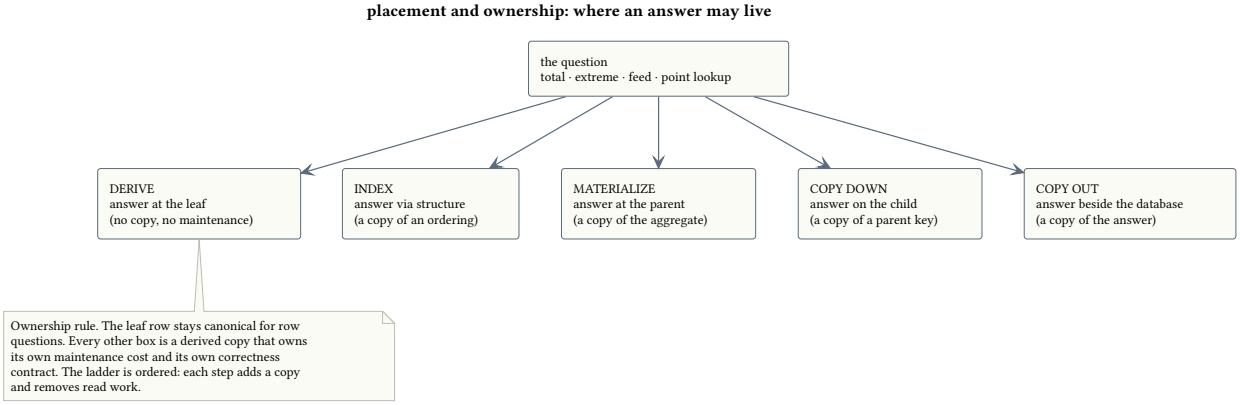


Figure 2: **Normalized rows: mechanism and trade-off.**

Form: structure · Evidence: conceptual illustration

> **Text equivalent.** A read can derive from facts or use an index, aggregate, copied key or external copy. Every added copy needs maintenance.

How it works

- Give each entity a key.
- Store relationships as references.
- Join or aggregate when serving a combined answer.

Thrive or die: the expected boundary

It fits when	It does not fit when
Facts change independently.	Nearly every read reconstructs an expensive aggregate.
Children grow without a small fixed bound.	The same remote join dominates the workload.
Integrity and selective updates matter.	A proven read bottleneck justifies maintained copies.

What it costs

Reads	Joins and aggregation remain on the read path.
Writes	Update the owning fact once.
Storage	Separate rows plus relationship indexes.
Correctness	References must resolve to valid owners.
Operations	Migrations preserve keys and relationships.

In practice

Follow the matching study case for tested variations and registered results. A plausible boundary is not a measured crossover.

The checklist

Question	Answer	Evidence
What is the invariant?	A fact has one authoritative owner.	mechanism
Where does the information live?	Entity rows linked by keys.	mechanism
Where is the contention?	Frequently changed rows and shared indexes.	mechanism
What work moved from reads to writes?	None; combined answers are computed when requested.	mechanism
What state is derived?	Query results.	mechanism
What can become stale?	Maintained external copies, if introduced.	mechanism
Under which workload does it stop working?	Repeated expensive reconstruction of the same answer.	gap
Which experiment would prove it?	Compare matched indexes before adding a copied fact.	gap

Indexes

STRATEGY · INDEXES

Key idea

Store an access path so a query reaches the rows it needs without searching all rows.

The problem

A donation feed filters by charity and orders by time. An index helps only when its keys support that question.

two different tools: the index narrows the scan, the hot row serialises the writers

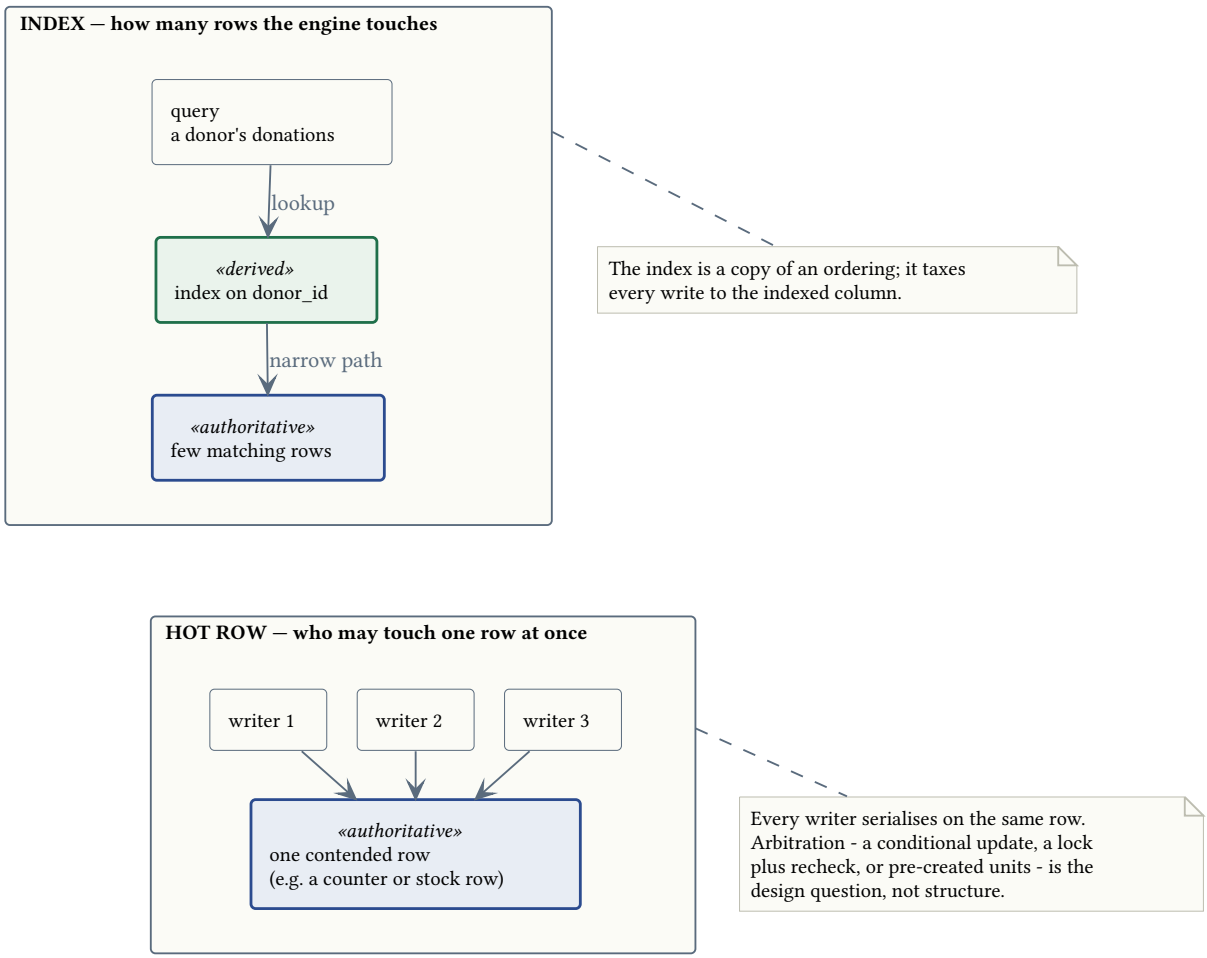


Figure 3: **Indexes: mechanism and trade-off.**

Form: structure · Evidence: conceptual illustration

Text equivalent. An index shortens a read's search path. It does not remove the shared row on which concurrent writers collide.

How it works

- Match leading keys to filters and ordering.
- Read the actual plan, including heap visits.
- Maintain the index when indexed values change.

Thrive or die: the expected boundary

It fits when	It does not fit when
Queries select a small useful subset.	Most rows are needed anyway.
Filters and ordering match index keys.	Writes dominate and many indexes duplicate work.
Read savings repay index maintenance.	An index is blamed for a separate hot-row bottleneck.

What it costs

Reads	Fewer candidate rows for matching queries.
Writes	Maintain each affected index.
Storage	Extra key and row-location entries.
Correctness	An access path must preserve the answer.
Operations	Review plans and retire unused indexes.

In practice

Follow the matching study case for tested variations and registered results. A plausible boundary is not a measured crossover.

The checklist

Question	Answer	Evidence
What is the invariant?	Return the same qualifying rows.	mechanism
Where does the information live?	Base table plus derived access paths.	mechanism
Where is the contention?	Index pages and hot base rows.	mechanism
What work moved from reads to writes?	Access-path maintenance.	mechanism
What state is derived?	Index entries.	mechanism
What can become stale?	A transactional index should track committed base rows.	mechanism
Under which workload does it stop working?	Low selectivity or excessive write maintenance.	gap
Which experiment would prove it?	Match data and SQL, change one index, inspect plans.	gap

Rolldown

STRATEGY · ROLLDOWN

Key idea

Copy a parent key onto its children to shorten their read path.

The problem

A donation lookup needs its charity, but the normalized route goes through the donor. A copied charity key removes that hop.

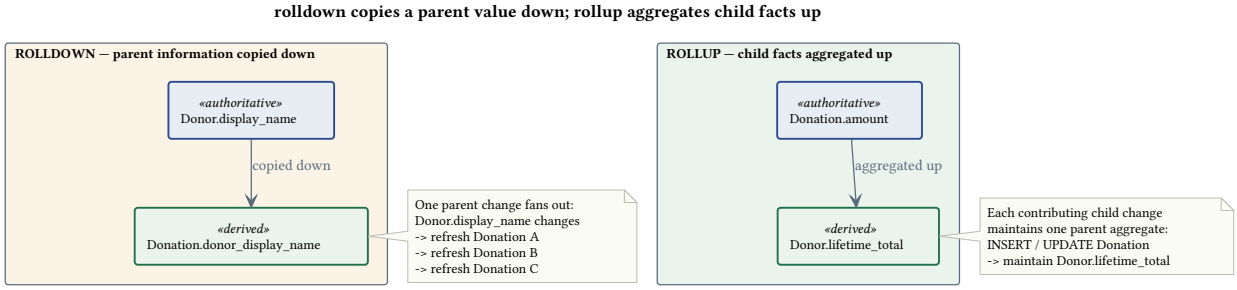


Figure 4: **Rolldown: mechanism and trade-off.**

Form: structure · Evidence: conceptual illustration

> **Text equivalent.** Rollup maintains a parent aggregate when children change. Rolldown maintains child copies when parent information changes.

How it works

- Copy the parent value when creating a child.
- Use it directly in child filters or indexes.
- Repair every affected child when the parent fact changes.

Thrive or die: the expected boundary

It fits when	It does not fit when
Parent relationships rarely change.	Parents change frequently.
Child reads repeatedly need the parent key.	A parent has many children to repair.
Fan-out updates can be made correct.	The gain actually comes from an unmatched index.

What it costs

Reads	May remove a relationship hop.
Writes	Parent changes fan out to children.
Storage	A copied field per child.
Correctness	Copied keys must agree with their authoritative parent.
Operations	Backfill and repair copied information.

In practice

Follow the matching study case for tested variations and registered results. A plausible boundary is not a measured crossover.

The checklist

Question	Answer	Evidence
What is the invariant?	Each copied key matches its owner.	mechanism
Where does the information live?	Parent fact and child copies.	mechanism
Where is the contention?	Children touched by a parent change.	mechanism
What work moved from reads to writes?	Resolving the parent relationship.	mechanism
What state is derived?	Copied parent information.	mechanism
What can become stale?	Children skipped during maintenance.	mechanism
Under which workload does it stop working?	Large fan-out with frequent parent changes.	gap
Which experiment would prove it?	Hold SQL and indexes fixed; vary the copied key.	gap

Rollup

STRATEGY · ROLLUP

Key idea

Store an aggregate on its parent so reads skip the children and writes maintain the answer.

The problem

A charity dashboard repeatedly sums donations. A stored total replaces that repeated scan with a parent lookup.

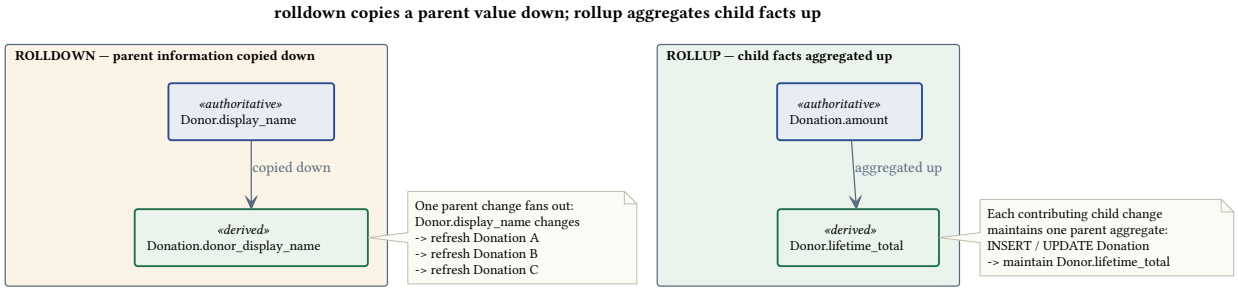


Figure 5: **Rollup: mechanism and trade-off.**

Form: structure · Evidence: conceptual illustration

> **Text equivalent.** Rollup maintains a parent aggregate when children change. Rolldown maintains child copies when parent information changes.

How it works

- Keep child rows as the facts of record.
- Update the total with each contributing mutation.
- Choose trigger or application maintenance deliberately.

Thrive or die: the expected boundary

It fits when	It does not fit when
Totals are read much more often than changed.	Many writers collide on one parent total.
The aggregate definition is stable.	Aggregate rules change frequently.
Maintenance can be atomic with the source change.	Historical answers still require scanning the facts.

What it costs

Reads	Fetch a maintained total.
Writes	Update shared parent state.
Storage	Stored totals and supporting indexes.
Correctness	Every insert, correction and deletion must maintain totals.
Operations	Rebuild or reconcile when rules change.

In practice

Follow the matching study case for tested variations and registered results. A plausible boundary is not a measured crossover.

The checklist

Question	Answer	Evidence
What is the invariant?	The total equals its contributing facts.	mechanism
Where does the information live?	Child facts and parent aggregate.	mechanism
Where is the contention?	Shared aggregate row.	mechanism
What work moved from reads to writes?	Repeated aggregation.	mechanism
What state is derived?	Totals and other maintained summaries.	mechanism
What can become stale?	An aggregate with a missed mutation.	mechanism
Under which workload does it stop working?	Write-heavy hot parents or large replacement operations.	gap
Which experiment would prove it?	Compare maintainers and sweep write size and read/write mix.	gap

Embedding

STRATEGY · EMBEDDING

Key idea

Store bounded child information with its parent when the application reads and changes that unit together.

The problem

A product can read a configuration as one document. A growing donation history may require independent child access instead.

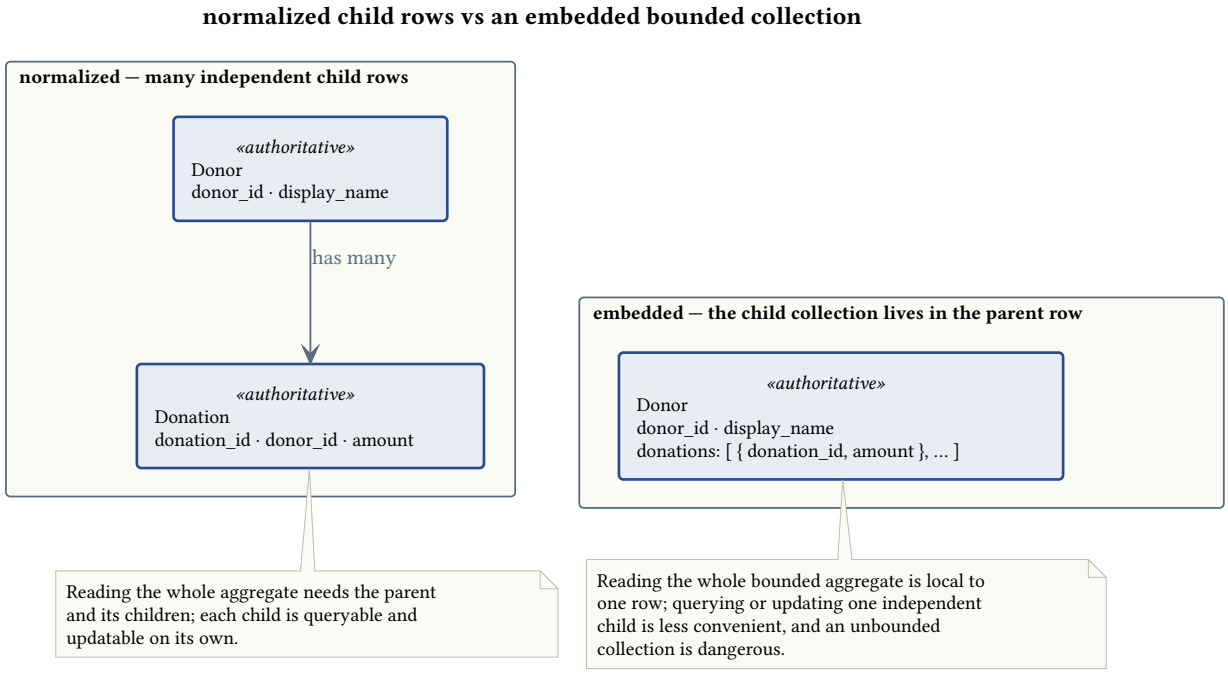


Figure 6: **Embedding: mechanism and trade-off.**

Form: structure · Evidence: conceptual illustration

> **Text equivalent.** Normalized children are independently addressable rows. Embedding places a bounded child collection inside the parent's update unit.

How it works

- Choose the atomic document boundary.
- Read or rewrite the embedded unit.
- Bound growth and define concurrent update behavior.

Thrive or die: the expected boundary

It fits when	It does not fit when
Parent and children are read together.	Children grow without a useful bound.
The collection has a practical size bound.	Small child edits rewrite a large unit.
Whole-unit replacement matches the write pattern.	Writers contend on the same parent document.

What it costs

Reads	One unit can answer a combined read.
Writes	May rewrite or lock a larger unit.
Storage	Representation-dependent duplication and document overhead.
Correctness	Concurrent edits must not overwrite one another.
Operations	Set size limits and migration rules.

In practice

Follow the matching study case for tested variations and registered results. A plausible boundary is not a measured crossover.

The checklist

Question	Answer	Evidence
What is the invariant?	The embedded unit represents its child facts correctly.	mechanism
Where does the information live?	Child information inside the parent.	mechanism
Where is the contention?	The parent document or row.	mechanism
What work moved from reads to writes?	Assembling the child collection.	mechanism
What state is derived?	Only explicitly copied or summarized fields.	mechanism
What can become stale?	Copies outside the authoritative document.	mechanism
Under which workload does it stop working?	Large units, independent child reads or concurrent small edits.	gap
Which experiment would prove it?	Compare bounded sizes and matched read/write contracts.	gap

Database SELECTs

Two SQL expressions are comparable only if they return the same logical answer. Establish grain, duplicates, NULLs, ties and ordering first.

Strategies: The answer contract · Joins and filter placement · CTE folding and materialization · Membership and absence · Aggregation and per-parent top-N · Pagination and access paths.

The answer contract

STRATEGY · THE ANSWER CONTRACT

Key idea

Define the answer before choosing the SQL spelling.

The problem

A charity needs either one row per donor or one row per donation. Joining children changes row multiplicity unless the query restores the intended grain.



Figure 7: The answer contract: the operation path.

Form: sequence · Evidence: conceptual illustration

Text equivalent. Business question → Answer contract → Oracle-checked result. Arrows show logical order, not a measured duration.

How it works

- Name the output grain.
- Specify duplicates, NULLs, ties and order.
- Check answers against an independent oracle.

A small example

Business question	Required grain
Who donated?	One row per qualifying donor.
Which donations arrived?	One row per donation, including repeated donors.
Who has not donated?	One row per donor with no matching donation.

A join can multiply donor rows. DISTINCT repairs only the intended grain; it must not erase required donation rows.

Thrive or die: the expected boundary

It fits when	It does not fit when
A query has an explicit business question.	An inner join silently drops required parents.
Edge cases are part of verification.	Deduplication removes meaningful repeated facts.
Alternative spellings preserve the contract.	A faster query answers a narrower question.

What it costs

Reads	Work required by the answer contract.
Writes	None unless a maintained answer is introduced.
Storage	Intermediate rows or maintained results.
Correctness	Validate all alternatives against the same oracle.
Operations	Keep wrong-answer controls.

In practice

Follow the matching study case for tested variations and registered results. A plausible boundary is not a measured crossover.

The checklist

Question	Answer	Evidence
What is the invariant?	Same logical answer across alternatives.	mechanism
Where does the information live?	Base facts and query results.	mechanism
Where is the contention?	Execution resources; not a new storage invariant.	mechanism
What work moved from reads to writes?	None for plain query spelling.	mechanism
What state is derived?	The returned relation.	mechanism
What can become stale?	Source snapshots or external copies.	mechanism
Under which workload does it stop working?	Unspecified grain, multiplicity, NULLs, ties or order.	gap
Which experiment would prove it?	Compare edge-case outputs before timing.	gap

Joins and filter placement

STRATEGY · JOINS AND FILTER PLACEMENT

Key idea

Equivalent inner-join spellings often share a plan; moving an outer-join filter can change the answer.

The problem

A customer page needs each order above 100 together with its customer. Does writing the filter before the join reduce work?



Figure 8: Query spelling and execution.

Form: sequence · Evidence: conceptual illustration

Text equivalent. Different SQL expressions can become the same scan and join. This is a conceptual optimization path, not proof of a particular engine plan.

How it works

- Fix the returned columns and row multiplicities.
- Write the relationship and amount predicate explicitly.
- Inspect the actual plan; SQL layout does not prescribe execution order.

A small example

Illustrative tables: `customers(id, name)` and `orders(id, customer_id, amount)`. Ana (id 1) has orders 11 and 14 worth 200 each. Ben (id 2) has an order worth 80 and one with NULL amount; Cara (id 3) has none.

All six forms return `(Ana, 11, 200)` and `(Ana, 14, 200)`. These are example values, not measurements. Without ORDER BY, row order is unspecified.

1. Explicit join, WHERE filter

```
SELECT c.name, o.id AS order_id, o.amount
FROM customers c
JOIN orders o ON o.customer_id = c.id
WHERE o.amount > 100;
```

2. Filter inside ON

```
SELECT c.name, o.id AS order_id, o.amount
FROM customers c JOIN orders o
  ON o.customer_id = c.id AND o.amount > 100;
```

3. Filtered derived table

```
SELECT c.name, o.id AS order_id, o.amount
FROM customers c JOIN (
  SELECT id, customer_id, amount FROM orders
  WHERE amount > 100
) o ON o.customer_id = c.id;
```

4. CTE naming the filtered orders

```
WITH expensive_orders AS (
  SELECT id, customer_id, amount FROM orders
  WHERE amount > 100
)
SELECT c.name, o.id AS order_id, o.amount
FROM customers c
JOIN expensive_orders o ON o.customer_id = c.id;
```

5. CTE naming the joined rows

```
WITH customer_orders AS (
  SELECT c.name, o.id AS order_id, o.amount
  FROM customers c
  JOIN orders o ON o.customer_id = c.id
)
SELECT name, order_id, amount
FROM customer_orders WHERE amount > 100;
```

6. Comma join, both predicates in WHERE

```
SELECT c.name, o.id AS order_id, o.amount
FROM customers c, orders o
WHERE o.customer_id = c.id AND o.amount > 100;
```

Expected performance: the direct join, ON filter and comma join often converge to the same plan. Simple derived tables and folded CTEs can converge too. A plan match is a hypothesis to test, not a guarantee of identical end-to-end timing.

When does a derived table help? A derived table that only filters is merged into the join, so it is a spelling, not an optimization. It becomes a separate step only when it groups, deduplicates, limits or ranks rows. Then it can help or hurt, and it may answer a different question.

A different answer with LEFT JOIN. The first expression keeps Ben and Cara with NULL order columns. The second drops them:

Preserve every customer

```
SELECT c.name, o.id AS order_id, o.amount
FROM customers c LEFT JOIN orders o
  ON o.customer_id = c.id AND o.amount > 100;
```

Keep only qualifying customer-order matches

```
SELECT c.name, o.id AS order_id, o.amount
FROM customers c LEFT JOIN orders o
  ON o.customer_id = c.id
WHERE o.amount > 100;
```

Filtering the child input before LEFT JOIN is equivalent to the first form. EXISTS instead returns Ana once when the question asks only who has a qualifying order; it cannot replace required order details.

Documentation: PostgreSQL 17 joins and planner transformations. Other engines and versions require their own plan checks.

Thrive or die: the expected boundary

It fits when	It does not fit when
A simple inner join preserves the required answer.	A child WHERE predicate discards required outer-join parents.
A readable spelling makes the relationship clear.	A subquery changes duplicates, aggregation or limit semantics.
The optimizer can move the filter into an input scan.	Written filter order is treated as a timing guarantee.

What it costs

Reads	The chosen scan and join, not the position of WHERE.
Writes	No extra maintained state.
Storage	Plan-dependent join and intermediate-row memory.
Correctness	Preserve duplicates and required unmatched parents.
Operations	Compare EXPLAIN ANALYZE and replicated timings with matched inputs.

In practice

Follow the matching study case for tested variations and registered results. A plausible boundary is not a measured crossover.

The checklist

Question	Answer	Evidence
What is the invariant?	Return the same bag of customer-order rows.	mechanism
Where does the information live?	Customer and order base tables.	mechanism
Where is the contention?	Scan, join and client resources.	mechanism
What work moved from reads to writes?	None for these query forms.	mechanism
What state is derived?	Transient joined rows.	mechanism
What can become stale?	Results reflect the selected database snapshot.	mechanism
Under which workload does it stop working?	A changed answer or an unfavorable scan/join plan.	gap
Which experiment would prove it?	Verify bags, sweep selectivity and compare plans across replicated runs.	gap

CTE folding and materialization

STRATEGY · CTE FOLDING AND MATERIALIZATION

Key idea

A CTE names intermediate work; folding permits joint optimization, while materialization can reuse work and block predicate pushdown.

The problem

The same customer-order query can scan few qualifying rows or create a large intermediate result. Reusing expensive work poses a different tradeoff.



Figure 9: The CTE execution choice.

Form: sequence · Evidence: conceptual illustration

Text equivalent. Folding combines the CTE with its parent for optimization. Materialization evaluates a separate result that consumers can reuse. No branch is universally faster.

How it works

- Check the pinned engine's CTE syntax and default behavior.
- Keep one-use selective access separate from repeated expensive reuse.
- Verify folding or materialization in the actual plan.
- Measure saved computation against intermediate rows, memory and spills.

A small example

Format	Expected behavior on PostgreSQL 17
`WITH q AS (...)`	A nonrecursive, side-effect-free, single-use CTE normally folds; multiple references normally materialize.
`WITH q AS MATERIALIZED (...)`	Force separate CTE evaluation; outer predicates cannot be pushed inside it.
`WITH q AS NOT MATERIALIZED (...)`	Permit eligible CTE folding; several references may repeat computation.

Selective one-use example: add MATERIALIZED to form 5. The amount predicate stays outside the CTE, so it cannot reduce the joined rows inside. Add NOT MATERIALIZED to allow folding when the CTE is eligible.

Repeated-use example: several consumers need an expensive deterministic calculation over the same input. Materialization may compute it once; folding may compute it for each consumer. Selective consumers can still favor folding by avoiding most input rows.

These are predictions. Compare default, MATERIALIZED and NOT MATERIALIZED with identical answers and indexes. Record unsupported syntax or different defaults on the pinned YugabyteDB engine explicitly.

Documentation: PostgreSQL 17 CTE folding and materialization. This feature predates PostgreSQL 17; engine support is not established by a version-number guess.

Thrive or die: the expected boundary

It fits when	It does not fit when
Folding lets a selective outer filter reach an indexed base scan.	A one-use materialized result includes mostly discarded rows.
Materialization avoids repeatedly evaluating an expensive input.	Separate evaluation blocks a useful outer predicate.
The reused result fits the available execution memory.	Forced folding repeats expensive work across many references.

What it costs

Reads	Filter pushdown versus shared or repeated computation.
Writes	No persisted rollup or refresh job for a SELECT CTE.
Storage	Transient result memory and possible temporary-file spills.
Correctness	Both policies must preserve the same answer and snapshot.
Operations	Inspect engine support, CTE scans, loops and temp/RPC counters.

In practice

Follow the matching study case for tested variations and registered results. A plausible boundary is not a measured crossover.

The checklist

Question	Answer	Evidence
What is the invariant?	Same values and multiplicities under either policy.	mechanism
Where does the information live?	Base facts and a transient CTE result.	mechanism
Where is the contention?	Repeated CPU work or intermediate memory/I/O.	mechanism
What work moved from reads to writes?	None; materialization here is per query, not a stored read model.	mechanism
What state is derived?	The CTE output.	mechanism
What can become stale?	Only the query snapshot; there is no later refresh lifecycle.	mechanism
Under which workload does it stop working?	Selective one-use access or repeated reuse with the wrong policy.	gap
Which experiment would prove it?	Sweep selectivity and reuse separately with oracle checks and actual plan evidence.	gap

Membership and absence

STRATEGY · MEMBERSHIP AND ABSENCE

Key idea

Use a membership test when the question asks whether a related row exists.

The problem

A charity asks for donors with a donation, not every donor-donation combination. Absence queries must also define NULL behavior.



Figure 10: Membership and absence: the operation path.

Form: sequence · Evidence: conceptual illustration

Text equivalent. Parent candidate → Related-row test → Keep or reject parent. Arrows show logical order, not a measured duration.

How it works

- Use existence semantics for a yes/no relationship.
- Check join duplication before adding DISTINCT.
- Test absence with nullable inputs.

A small example

Illustrative SQL: these generic table names show the contract, not a benchmark implementation.

```sql
SELECT p.id
FROM parent AS p
WHERE EXISTS (
  SELECT 1 FROM child AS c
  WHERE c.parent_id = p.id
);
```

Use NOT EXISTS for absence. A nullable NOT IN input can make a predicate unknown instead of true.

Thrive or die: the expected boundary

It fits when	It does not fit when
Only membership or absence is needed.	The output requires child details.
Child fan-out is high.	NOT IN receives NULL without an explicit contract.
NULL handling is explicit.	An equivalent spelling is assumed faster everywhere.

What it costs

Reads	Plan-dependent probes or scans.
Writes	No extra maintained state.
Storage	Transient plan structures.
Correctness	Membership must preserve NULL and duplicate semantics.
Operations	Inspect plans under measured fan-out regimes.

In practice

Follow the matching study case for tested variations and registered results. A plausible boundary is not a measured crossover.

The checklist

Question	Answer	Evidence
What is the invariant?	Return exactly the required parent membership.	mechanism
Where does the information live?	Parent and child rows.	mechanism
Where is the contention?	Read execution resources.	mechanism
What work moved from reads to writes?	None for equivalent SQL spellings.	mechanism
What state is derived?	Membership answer.	mechanism
What can become stale?	The selected snapshot can differ from later writes.	mechanism
Under which workload does it stop working?	Wrong NULL behavior or changed output grain.	gap
Which experiment would prove it?	Sweep fan-out with matched answers and negative controls.	gap

Aggregation and per-parent top-N

STRATEGY · AGGREGATION AND PER-PARENT TOP-N

Key idea

Choose between repeated probes and shared processing according to the requested parent set.

The problem

A page requests a total or latest donation for each donor. Processing all donations may waste work when the page asks for few donors.



Figure 11: Aggregation and per-parent top-N: the operation path.

Form: sequence · Evidence: conceptual illustration

Text equivalent. Requested parents → Probe or process together → Totals or top-N. Arrows show logical order, not a measured duration.

How it works

- Define empty-parent and tie behavior.
- Compare a per-parent probe with a shared aggregate.
- Vary the requested parent set while holding facts fixed.

A small example

Strategy	Work it performs
Correlated aggregate	Compute the child answer separately for each requested parent.
Pre-aggregate and join	Group children, then join those summaries to requested parents.
LATERAL / ordered probe	Use a matching order and limit for each parent's requested top rows.

COUNT of an empty set yields zero. A missing joined summary needs an explicit zero/default rule; top-N ties need their own contract.

Thrive or die: the expected boundary

It fits when	It does not fit when
A narrow request has a useful lookup path.	Every parent causes an expensive repeated scan.
Shared processing amortizes across many requested parents.	Pre-aggregation processes mostly unrequested data.
The selected plan matches the request shape.	A top-one query silently discards tied answers.

What it costs

Reads	Per-parent probes or shared processing.
Writes	No extra writes for read-time aggregation.
Storage	Intermediate aggregates, sorting or ranking.
Correctness	Empty groups and ties need an explicit rule.
Operations	Measure several regimes and query plans.

In practice

Follow the matching study case for tested variations and registered results. A plausible boundary is not a measured crossover.

The checklist

Question	Answer	Evidence
What is the invariant?	Aggregate and top-N answers preserve empty groups and ties.	mechanism
Where does the information live?	Child facts plus transient processing.	mechanism
Where is the contention?	Query execution resources.	mechanism
What work moved from reads to writes?	None until a stored rollup is introduced.	mechanism
What state is derived?	Read-time summaries and ranks.	mechanism
What can become stale?	Maintained aggregates, if added separately.	mechanism
Under which workload does it stop working?	An unfavorable requested-parent count or access path.	gap
Which experiment would prove it?	Sweep parent count with oracle checks and shuffled measurement order.	gap

Pagination and access paths

STRATEGY · PAGINATION AND ACCESS PATHS

Key idea

A cursor can continue from an ordered key; OFFSET skips through an ordered prefix.

The problem

A donation feed needs the next page. Deep page numbers and a stable continuation cursor impose different navigation contracts.



Figure 12: Pagination and access paths: the operation path.

Form: sequence · Evidence: conceptual illustration

Text equivalent. Ordered rows → Last-key cursor → Next matching page. Arrows show logical order, not a measured duration.

How it works

- Define a total order with a tie-breaker.
- Use a cursor carrying the last ordering keys.
- Compare equivalent pages at several depths.

A small example

Illustrative ascending order: the cursor holds both ordering keys.

```
WHERE (created_at, id) > (:last_time, :last_id)
ORDER BY created_at, id
LIMIT :page_size
```

The id resolves timestamp ties. OFFSET supports page-number navigation; this cursor supports continuation from a known key. A flat cost per page needs an index that matches the whole order.

Access paths for a disjunction. One OR over an indexed column can use that index once. Rewriting it as UNION ALL is safe only when the branches cannot overlap. A predicate written as an expression needs an index on that same expression.

Thrive or die: the expected boundary

It fits when	It does not fit when
Navigation continues from the previous page.	Users need arbitrary page-number jumps.
Ordering keys have a matching access path.	Order lacks a stable tie-breaker.
Cursor semantics match concurrent change behavior.	A shallow-page result is extrapolated to all depths.

What it costs

Reads	Continue after a key or scan and discard a prefix.
Writes	Normal index maintenance.
Storage	Cursor state plus access-path entries.
Correctness	Define behavior when rows appear, move or disappear.
Operations	Version cursor formats and bound page sizes.

In practice

Follow the matching study case for tested variations and registered results. A plausible boundary is not a measured crossover.

The checklist

Question	Answer	Evidence
What is the invariant?	Return the correct next page under the chosen order.	mechanism
Where does the information live?	Rows, index and cursor keys.	mechanism
Where is the contention?	Query execution resources.	mechanism
What work moved from reads to writes?	Only ordinary index maintenance.	mechanism
What state is derived?	Continuation cursor.	mechanism
What can become stale?	A cursor can refer to rows changed since its creation.	mechanism
Under which workload does it stop working?	Unsupported jumps or an unfavorable engine/depth regime.	gap
Which experiment would prove it?	Verify identical pages and sweep depth by engine.	gap

Concurrent writes and time

Start with the business invariant: no lost increment, no oversold seat, or no stale owner publishing a result. A faster invalid design is a failed design.

Strategies: Hot rows · Optimistic version checks · Pessimistic locking · History and current state · Expiry and clock authority.

Hot rows

STRATEGY · HOT ROWS

Key idea

A shared mutable row concentrates otherwise independent writers into one coordination point.

The problem

Many donations may update one charity total. More workers do not remove the need to order updates to that row.



Figure 13: Hot rows: the operation path.

Form: sequence · Evidence: conceptual illustration

Text equivalent. Many writers → One mutable key → Ordered successful changes. Arrows show logical order, not a measured duration.

How it works

- Identify which key every operation changes.
- Separate useful parallel work from serialized work.
- Measure correctness, conflicts and tail behavior together.

Thrive or die: the expected boundary

It fits when	It does not fit when
Contention is low or bounded.	Writers overwhelmingly target one key.
The shared row is the intended arbitration point.	Retries amplify already overloaded work.
Rejected work is handled explicitly.	Throughput is reported without invariant checks.

What it costs

Reads	May wait behind writers.
Writes	Serialized updates or conflicting attempts.
Storage	Little extra state by itself.
Correctness	Acknowledged changes must survive.
Operations	Track queueing, rejection and conflict behavior.

In practice

Follow the matching study case for tested variations and registered results. A plausible boundary is not a measured crossover.

The checklist

Question	Answer	Evidence
What is the invariant?	Acknowledged changes preserve the business invariant.	mechanism
Where does the information live?	Shared mutable rows.	mechanism
Where is the contention?	The popular key or row.	mechanism
What work moved from reads to writes?	No automatic read/write relocation.	mechanism
What state is derived?	Counters or summaries when present.	mechanism
What can become stale?	Cached copies and abandoned read decisions.	mechanism
Under which workload does it stop working?	Concentrated offered work exceeds serialized capacity.	gap
Which experiment would prove it?	Sweep skew and writers with a violating control.	gap

Optimistic version checks

STRATEGY · OPTIMISTIC VERSION CHECKS

Key idea

Read a version, then commit only if that version is still current.

The problem

Two administrators edit the same configuration. A conditional update detects whether another writer changed it first.



Figure 14: Optimistic writes: validate before acceptance.

Form: sequence · Evidence: conceptual illustration

Text equivalent. A writer reads a version and attempts an update conditional on that version. An intervening change makes the update fail; the writer must re-read before retrying.

How it works

- Read the state and its version.
- Write using a version condition.
- On conflict, retry from fresh state within a defined budget.

Thrive or die: the expected boundary

It fits when	It does not fit when
Conflicts are uncommon.	A hot key creates frequent conflicts.
Attempts are cheap to repeat.	Side effects occur before the conditional commit.
Bounded rejection is acceptable.	Retries repeat expensive work without a bound.

What it costs

Reads	Read state and version.
Writes	Conditional writes; conflicts may require another attempt.
Storage	A version token.
Correctness	Never accept an update based on an obsolete version.
Operations	Bound retries and define client-visible rejection.

In practice

Follow the matching study case for tested variations and registered results. A plausible boundary is not a measured crossover.

The checklist

Question	Answer	Evidence
What is the invariant?	No acknowledged update overwrites an intervening change.	mechanism
Where does the information live?	Source row plus version.	mechanism
Where is the contention?	The version-checked hot key.	mechanism
What work moved from reads to writes?	None inherently.	mechanism
What state is derived?	Version token.	mechanism
What can become stale?	The state read before a conflicting commit.	mechanism
Under which workload does it stop working?	Frequent conflicts or expensive repeated attempts.	gap
Which experiment would prove it?	Match the invariant and sweep concurrent writers.	gap

Pessimistic locking

STRATEGY · PESSIMISTIC LOCKING

Key idea

Acquire the arbitration lock before reading state used to decide an update.

The problem

A seat reservation reads availability and changes it. Holding the lock keeps another writer from making the same decision concurrently.



Figure 15: Pessimistic locking: the operation path.

Form: sequence · Evidence: conceptual illustration

Text equivalent. Acquire row lock → Read and decide → Write and release. Arrows show logical order, not a measured duration.

How it works

- Lock the row before deciding.
- Keep protected work short.
- Commit or roll back to release the lock.

Thrive or die: the expected boundary

It fits when	It does not fit when
Conflicts would otherwise be frequent.	External work happens while holding the lock.
Protected operations are short.	Lock order causes deadlocks.
Waiting has a bounded, visible policy.	Clients wait indefinitely on a busy row.

What it costs

Reads	Protected reads may wait.
Writes	Queued updates instead of speculative conflicts.
Storage	Lock-management state.
Correctness	Read, decision and write must share protection.
Operations	Set timeouts and consistent lock ordering.

In practice

Follow the matching study case for tested variations and registered results. A plausible boundary is not a measured crossover.

The checklist

Question	Answer	Evidence
What is the invariant?	The protected decision observes state that cannot change underneath it.	mechanism
Where does the information live?	Authoritative rows and lock state.	mechanism
Where is the contention?	The locked row and waiting writers.	mechanism
What work moved from reads to writes?	No inherent relocation; work is ordered.	mechanism
What state is derived?	None inherently.	mechanism
What can become stale?	Copies read outside the protected transaction.	mechanism
Under which workload does it stop working?	Long critical sections or overload.	gap
Which experiment would prove it?	Compare version checks and locks under one invariant.	gap

History and current state

STRATEGY · HISTORY AND CURRENT STATE

Key idea

An event history records what happened; current state records the answer needed now.

The problem

A booking service must answer whether a seat is free and explain refunds. A log alone need not make the current answer cheap.

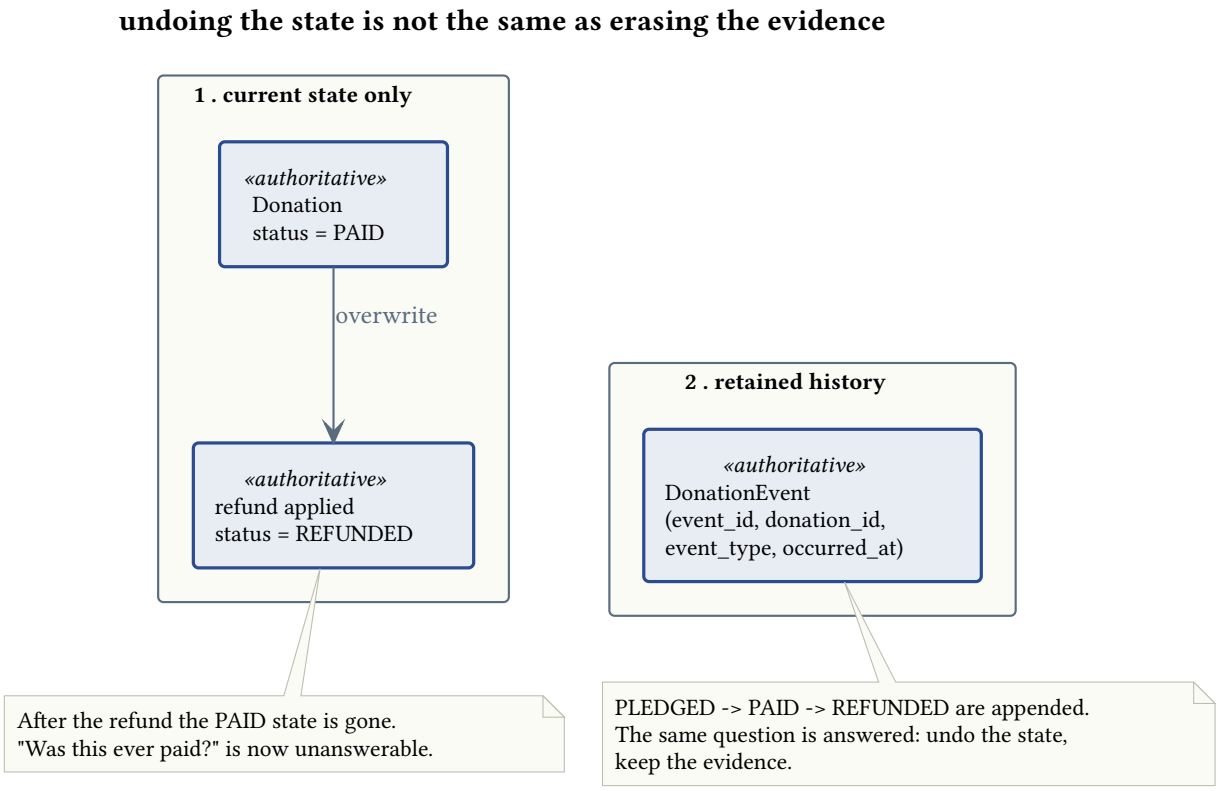


Figure 16: **History and current state: mechanism and trade-off.**

Form: structure · Evidence: conceptual illustration

> **Text equivalent.** Current state answers availability now. Retained events preserve past sales and refunds that current-state deletion would destroy.

How it works

- Keep immutable events for the historical contract.
- Maintain or derive a current-state projection.
- Decide how events and projections commit together.

Thrive or die: the expected boundary

It fits when	It does not fit when
Audit or reconstruction is required.	Every hot read replays a growing log.
Current reads have a maintained projection.	An event append is mistaken for concurrency protection.
Projection maintenance has a clear ownership rule.	History and projection can diverge silently.

What it costs

Reads	Read a projection or replay relevant events.
Writes	Append events and maintain any current projection.
Storage	Retained history plus current state.
Correctness	Events and acknowledged current state must reconcile.
Operations	Version events and support replay or repair.

In practice

Follow the matching study case for tested variations and registered results. A plausible boundary is not a measured crossover.

The checklist

Question	Answer	Evidence
What is the invariant?	Current state agrees with accepted historical changes.	mechanism
Where does the information live?	Event history and optional projection.	mechanism
Where is the contention?	Append coordination and shared current-state rows.	mechanism
What work moved from reads to writes?	Projection maintenance replaces replay on every read.	mechanism
What state is derived?	Current-state projection.	mechanism
What can become stale?	A projection that falls behind or misses an event.	mechanism
Under which workload does it stop working?	Growing replay or an incorrectly maintained projection.	gap
Which experiment would prove it?	Match concurrency strategy; compare replay and projection cost.	gap

Expiry and clock authority

STRATEGY · EXPIRY AND CLOCK AUTHORITY

Key idea

A deadline has meaning only when one authority decides whether it has passed.

The problem

A seat hold expires while a client attempts confirmation. The service needs one atomic decision about deadline and ownership.

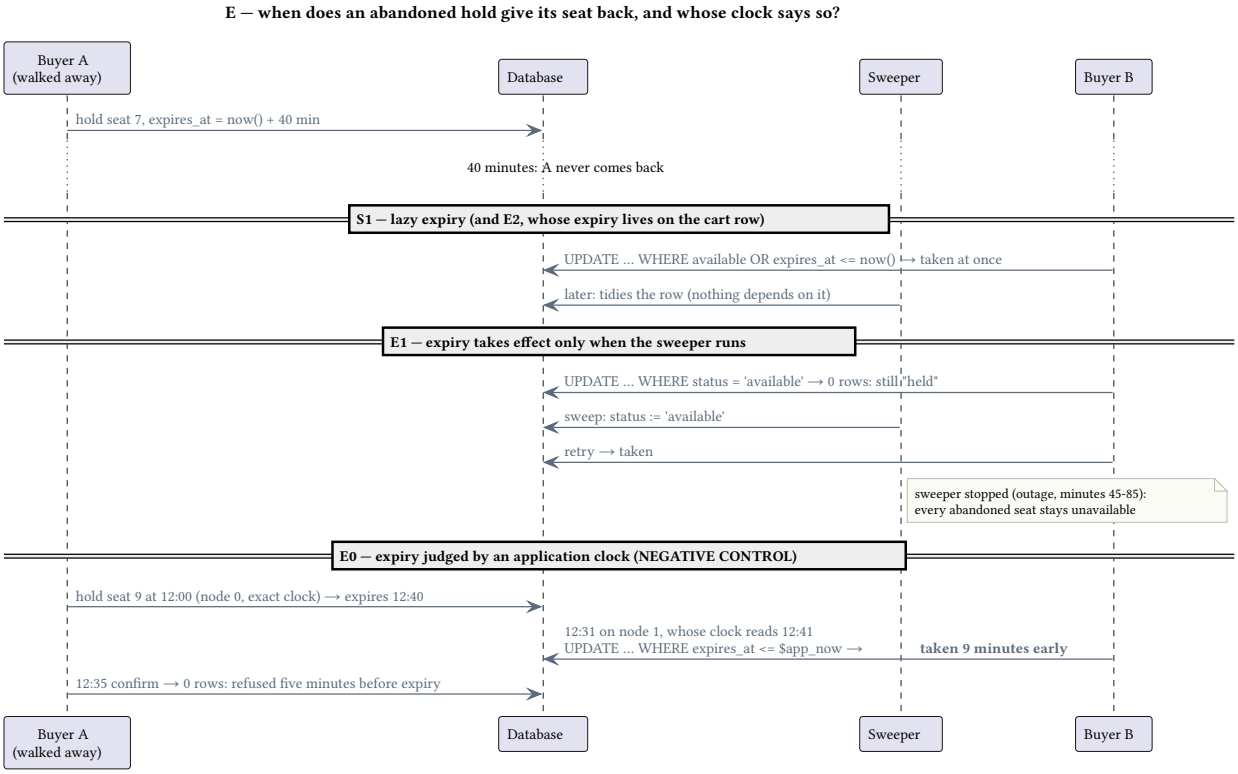


Figure 17: **Expiry and clock authority: mechanism and trade-off.**

Form: sequence · Evidence: conceptual illustration

> **Text equivalent.** A database-time guard can reclaim an expired hold without waiting for cleanup. A sweeper-only design delays availability; a skewed client clock can reclaim too early.

How it works

- Choose the authoritative clock.
- Check expiry inside the arbitration operation.
- Separate expiration semantics from cleanup timing.

Thrive or die: the expected boundary

It fits when	It does not fit when
Deadline checks use a defined authority.	Clients decide expiry using unrelated clocks.
Confirmation and expiration share arbitration.	Cleanup is the only protection against late confirmation.
Cleanup delay does not extend authorization.	A lease is assumed to prevent stale publication.

What it costs

Reads	Check ownership and authoritative time.
Writes	Arbitrate confirmations and expiry transitions.
Storage	Deadline and ownership token.
Correctness	No success after the authority rejects the deadline.
Operations	Monitor delayed cleanup and clock assumptions.

In practice

Follow the matching study case for tested variations and registered results. A plausible boundary is not a measured crossover.

The checklist

Question	Answer	Evidence
What is the invariant?	A hold can be confirmed only while valid for its owner.	mechanism
Where does the information live?	Hold row, deadline and owner token.	mechanism
Where is the contention?	The hold's arbitration record.	mechanism
What work moved from reads to writes?	No inherent relocation.	mechanism
What state is derived?	Availability projection or cache, if present.	mechanism
What can become stale?	Client beliefs and cached availability.	mechanism
Under which workload does it stop working?	Undefined clock authority or non-atomic transitions.	gap
Which experiment would prove it?	Race confirmation against expiry with an invalid control.	gap

Caches

A cache is another copy of an answer. Its read benefit and its freshness contract are separate questions.

source correctness belongs to the mutation; copy correctness belongs to the publication

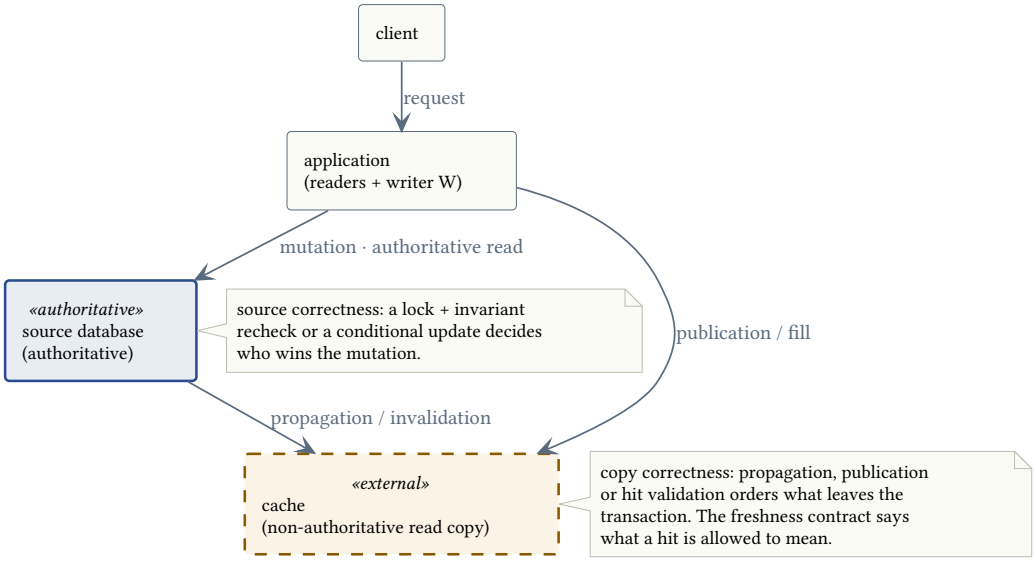


Figure 18: **Caches: the choices at a glance.**

Form: structure · Evidence: conceptual illustration

Text equivalent. The picture maps the mechanisms. Layout and arrows do not represent measured throughput.

Strategies: Cache-aside and freshness · Invalidation and stale fills · Leases and fencing.

Cache-aside and freshness

STRATEGY · CACHE-ASIDE AND FRESHNESS

Key idea

Read the copy when usable; otherwise load the source under an explicit freshness contract.

The problem

A charity page reuses a donor profile. If a write completes, must the very next read see it, or may the copy lag?

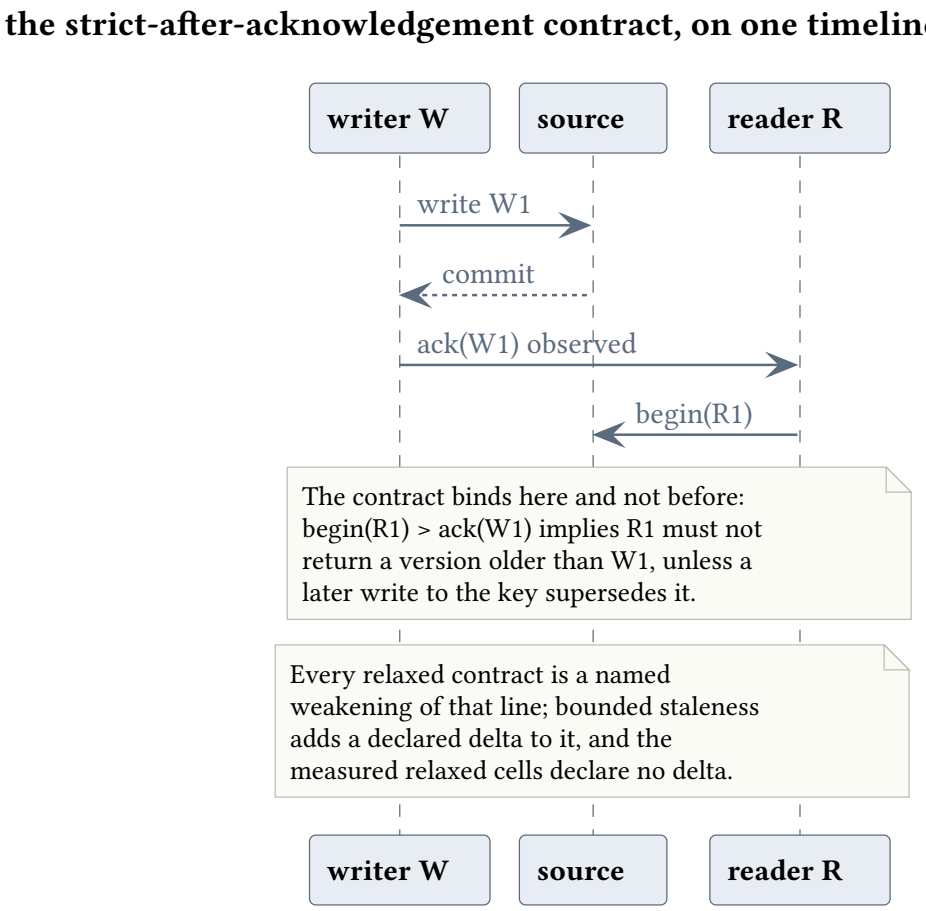


Figure 19: **Cache-aside and freshness: mechanism and trade-off.**

Form: sequence · Evidence: conceptual illustration

> **Text equivalent.** A read beginning after an acknowledged write must not return a version older than that write under the strict contract. Relaxed contracts weaken this boundary explicitly.

How it works

- Define what counts as an acceptable stale answer.
- Load misses from the authoritative source.
- Bound copy lifetime and record freshness violations.

Thrive or die: the expected boundary

It fits when	It does not fit when
Repeated reads reuse the same answers.	The required contract is immediate freshness.
A defined staleness window is acceptable.	Keys rarely repeat.
Hit behavior is measured alongside mutation behavior.	Hit throughput hides post-write stale reads.

What it costs

Reads	Hits avoid source work; misses still load it.
Writes	Maintain the source and any copy protocol.
Storage	Additional cached values.
Correctness	Freshness is a separate contract from source correctness.
Operations	Bound memory, expiry and stampede behavior.

In practice

Follow the matching study case for tested variations and registered results. A plausible boundary is not a measured crossover.

The checklist

Question	Answer	Evidence
What is the invariant?	Returned copies satisfy the declared freshness contract.	mechanism
Where does the information live?	Authoritative source plus cache.	mechanism
Where is the contention?	Popular cache keys and source misses.	mechanism
What work moved from reads to writes?	Copy population and maintenance.	mechanism
What state is derived?	Cached answer.	mechanism
What can become stale?	The cache after source changes.	mechanism
Under which workload does it stop working?	Low reuse or a stricter freshness requirement.	gap
Which experiment would prove it?	Measure stale-after-ack reads as well as throughput.	gap

Invalidation and stale fills

STRATEGY · INVALIDATION AND STALE FILLS

Key idea

Deleting a cache key does not order a reader that loaded old source data before the write.

The problem

A reader starts a miss load. A writer commits and invalidates. The old reader then fills the cache with its earlier value.

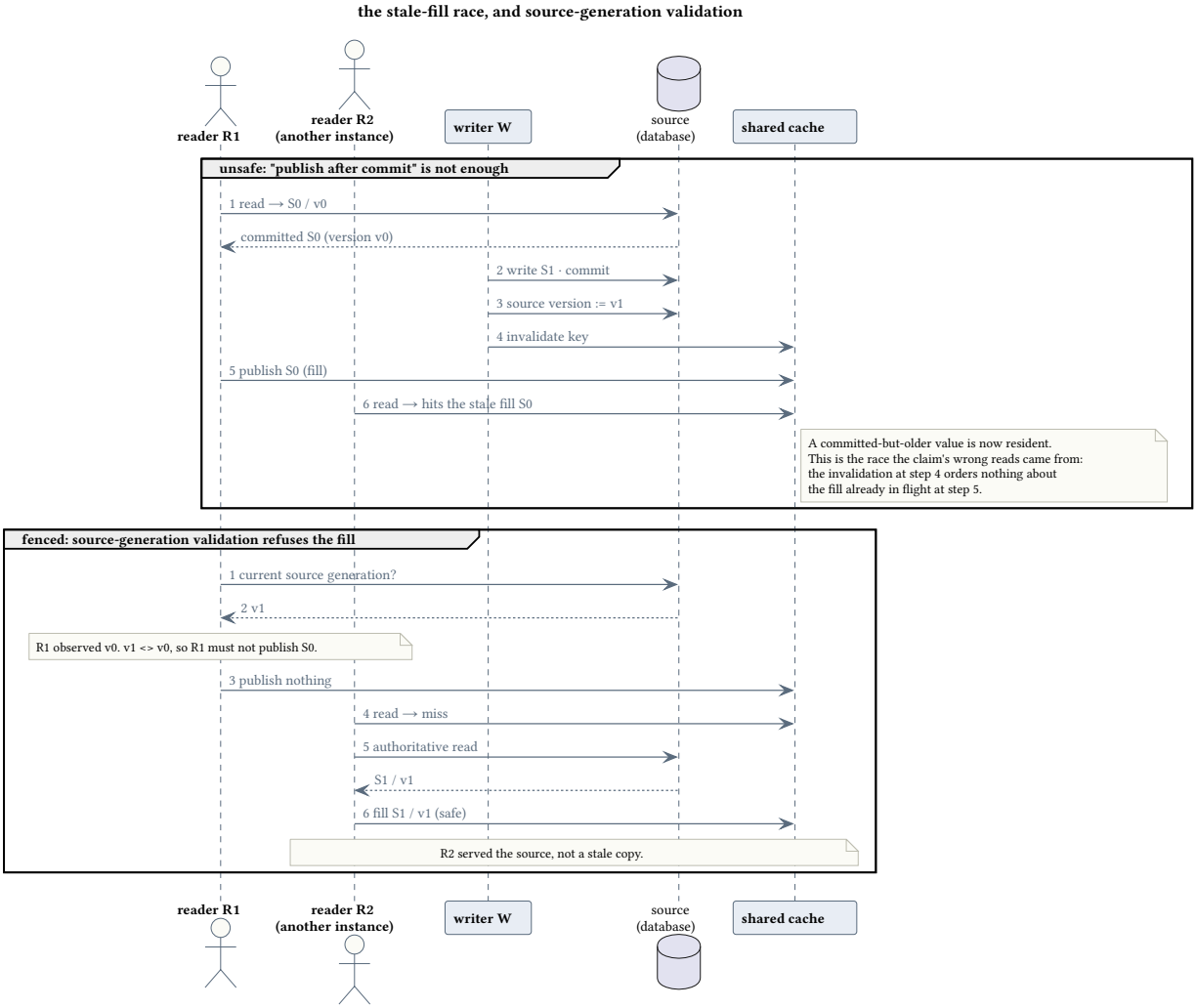


Figure 20: **Invalidation and stale fills: mechanism and trade-off.**

Form: sequence · Evidence: conceptual illustration

> **Text equivalent.** An old source load can publish after a writer invalidates the key. Coordinated source-generation validation must prevent publication that violates the chosen freshness boundary.

How it works

- Identify the fill that can outlive invalidation.
- Treat miss loading as part of the write/copy protocol.
- Validate the race with an adversarial schedule.

Thrive or die: the expected boundary

It fits when	It does not fit when
The contract tolerates the uncovered race.	Delete-after-write is claimed to guarantee freshness.
A version or shared protocol rejects old fills.	A fill can publish after ownership changes.
Every writer participates.	Negative controls never expose the intended race.

What it costs

Reads	Loads may be retried or rejected.
Writes	Coordinate invalidation with fill publication.
Storage	Version or coordination metadata where used.
Correctness	Source commit and copy publication must be ordered.
Operations	Test slow fills and skipped writer paths.

In practice

Follow the matching study case for tested variations and registered results. A plausible boundary is not a measured crossover.

The checklist

Question	Answer	Evidence
What is the invariant?	An obsolete load cannot violate the promised freshness boundary.	mechanism
Where does the information live?	Source, cache and in-flight loader.	mechanism
Where is the contention?	Publication of the same cache key.	mechanism
What work moved from reads to writes?	Coordination replaces unchecked copy publication.	mechanism
What state is derived?	The cached value.	mechanism
What can become stale?	A late fill loaded before a source mutation.	mechanism
Under which workload does it stop working?	Uncoordinated loaders or writers.	gap
Which experiment would prove it?	Pause an old load, commit a write, then release the fill.	gap

Leases and fencing

STRATEGY · LEASES AND FENCING

Key idea

A lease grants temporary ownership; a fencing token rejects publication older than the receiver's accepted token.

The problem

A slow cache loader loses its lease, but still finishes. The receiver must decide whether its result is authorized to publish.

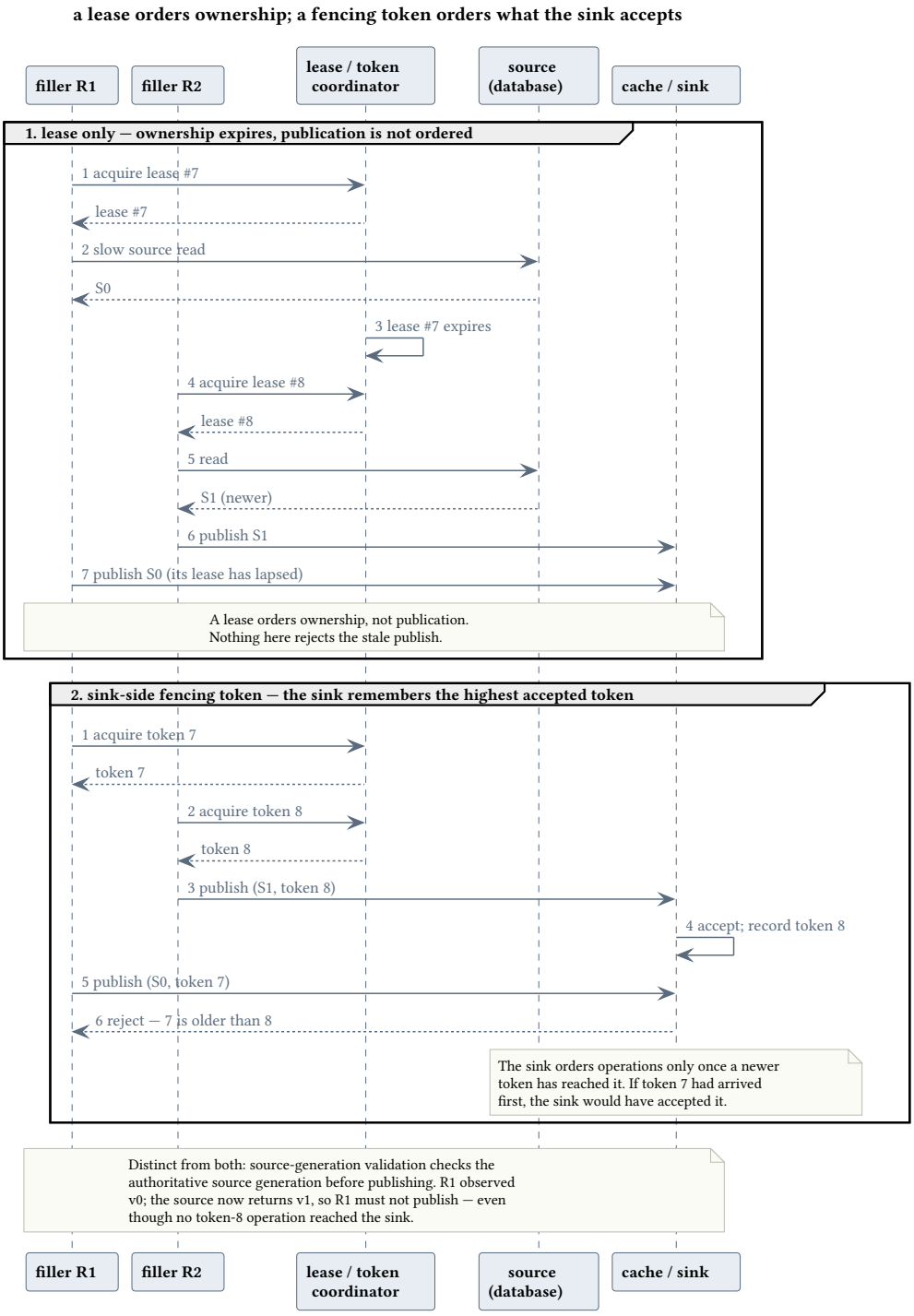


Figure 21: **Leases and fencing: mechanism and trade-off.**

Form: sequence · Evidence: conceptual illustration

Text equivalent. A receiver rejects a token below its accepted frontier. A lease can expire before any newer token arrives; source-generation validation is a separate freshness check.

How it works

- Issue an ownership token with the lease.
- Attach a monotonically increasing token to publication.
- Reject tokens below the accepted frontier; validate source freshness separately.

Thrive or die: the expected boundary

It fits when	It does not fit when
The receiver enforces token order.	Expiry alone is treated as publication protection.
Ownership can change while work is in flight.	One path bypasses receiver checks.
All publishers use the same protocol.	The token does not order competing owners.

What it costs

Reads	May wait for authorized loads.
Writes	Issue ownership and validate publication tokens.
Storage	Lease and fencing metadata.
Correctness	Reject lower tokens atomically; source freshness needs separate validation.
Operations	Track stalled workers and protocol bypasses.

In practice

Follow the matching study case for tested variations and registered results. A plausible boundary is not a measured crossover.

The checklist

Question	Answer	Evidence
What is the invariant?	No accepted publication has a token below the receiver's accepted frontier.	mechanism
Where does the information live?	Lease authority and fenced receiver.	mechanism
Where is the contention?	Ownership and publication record.	mechanism
What work moved from reads to writes?	Validation moves to publication.	mechanism
What state is derived?	Ownership token and cached answer.	mechanism
What can become stale?	Results produced after a worker loses authority.	mechanism
Under which workload does it stop working?	Unfenced or partial publication paths.	gap
Which experiment would prove it?	Delay an owner past transfer and attempt stale publication.	gap

Queues

Queue theory extends the same ideas about ownership, retries and copies. This repository has no registered queue throughput study.

Strategies: Asynchronous handoff · Idempotency and duplicate delivery.

Asynchronous handoff

STRATEGY · ASYNCHRONOUS HANDOFF

Key idea

A queue separates acceptance of work from completion of that work.

The problem

A service accepts a notification job. The user-facing response and the worker's later delivery are different completion events.



Figure 22: Asynchronous handoff: the operation path.

Form: sequence · Evidence: conceptual illustration

Text equivalent. Accept durable job → Worker claims work → Effect and acknowledgment. Arrows show logical order, not a measured duration.

How it works

- Persist the handoff under a defined durability rule.
- Let a worker claim and process the job.
- Acknowledge according to the retry contract.

Thrive or die: the expected boundary

It fits when	It does not fit when
Deferred completion is acceptable.	The request requires an immediate final answer.
Bursts can be absorbed within a bounded backlog.	An unbounded backlog hides overload.
Failures have an explicit retry policy.	Acknowledgment occurs before durable required effects.

What it costs

Reads	Read status instead of assuming completion.
Writes	Persist and process the handoff.
Storage	Backlog, payloads and delivery state.
Correctness	Accepted, processed and completed are distinct states.
Operations	Bound backlog; monitor age and failed jobs.

In practice

These are design expectations. The case or coverage entry identifies what has not been measured. A plausible boundary is not a measured crossover.

The checklist

Question	Answer	Evidence
What is the invariant?	An accepted job follows its stated completion contract.	mechanism
Where does the information live?	Producer state, queued job and consumer effect.	mechanism
Where is the contention?	Job claiming and constrained consumers.	mechanism
What work moved from reads to writes?	Request-path work moves to asynchronous processing.	mechanism
What state is derived?	Job status and downstream projections.	mechanism
What can become stale?	Completion status or downstream copies.	mechanism
Under which workload does it stop working?	Arrival demand exceeds sustainable service capacity.	gap
Which experiment would prove it?	Measure offered load, backlog age, failures and completed work.	gap

Idempotency and duplicate delivery

STRATEGY · IDEMPOTENCY AND DUPLICATE DELIVERY

Key idea

A retry must not repeat a business effect that was already accepted.

The problem

A worker applies a job's effect, then crashes before acknowledging it. Redelivery can repeat the operation.

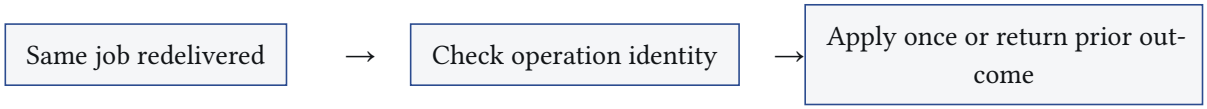


Figure 23: Idempotency and duplicate delivery: the operation path.

Form: sequence · Evidence: conceptual illustration

Text equivalent. Same job redelivered → Check operation identity → Apply once or return prior outcome. Arrows show logical order, not a measured duration.

How it works

- Give the logical operation a stable identity.
- Make effect and duplicate detection atomic where possible.
- Define behavior for external effects outside that transaction.

Thrive or die: the expected boundary

It fits when	It does not fit when
Duplicate delivery is expected.	Acknowledge-after-effect is assumed to prevent duplicates.
Effects can be keyed or conditionally applied.	Deduplication state expires before possible redelivery.
Retry behavior is part of the business contract.	External effects are repeated without their own protection.

What it costs

Reads	Check status or operation identity.
Writes	Record effect and deduplication outcome.
Storage	Operation identities and retained outcomes.
Correctness	Retries must preserve the logical operation's effect.
Operations	Bound retention and handle poison jobs.

In practice

These are design expectations. The case or coverage entry identifies what has not been measured. A plausible boundary is not a measured crossover.

The checklist

Question	Answer	Evidence
What is the invariant?	Repeated delivery preserves one accepted logical effect.	mechanism
Where does the information live?	Job identity, effect and deduplication record.	mechanism
Where is the contention?	The same logical operation identity.	mechanism
What work moved from reads to writes?	Duplicate detection occurs at effect time.	mechanism
What state is derived?	Delivery and completion status.	mechanism
What can become stale?	A status record separated from its actual effect.	mechanism
Under which workload does it stop working?	Non-atomic effect tracking or unsafe external side effects.	gap
Which experiment would prove it?	Crash between effect and acknowledgment, then redeliver.	gap

Cluster placement

A logical layout is an intention. A placement claim requires observed tablet, leader and endpoint behavior, alongside the resource budget.

Strategies: Colocation and distributed access.

Colocation and distributed access

STRATEGY · COLOCATION AND DISTRIBUTED ACCESS

Key idea

Place related work deliberately, then verify where the engine actually stores and serves it.

The problem

Joining related rows can involve several nodes. Containers sharing a laptop do not demonstrate database-level data colocation.

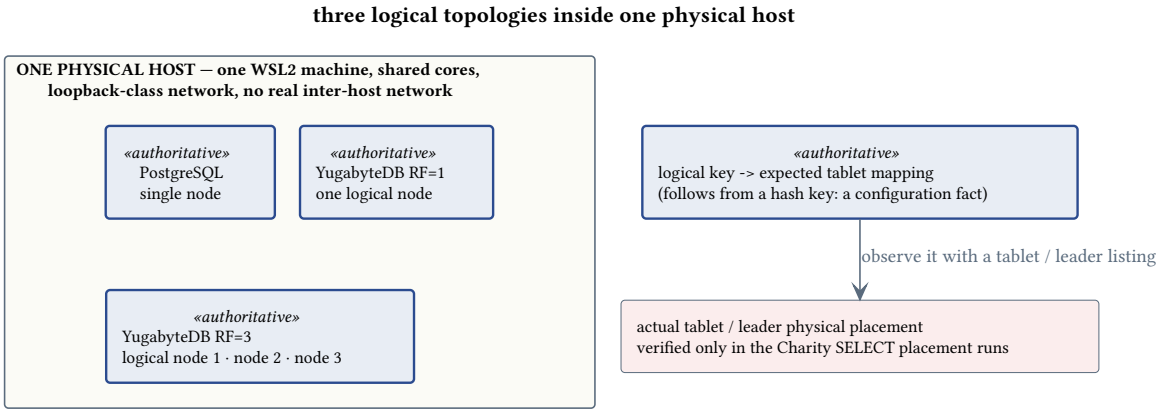


Figure 24: **Colocation and distributed access: mechanism and trade-off.**

Form: structure · Evidence: conceptual illustration

> **Text equivalent.** Logical keys and placement labels are separate from observed tablets, leaders and serving endpoints. Containers sharing a host do not establish physical data colocation.

How it works

- Hold logical data and replication fixed.
- Verify actual data and leader placement.
- Separate balanced endpoints from equal resource budgets.

A small example

Two cases test this strategy. The cache case compares key layouts whose physical placement was not observed. The Charity placement variation records tablets and leaders for every run, so its layouts are verified rather than intended.

Thrive or die: the expected boundary

It fits when	It does not fit when
Related operations align with a verified placement key.	Layout labels stand in for placement evidence.
Node and client budgets are explicit.	Cluster nodes silently receive a larger total budget.
Per-node distribution is observed.	One endpoint is assumed to represent balanced cluster capacity.

What it costs

Reads	May add or avoid cross-node work.
Writes	Replication and coordination costs remain.
Storage	Replicated data and engine metadata.
Correctness	Placement must preserve the logical answer and durability contract.
Operations	Inspect per-node resources and distribution.

In practice

Follow the matching study case for tested variations and registered results. A plausible boundary is not a measured crossover.

The checklist

Question	Answer	Evidence
What is the invariant?	The same logical answer and resource framing are compared.	mechanism
Where does the information live?	Tables, tablets, replicas and leaders.	mechanism
Where is the contention?	Hot partitions and serving endpoints.	mechanism
What work moved from reads to writes?	No automatic read/write relocation.	mechanism
What state is derived?	Replicas and any colocated derived state.	mechanism
What can become stale?	Replicas or copies under their consistency contract.	mechanism
Under which workload does it stop working?	Unverified placement, skew or unbalanced serving.	gap
Which experiment would prove it?	Match resources and workload; verify placement before interpreting effects.	gap

Part II · Study cases

Each case starts with its business problem, then applies the strategies from Part I. Summary numbers belong to the named comparison and environment, not to a universal ranking.

Study-case coverage map

Case	Theory put into practice
Charity donations	Schemas, indexes, copied keys, rollups and embedding; measured SELECT templates from join and filter to pagination, verified cluster placement and the template guide.
Ticket booking	Hot rows, history and current state.
Reserved seating	Expiry, ownership and bounded embedding.
Configuration portal	Rollup maintenance and concurrent editing.
Cache consistency	Freshness, invalidation, fencing and cluster access.

Coverage gap

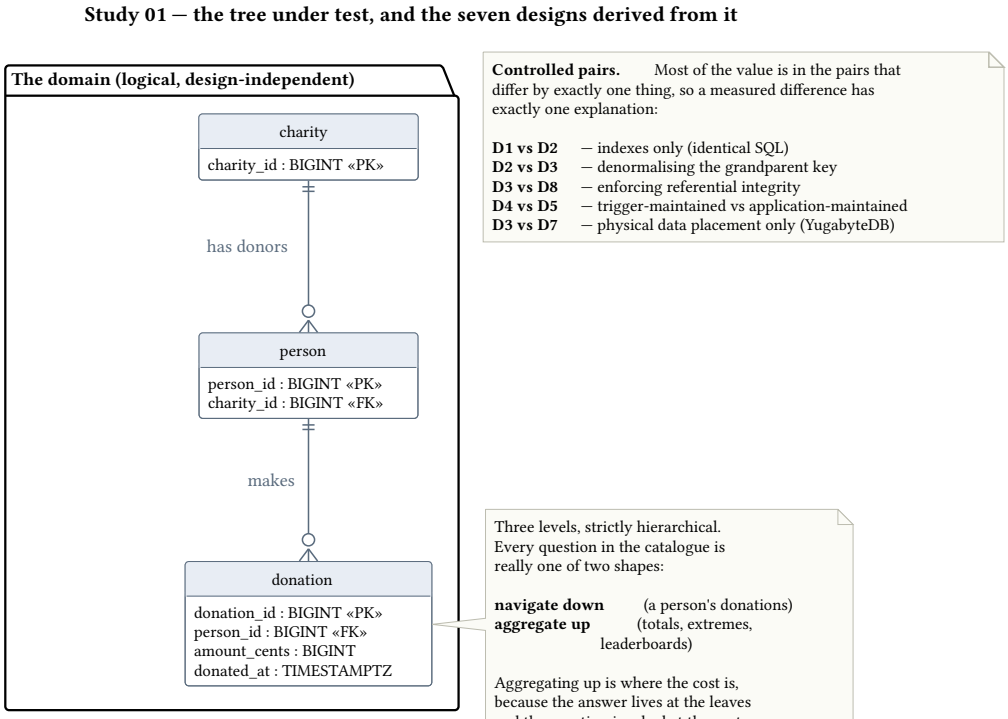
The SELECT families are measured on one laptop with one sequential reader. Their YugabyteDB membership, absence and pagination figures come from scan-only plans, and only the join family has a cluster arm. Queues have no registered measured case. Their expected boundaries are teaching examples, not benchmark results.

Charity donations

CASE · STUDY 01 · CHARITY ⟶ PERSON ⟶ DONATION

A charity site lists donors, their donations and the latest activity. Its dashboard repeatedly asks for totals; donation entry changes the facts underneath.

Keep fixed: the same donors, donations and answers. Change where information lives and how each question reaches it.



design	what changes relative to the plain tree
D1 normalized-minimal	nothing. PKs and FKs only, no secondary indexes.
D2 normalized-indexed	same tables, secondary indexes added. Identical SQL to D1.
D3 flattened-fk	donation carries charity_id — the grandparent key is copied down one level.
D4 rollup-trigger	person and charity carry aggregates, maintained by a trigger.
D5 rollup-app	same aggregates, maintained by the application instead.
D6 embedded-jsonb	the donation table disappears into a JSONB array on person.
D7 yb-child-colocated	same columns as D3; the donation PRIMARY KEY is rebuilt for data placement.
D8 flattened-nofk	D3 with the FOREIGN KEY constraints removed — everything else identical.

Figure 25: **One business tree, several ways to serve it.**

Form: structure · Evidence: implemented design contract

> **Text equivalent.** Donations belong to people, who belong to charities. Variants change joins, stored totals or child storage. The overview's pair labels describe design intent; claim limits qualify confounding and unverified placement.

Variation: find donations without copying everything

Variant	What changes	Theory
D1 minimal	Keep the normalized tree with minimal indexes.	Normalized facts
D2 indexed	Add the query index; keep the tree.	Indexes
D3 flat+FK	Copy the charity key onto donations; retain the foreign key.	Rolldown

Observed: D2 scores about 10-11x D1 on the blended read score; D3 reaches about 14-16x in two whole runs. Carry the ordering and bands, rather than the first run's point figures.

`v2-01-normalized-index-first` · **replicated** · card, p. 96

Observed: D3′s advantage over D2 survives fresh loads and reversed whole-run order. The repeated band is 1.59-1.84x; its insert-cost ratio remains imprecise. `v2-05-d3-read-advantage-repeated` · **replicated** · card, p. 107

Thrives — inferred fit: repeatable read paths that the index or copied key can answer directly.

Dies — inferred boundary: parent reassignment becomes costly if a copied key must fan out across many children. This is a maintenance risk, not a measured crossover here.

Variation: answer the dashboard from stored totals

Change: D4 uses triggers and D5 uses application code to maintain totals on parents. Readers no longer aggregate all contributing donations.

Observed summary: D4 scores about 61-85x D1 on aggregate reads; D5 about 64-83x. Inserts run at about a third to two-fifths of D1′s rate. `v2-02-rollup-family` · **replicated** · card, p. 108

Thrives — inferred fit: frequently viewed totals with relatively few contributing writes.

Dies — observed cost: every insert pays maintenance. A write-heavy workload must justify that penalty; the study does not establish a universal read/write threshold.

Theory: Rollup. For a replacement-size effect, continue to Configuration portal.

Variation: put donations inside the donor

Change: D6 stores donation children in a JSON/array column on the donor.

Observed summary: reads score about 1.7-2.2x D1, below both indexed designs. Inserts run at about 0.57-0.60x D1′s rate. `v2-03-embedding-not-ahead` · **replicated** · card, p. 109

Dies — observed: this embedding package adds write cost without beating the indexed read alternatives.

Thrives — not established: whole-document access may fit other workloads. That expectation is not a win measured in this charity case.

Theory: Embedding.

Variation: ask who donated last in a period

Define the answer first: a donor's latest donation falling inside a period differs from their latest donation **within** that period. One filters a global latest value; the other computes a period-local latest value.

Observed summary: the rollup-column index beats the recency flag by 1.9-2.3x on q13 and 2.6-3.1x on q15 trailing. It is level on q14 and slightly slower on q16. `v2-04-recency-maintained-index` · **replicated** · card, p. 108

Thrives or dies — observed: the winning path depends on the actual question. A group headline hides q16′s opposite direction.

Theory: Answer contracts, Aggregation and Pagination.

Variation: join donations and filter them

Business question: return each qualifying donation with its donor, one row per donation. The theory's customers and orders map to people and donations. The measured table holds 200,000 donations; the filter keeps 0.01% to 90% of them.

Observed — the six forms: the six join and filter forms ran the same plan in all 120 engine, phase and selectivity pairs. No form is faster. Write the one that reads best.

`v6-sel-01-inner-join-spellings-same-plan` · **replicated** · card, p. 100

Observed — the filtered derived table: filtering donations in a derived table, then joining, ran the same plan as JOIN … WHERE at every selectivity from 0.01% to 90%, on both engines and in both phases. It never outperformed the plain join, and no measured situation favours it.

`v6-sel-01-inner-join-spellings-same-plan` · **replicated** · card, p. 100

Inferred — when a derived table is more than a spelling: the planner merges a derived table that only filters. It keeps a separate step when the derived table groups, deduplicates, limits or ranks rows. The measured example is pre-aggregation, in the totals variation.

Observed — a CTE fence: forcing MATERIALIZED on a CTE that holds the join, with the filter outside, cost 84–101x on PostgreSQL and 34–49x on YugabyteDB at 0.01%. At 90% it cost 1.02–1.44x and 0.93–1.03x. With the filter inside the CTE, forcing it cost at most 1.61x on PostgreSQL.

`v6-sel-02-materialized-joined-cte-penalty` · **replicated** · card, p. 101

Observed — optional donations: LEFT JOIN with the donation filter in ON kept every donor. The same filter in WHERE dropped donors without a qualifying donation; that wrong answer was detected in all 12 runs. It is a different question, not a faster spelling.

`v6-sel-01-inner-join-spellings-same-plan` · **replicated** · card, p. 100

Thrives — observed fit: any readable join form with the engine's default CTE policy, or a filter written inside the CTE.

Dies — observed: a materialization hint that keeps a selective outer filter away from the join.

Variation: reuse one expensive CTE

Business question: several consumers read the same expensive per-donation token. Compute it once, or inline it into every consumer? One consumer selects 1% of donations; the other reads all of them.

Observed — the crossing: inlining (NOT MATERIALIZED) against MATERIALIZED crossed between one and two references, on PostgreSQL with the unselective consumer and on YugabyteDB with both. At two references inlining took 1.43–2.06x the materialized time; at sixteen, 3.9–12.1x.

`v6-sel-03-cte-reuse-crossing` · **replicated** · card, p. 101

Observed — the exception: on PostgreSQL with the selective consumer, inlining stayed faster at every count from 1 to 16: 0.17–0.27x at one reference, 0.68–0.81x at sixteen. Both engines' defaults materialize from two references, so forcing NOT MATERIALIZED was 1.2–4.8x faster there.

`v6-sel-04-cte-reuse-postgresql-selective-no-crossing` · **replicated** · card, p. 101

Thrives or dies — observed: the default policy fits three of the four measured regimes. An inlining hint helps only the selective PostgreSQL consumer. The crossing is bracketed between one and two references, not located.

Variation: who has, or has not, donated

Business question: list donors with a qualifying donation, or donors with none, each donor once. The generated SELECT fixture of Study 01 stands in for donors and donations: 10,000 donations over 10,000, 1,000 or 100 donors. That gives fanout 1, 10 and 100.

Observed — membership: JOIN plus DISTINCT took 1.40–1.69x the EXISTS time at fanout 1 and 3.64–4.23x at fanout 100 on PostgreSQL. On YugabyteDB it was 1.10–1.16x at fanout 1 and 0.95–1.01x above it. Fanout and donor count change together here.

`v6-sel-05-membership-join-distinct-penalty` · **replicated** · card, p. 102

Observed — IN or EXISTS: neither engine showed a resolved difference; PostgreSQL ranged 0.86–1.30 with mixed signs. Choose the form that reads better.

`v6-sel-06-membership-in-exists-no-difference` · **repeated — no prior prediction** · card, p. 102

Observed — absence: NOT IN over a donation key containing a NULL returned no donors in all 18 checks. NOT EXISTS, LEFT JOIN with IS NULL, and NOT IN over a NOT NULL subquery returned the right donors.

`v6-sel-07-nullable-not-in-empty` · **replicated** · card, p. 102

Observed — anti-join cost: at fanout 100 on PostgreSQL, NOT EXISTS took 0.21–0.24 ms. LEFT JOIN with IS NULL was 2.71–3.27x slower and NOT-NULL NOT IN 2.85–3.16x. On YugabyteDB the three stayed within 0.997–1.097x.

`v6-sel-08-anti-join-not-exists-postgresql` · **repeated — no prior prediction** · card, p. 103

Thrives or dies — observed: EXISTS and NOT EXISTS fit both questions. JOIN plus DISTINCT dies at high fanout on PostgreSQL. A NOT IN that can meet a NULL always dies.

Variation: donor totals and each donor's top donation

Business question: one total per requested donor, or each donor's largest donation. The request asks for 1 to 10,000 donors on the same generated fixture, two donations per donor.

Observed — totals on PostgreSQL: a correlated SUM per donor took 0.47–0.90x the join-then-GROUP-BY time for one donor. It took 1.26–1.72x at 100 donors and 2.49–2.70x at 10,000.

`v6-sel-09-aggregate-correlated-postgresql-crossover` · **replicated** · card, p. 103

Observed — totals on YugabyteDB: the correlated SUM never won. It took 1.07–1.23x at one donor and 398–541x at 10,000: 6.7–20.0 s per query, against 16–17 ms on PostgreSQL.

`v6-sel-10-aggregate-yugabytedb-no-reversal` · **replicated** · card, p. 103

Observed — pre-aggregation in a derived table: grouping each donor's donations in a derived table before the join took 5.7–7.9x the join-then-GROUP-BY time for one donor and 1.56–1.90x at 1,000 donors on PostgreSQL. Only at 10,000 donors did it dip to 0.77–0.80x, inside the measurement drift. On YugabyteDB it never won.

`v6-sel-18-aggregate-preagg-derived-table` · **replicated** · card, p. 106

Not measured — the textbook case for pre-aggregation: joining two child collections at once multiplies rows, and grouping each first avoids that. This fixture joins one child collection, so that advantage was not tested; the multiplied answer was detected as a wrong answer instead.

Observed — the top donation: a LATERAL probe per donor beat `ROW_NUMBER()` only for one donor on YugabyteDB, at 0.54–0.70x. It lost from 10 donors, reaching 257–289x at 10,000. On PostgreSQL it lost from 100 donors.

`v6-sel-11-topn-lateral-probe-crossover` · **replicated** · card, p. 104

Observed — DISTINCT ON: on PostgreSQL it took 0.61–0.64x the `ROW_NUMBER()` time at 10,000 donors. At 10 and 100 donors the sign was mixed. It answers only the single top row.

`v6-sel-12-topn-distinct-on-postgresql` · **replicated** · card, p. 104

Thrives or dies — observed: per-donor probes fit a handful of donors. Join-then-aggregate and `ROW_NUMBER()` with a tiebreak fit large requests. A double-counting join and a tiebreak-free `ROW_NUMBER()` were detected as wrong answers in every run.

Variation: page through donations, or match two ranges

Business question: show the next five donations in amount order, or the donations inside either of two amount ranges. The generated fixture holds 12,000 donations.

Observed — pages: on PostgreSQL, keyset took 1.06–1.29x the OFFSET time at depths 1–100 and 0.21–0.24x at depth 10,000. Both returned the same page at every depth under a fixed snapshot. The crossing lies between depth 100 and 1,000.

`v6-sel-15-keyset-offset-crossover` · **replicated** · card, p. 105

Limit — the cursor: this keyset form had no composite (amount, id) index. The flat keyset cost that the theory expects with a matching index was not tested.

Observed — OR or UNION: rewriting the two-range OR as UNION ALL or UNION never beat OR. On PostgreSQL's broad ranges they took 1.23–1.55x and 1.25–1.50x the OR time.

`v6-sel-13-or-not-beaten-by-union` · **replicated** · card, p. 104

Observed — an expression index: with the query unchanged, an index matching `(amount + 0)` cut PostgreSQL's selective expression OR to 0.28–0.33x. The plan switched to that index.

`v6-sel-14-expression-index-postgresql` · **repeated — no prior prediction** · card, p. 105

Thrives or dies — observed: OFFSET fits shallow pages and keyset deep ones. OR fits a disjunction, with an index that matches the expression. UNION ALL over overlapping ranges returned duplicates; the control caught it in every run.

Coverage gap

YugabyteDB's membership, absence, pagination and range figures come from scan-only plans: its default hash-sharded indexes could not serve these predicates. These three families also lack a cluster arm.

`v6-sel-gap-01-yugabytedb-scan-only-and-single-node-families` · **gap** · coverage gap · card, p. 106

Variation: place the joined donations on three nodes

Business question: on a three-node YugabyteDB cluster, should the join's tables share one tablet, keep one tablet each, or split into three? Tablets and leaders were checked before and after every run.

Observed — splitting: three tablets with leaders on three nodes made a 2,000-row join 0.73x and a 180,000-row join 0.54x the single-tablet layout (isolated phase). The 20-row join was 1.22x slower.

`v6-sel-16-placement-splitting-effect` · **replicated** · card, p. 105

Observed — colocation: a colocated database took 0.71x the split layout's time at a 20-row join and 0.78x at 2,000 rows. At 180,000 rows it was 1.21x slower, and 1.54x with a concurrent writer.

`v6-sel-17-placement-colocated-vs-distributed` · **repeated — no prior prediction** · card, p. 106

Thrives or dies — observed: split data fits broad reads; colocation fits point and medium reads. All nodes are containers on one laptop, so this is data placement without a network.

Theory: Colocation and distributed access.

Choosing a SELECT template

Start from the answer, then the readable form. Switch only where the observed boundary applies.

Required answer	Start with	Switch or avoid — observed
Donations with a filter	Any readable join form	A filtered derived table is the same plan: no measured gain. Avoid MATERIALIZED on a CTE with a selective outer filter. `v6-sel-01`, `v6-sel-02`
A CTE used several times	The engine's default	Inline only for a selective PostgreSQL consumer. `v6-sel-03`, `v6-sel-04`
Donors with optional donations	LEFT JOIN, filter in ON	Never move the child filter to WHERE. `v6-sel-01`
Donors who donated	EXISTS or IN	Avoid JOIN plus DISTINCT at high fanout on PostgreSQL. `v6-sel-05`, `v6-sel-06`
Donors who never donated	NOT EXISTS	Never NOT IN over a nullable key. `v6-sel-07`, `v6-sel-08`
One total per donor	Join, then GROUP BY	Correlated SUM only for very few donors, on PostgreSQL. A pre-aggregating derived table lost below 10,000 donors. `v6-sel-09`, `v6-sel-10`, `v6-sel-18`
Top donation per donor	`ROW_NUMBER()` with a tiebreak	LATERAL for a single donor; DISTINCT ON at scale on PostgreSQL. `v6-sel-11`, `v6-sel-12`
The next page	OFFSET for shallow pages	Keyset past about depth 1,000 on PostgreSQL. `v6-sel-15`
Either of two ranges	OR	UNION ALL never won; index the expression you filter on. `v6-sel-13`, `v6-sel-14`

The full claim behind each id is linked from the badges above, and the plans each claim compares are quoted in Query plans behind the SELECT claims. Every row comes from one laptop, one sequential reader and named engines; no row is a universal ranking.

Coverage gap

Native document, key-value, graph and wide-column engines were not measured. SQL JSON storage cannot establish their performance. `v2-gap-06-native-datastore-families` · **gap** · coverage gap · card, p. 110

Reproduce: `studies/01-charity-tree`; the SELECT experiment starts at `studies/01-charity-tree/protocols/select-query-templates.md`. Each badge links to the exact claims, reports, run tags and limits. The blended score combines queries; it is not the speed of every query.

General-admission ticket sales

CASE · STUDY 02 · BAND → EVENT → TICKET

A show company sells interchangeable admissions. Buyers need any available unit of capacity, while organisers edit events and report sales and refunds.

Business invariant: committed tickets must never exceed an event's capacity. A capacity unit here is not a physical seat chosen on a map.

Conceptual sequence: available capacity → successful booking → cancellation or refund.
Current inventory answers "what is available?" A retained sale/refund record answers "what happened?"

Variation: spread the hot drop across seat rows

Variant	Arbitration point	Theory
Counter designs	All buyers compete on one event or inventory counter.	Hot rows
P3 CAS	A conditional update wins one pre-created capacity row.	Optimistic writes
P2 skip-locked	Lock one free row and skip rows other buyers already lock.	Pessimistic writes

Observed summary: P3 and P2 sell 4-8x faster than single-counter designs in the hot-drop comparison. The range reproduced; absolute throughput did not. `v2-06-hot-drop-designs` · **replicated** · card, p. 97

Thrives — observed: many buyers want any capacity unit. Independent rows allow useful progress around contention.

Dies — inferred boundary: skipping another buyer's row cannot honour a request for a specific physical seat. Use Reserved seating for that different contract.

Watch: retry backoff matters. A counter on the event row can also block an organiser's unrelated edit; moving inventory away changes who shares the contention point.

Variation: retain the sale after a refund

Change: X1 adds an append-only ledger to P3. Cancellation returns capacity without erasing the financial event.

Observed summary: refunds are unanswerable in 14 of 15 designs; X1 alone answers the refund report. The ledger adds about 33-35% by table size, or about 35-36% counting every relation. `v2-07-refund-answerability-and-storage` · **replicated** · card, p. 111

Thrives — observed answerability: a business that needs refund reporting must retain the erased event somewhere.

Dies — observed boundary: current inventory alone cannot reconstruct refunds after deletion. Fast booking does not repair a missing fact.

Theory: History versus current state. Storage percentages depend on the counting rule; they are not one universal overhead.

Variation: pay for history under contention

Regime	Observed ledger cost over CAS
128 buyers; 10 seats	About 1.1x; direction unreliable at this smallest tier.
128 buyers; 100-1000 seats	About 1.2-1.4x.
128 buyers; 10000 seats	About 1.8x.
128 buyers; 100000 seats	About 2.5x.

`v2-08-ledger-race-cost-128-buyers` · **replicated** · card, p. 112

Observed: at 1,000 seats, PostgreSQL CAS is faster at every tested buyer count, 8 to 128, in both design orders. The earlier apparent change of winner was a confounded comparison.
`v2-09-postgres-ledger-ratio-unsettled` · **replicated** · card, p. 113

Thrives — inferred business choice: keep the ledger when historical answers justify the booking cost.

Dies — observed shortcut: choosing only the fastest current-state design leaves required refund answers unavailable. Buyer count and event size must travel with any performance summary.

Coverage gap

A ledger load-time multiplier is not published: unchanged code produced incompatible readings. Loading overhead is unresolved, rather than a fixed number.
`v2-gap-01-ledger-load-multiplier-unstable` · **gap** · unstable measurement · card, p. 113

Reproduce: `studies/02-ticket-booking`. Follow badges for producing tags and reports. Sell-rate evidence does not license quoting the old client-throttled read levels.

Reserved seats and checkout promises

CASE · STUDY 03 · VENUE ⟶ EVENT ⟶ SEAT

Buyers choose physical seats and hold a block while paying. The system must honour a valid hold, refuse an expired confirmation and return abandoned seats to availability.

Different from general admission: another free seat is not a substitute for the one the buyer selected. A row's uniqueness protects sale identity; the hold protocol protects the time-limited promise.

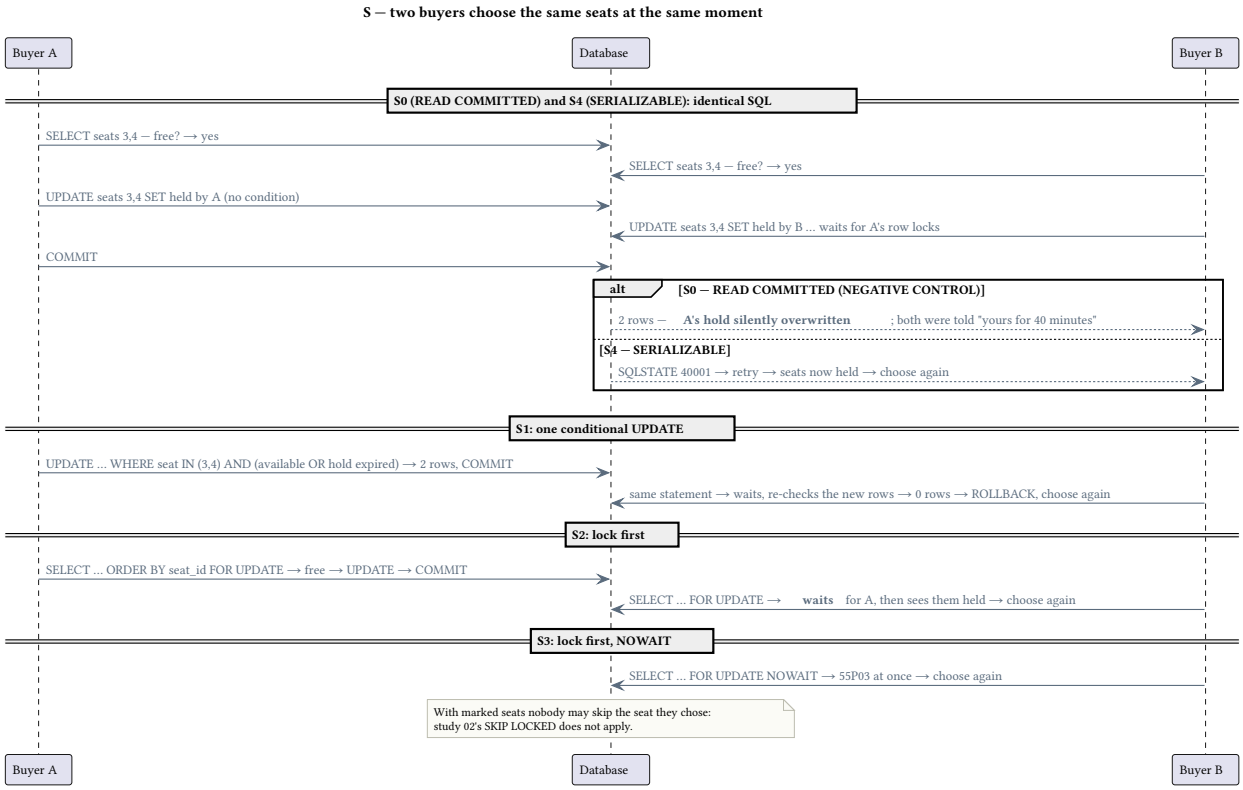


Figure 26: **Two buyers attempt to hold the same seats.**

Form: sequence · Evidence: implemented design contract

> **Text equivalent.** Separate checks let both buyers see free seats before either writes. A conditional update rechecks availability; lock-first waits and then checks. The illustration shows hold acquisition, not the confirmation refusal measurements below.

Variation: check the promise inside the write

Variant	What protects the hold	Theory
s0 control	Check, then change; the gap allows another buyer to intervene.	Expiry races
s1 conditional	The write checks the current hold and state.	Optimistic writes
s2 lock first	Lock the relevant rows before checking and changing them.	Pessimistic writes
s1r retry once	Retry the identical guarded confirmation after a transient refusal.	Promise boundary

Observed summary: at the 10,000-seat tier, YugabyteDB s1 refused valid holds twice in one run and three times in reverse order; s2 refused once in each. The identical statement then matched every seat inside the refusing transaction.

Observed correction: s1r refused nothing; every retry succeeded. PostgreSQL's correct variants refused nothing across the measured tiers in both orders. This is a correctness result, not a throughput ranking.

v2-10-guarded-confirm-refusals · **replicated** · card, p. 98

Thrives — observed: atomic guards or locks stop the check/change race. The engine-specific retry addressed the observed transient refusal.

Dies — observed: the unguarded negative control stole valid holds on both engines. A friendly pre-payment check cannot replace arbitration at confirmation.

Variation: read a section as one document

Change: L2 stores the section seat map as a document rather than assembling it from separate seat rows. L3 also changes the section's sharding package.

Observed summary: the replicated L2 comparison measures 24.4x and 27.8x on YugabyteDB single-node, versus 0.83x on PostgreSQL single-node. PostgreSQL shows no attributable advantage. `v2-11-section-seat-map` · **replicated** · card, p. 110

Thrives — observed: the YugabyteDB section-map package improves this broad read.

Dies — observed boundary: that gain does not transfer to PostgreSQL. The sharded L3 package also collapses point lookups in its primary single run; this has not been accepted as replicated evidence. A section read is not every read.

Theory: Embedding and Placement. The family/package comparison changes more than one decision, so it cannot isolate embedding alone.

Variation: choose the hold window from history

Business question: how many holds convert to sales, expire or get released? That funnel would inform the checkout policy.

> **Coverage gap**
>
> No measured design answers this hold-funnel report. Releasing an expired hold clears its row, making an expired hold indistinguishable from a seat never held. `v2-gap-02-hold-funnel-unanswerable` · **gap** · schema limitation · card, p. 114

Thrives — inferred design need: retained hold events would preserve that distinction. A history extension is a proposal here, not an executed design or measured result.

Theory: History versus current state.

Reproduce: `studies/03-reserved-seating`. Badges identify the runs and limits. The configured hold policy and compressed benchmark time must not be confused with a measured optimal checkout duration.

Configuration publication

CASE · STUDY 04 · INSTALLED PRODUCT → CONFIGURATION ENTRIES

A portal publishes configuration for installed products. Each installation owns its entries; installations of the same product definition must remain separate.

Readers request the effective configuration, an individual key or the installation overview. Writers edit keys, publish a batch or replace the whole configuration.

Conceptual maintenance path: change entries → maintain installation summary → commit publication. A trigger repeats work as rows change. An application maintainer can collect the batch and maintain its summary together.

Variation: store entries as rows or as a document

Variant	Representation	Theory
n1_rows_indexed	An indexed row for each key: the reference.	Normalized facts
n2_rows_rolldown	Parent keys copied onto entry rows.	Rolldown
d1_doc_row	One whole configuration document per installation.	Embedding

Thrives — inferred fit: entry rows suit independent key access; a document suits fetching the whole configuration together.

Dies — inferred boundary: a whole document can amplify small edits; copied parent keys add maintenance paths. These are selection considerations, not numeric wins established by the claims below.

Variation: choose who maintains the overview

Change: n3_rollup_trigger and n4_rollup_app maintain the same overview facts. The comparison changes whether database triggers or application code do the maintenance.

Observed summary: whole-configuration replacement costs 4.0-5.7x more with the trigger. The slowest 1% is 22-46x worse; repeated measurements disagree substantially, so use ranges.

v2-13-trigger-rollup-cost · **replicated** · card, p. 108

Replacement size	Application throughput relative to trigger
10 entries	Roughly 1.1-3.2x.
60 entries	Roughly 2.5-9.0x.
240 entries	Roughly 10-24x.

v2-18-maintenance-size-interaction · **replicated** · card, p. 109

Thrives — inferred fit: triggers centralise maintenance when multiple write paths must obey it.

Dies — observed cost: large replacements multiply the trigger penalty. The application variant is faster in every measured size arrangement; neither variant records a rollup mismatch.

Theory: Rollup. The experiment cannot separate trigger firing frequency from the work each firing performs.

Variation: many writers edit one hot installation

Change: c1_optimistic_version checks a version and retries bounded conflicts. c2_pessimistic_lock acquires the parent lock before reading and updating.

Observed summary: under one hot key at 16 writers, lock-before-update delivers about 1.6-1.8x the version-check throughput. Removing the retry budget does not explain that advantage.

v2-12-lock-vs-version-upper-bound · **replicated** · card, p. 98

Observed crossing: the two strategies cross between one and two writers; repeated runs preserve that boundary and the divergence above it. An unexplained one-writer excursion remains recorded. v2-17-optimistic-pessimistic-crossover · **replicated** · card, p. 100

Thrives — observed regime: version checking leads at one writer in the reproduced crossing; locking leads as the hot-key contention increases.

Dies — observed correctness: unchecked read-modify-write loses acknowledged updates. Both guarded designs lose none; correctness precedes the speed comparison.

Theory: Optimistic writes, Pessimistic writes and Hot rows. The crossing describes this hot-key workload, not every configuration portal.

Coverage gap

Eight of the eighteen planned designs are unimplemented. Cardinality, cadence/churn, equal-total resources, placement and YugabyteDB coverage remain incomplete; the active gap card also retains the earlier repeated-randomized-trials gap. v2-gap-03-study04-missing-designs · **gap** · coverage gap · card, p. 99

Reproduce: `studies/04-configuration-portal`. Badges name the exact replacement and concurrency experiments. Treat the later repeated claims as targeted coverage, rather than completion of the whole study plan.

A cached donor portal

CASE · STUDY 05 · SOURCE DATABASE ⟶ DONOR PAGE COPY

A donor portal repeatedly serves a donor's identity, totals and latest donations. A cache avoids rebuilding that page; writes make the cached copy potentially obsolete.

Business contract: a cached value must correspond to committed database state. Relaxed freshness can return an older committed value; strict freshness forbids values older than writes acknowledged before the read began.

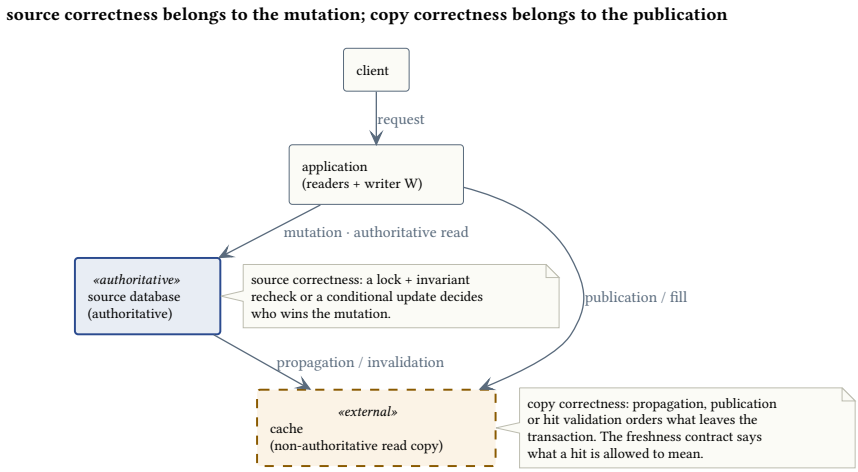


Figure 27: **A cache serves a copy of the donor page.**

Form: structure · Evidence: conceptual illustration

> **Text equivalent.** The database remains authoritative. A cache hit serves a stored read copy; a miss reaches the database and may publish a replacement. Copies require a separate freshness protocol.

Variation: cache a warm read

Variant	What changes	Theory
No cache	Build the donor page from the source database.	Source facts
Cache aside	On a miss, read the source and fill a cache entry.	Cache aside
Write-through	Maintain a cache copy through the coordinated write path.	Copies and writes
Strict protocol	Coordinate or validate freshness against acknowledged writes.	Freshness fencing

Observed summary: warm cached reads buy 2.2-2.5x throughput in the repeated PostgreSQL single-node + Redis comparison. The primary tail-latency claim did not reproduce.

`v2-14-cache-throughput-gain` · **replicated** · card, p. 114

Thrives — observed: warm, cacheable page reads avoid repeated source work.

Dies — inferred boundary: a miss-heavy or constantly changing workload may repay less of the copy's cost. No universal hit-rate crossover is established here.

Variation: coordinate copies across application instances

Observed failure: three logical instances sharing Redis under relaxed write-through record about 87% wrong reads. A lease private to one process cannot coordinate the other instances.

`v2-15-three-instance-staleness` · **mechanism** · card, p. 114

Thrives — inferred requirement: coordination must span every participant that can publish or mutate the shared value.

Dies — observed: adding a database version token without checking it does not remove stale reads. A single-instance result does not establish a multi-instance guarantee.

Theory: Lease and version fencing and Invalidation. For an asynchronous extension, see Idempotent queue consumers its full delivery behaviour is not measured here.

Variation: require strict freshness

Observed summary: on PostgreSQL, strict and relaxed read throughput differ by 7.3% and 5.7%, with the sign flipping inside trial spread. No read-throughput difference is resolved. `v2-16-strict-freshness-read-cost` · **replicated** · card, p. 115

Observed on YugabyteDB: the single-node strict/relaxed ratios, 1.043x and 1.105x, also lie inside spread. No winner is established. `v3-04-yb-single-strict-relaxed` · **replicated** · card, p. 116

Single-run cluster reading: three-node strict write-through records 942 ops/s versus relaxed 846. Each configuration is measured once; the opposite single-node direction does not justify an engine ranking. `v3-05-yb-cluster3-strict-relaxed` · **single run — direction only** · card, p. 116

Thrives — observed contract: strictness can be maintained in the measured cells without a resolved read penalty.

Dies — unsupported conclusion: "strict freshness is free" overstates the result. The write-path cost of the protocol is not quantified by these read comparisons.

Variation: keep the cache correct through churn

Observed duration: a sustained 1801 s phase crosses 6.0 real TTL boundaries with no wrong reads. Hard expiry never fires; early expiration does. `v3-01-churn-crosses-real-ttl` · **single run — direction only** · card, p. 115

Observed comparison: at 601 s, write-through and write-aside each cross 2.0 real TTL boundaries with no wrong reads; neither is declared faster. One aside cell nevertheless records an impossible cache value that its gate misses. `v3-02-churn-600-through-vs-aside` · **replicated** · card, p. 115

Thrives — scoped evidence: the tested coordination survives sustained churn for acknowledged-write freshness.

Dies — observed validation gap: no wrong reads is insufficient if an impossible value can pass the churn gate. The committed-state floor remains incompletely enforced there.

Variation: count cache resources and locality honestly

Observed accounting: the database-only and equal-total comparisons carry separate labels and budgets. Their repeated figures overlap trial spread; neither framing is pooled with the other. `v3-03-equal-total-framing-labelled` · **replicated** · card, p. 116

Single-run locality result: the donor-local layout reads faster than hashed distribution: warm reads 882 versus 673 ops/s, cacheable reads 852 versus 520. Both record no wrong reads. `v3-06-colocated-vs-noncolocated-locality` · **single run — direction only** · card, p. 107

Thrives — inferred fit: fewer distributed lookups may help a donor page. The observed comparison supports engine key locality on one host; it does not prove physical placement.

Theory: Placement and resource controls.

Coverage gap

Physical tablet/leader placement is unverified in this case. The layout labels describe intent, and all nodes share one host. Only the Charity placement variation records tablets and leaders. `v4-gap-01-physical-placement-unverified` · **gap** · instrument gap + coverage gap · card, p. 107

Coverage gap

Endpoint coverage is partial: earlier clusters use one endpoint, and this study records node count without a per-endpoint balance. The Charity SELECT cluster runs record a balanced per-endpoint count.

`v4-gap-02-endpoint-distribution-partial` · **gap** · coverage gap + instrument gap · card, p. 99

Coverage gap

Hard expiry, larger-than-harness scale, physical placement and open-loop arrivals remain unmeasured regimes. These are absent evidence, rather than null results.

`v3-gap-01-study05-remaining-regimes` · **gap** · coverage gap + instrument gap · card, p. 117

Reproduce: `studies/05-cache-consistency`. Follow badges for committed inputs and run-specific budgets. Redis is a cache in this study, not an authoritative datastore comparison.

Appendix · Evidence and reproduction

How to read and reproduce a result

Read the comparison, its workload and its limit before using a summary number. All measurements share one host; they do not model a real inter-node network.

Check	What to establish
Answer	Same grain, duplicates, NULLs, ties and ordering; independent correctness gate.
Comparison	One controlled decision, with matching data and resource framing.
Instrument	Trials, whole-run repeats, measurement order and drift.
Boundary	Observed regime, expected failure condition, or not measured.
Provenance	Claim card, inputs digest, producing run tag and committed results.

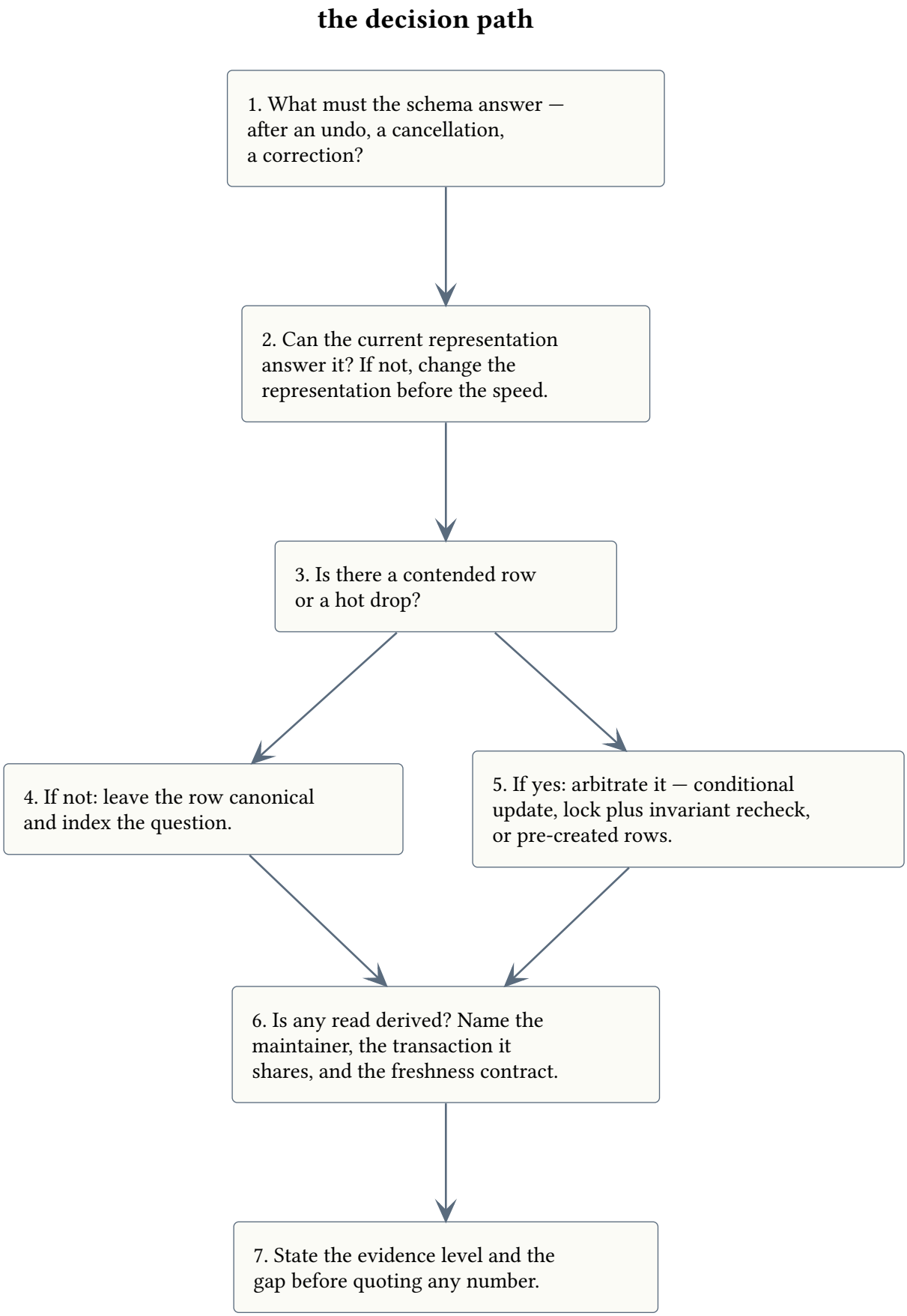


Figure 28: **Start with the required answer and the measured bottleneck.**

Form: structure · Evidence: conceptual illustration

Text equivalent. Choose an access path, information placement or coordination mechanism only after identifying the contract and the expensive work. The branches are a selection aid, not benchmark rankings.

Facts, derived state and history

The two pictures answer different questions: where a reusable answer is maintained, and whether the system must retain what happened.

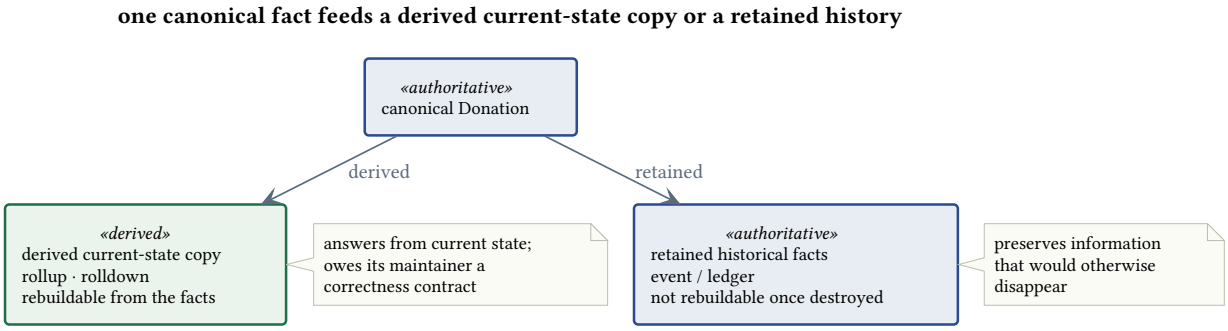


Figure 29: **A maintained answer and an event history have different roles.**

Form: structure · Evidence: conceptual illustration

> **Text equivalent.** Derived state summarizes source facts. History retains changes. A projection can make history cheap to read, but its maintenance needs a correctness rule.

Detailed coordination reference

The theory lessons use short diagrams. This expanded sequence shows the interleaving behind the optimistic and pessimistic write strategies.

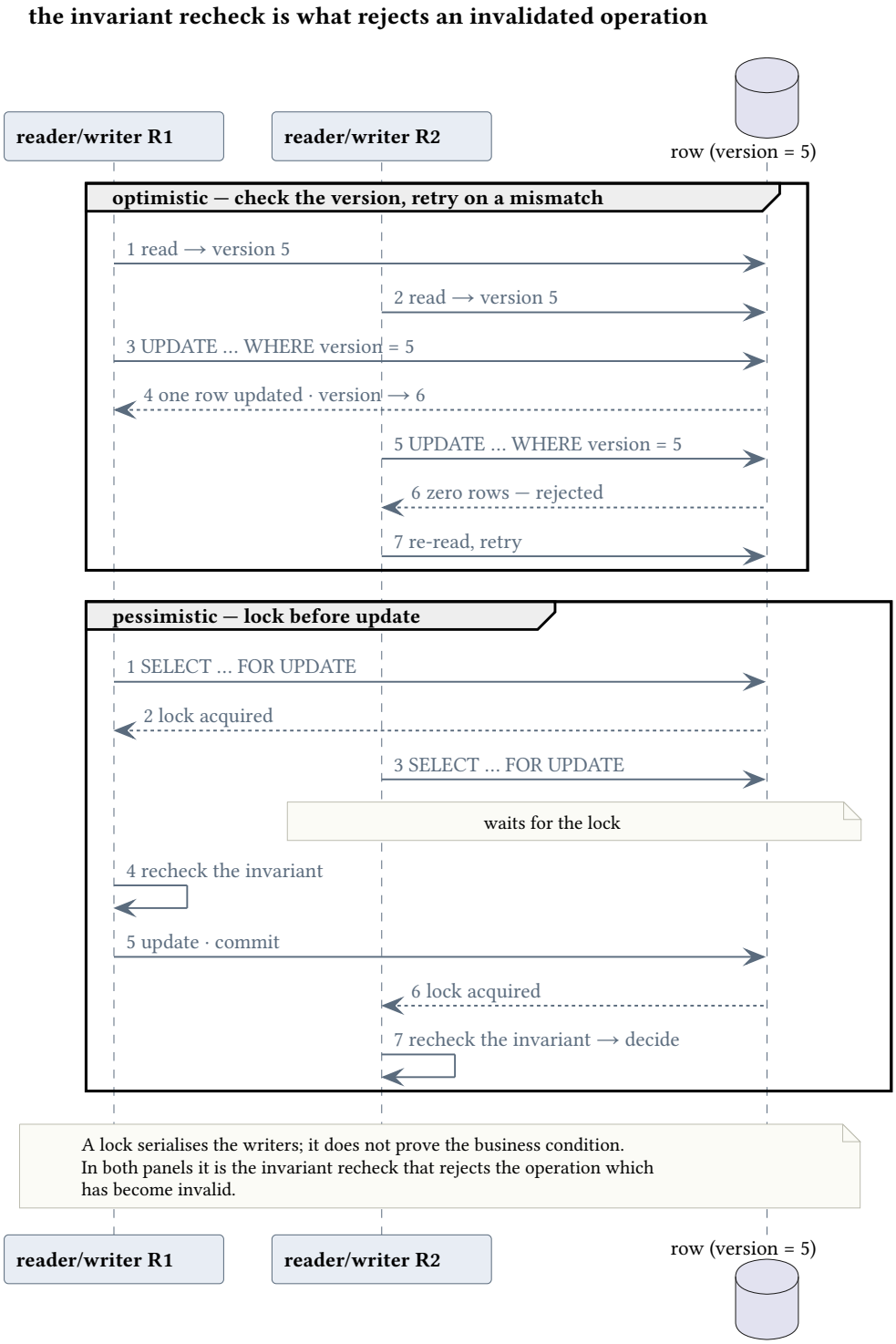


Figure 30: **A conditional write detects change; a lock orders writers before their decision.**

Form: sequence · Evidence: conceptual illustration

> **Text equivalent.** Optimistic writers reject a mismatched version and re-read. Pessimistic writers wait before reading. Both still recheck the business invariant.

Reproduce the book

```
book/build.sh
cat book/dist/build-manifest.json
```

The pinned Podman build records its source commit, source hash, registry digest, image and PDF hash.

Reproduce a measured run

- Open the cited claim card and follow its reports, input digest and run tags.
- A run tag identifies producing code; committed results live separately in the study's results directory.
- Use the named study runner. It acquires the benchmark lock; never start a competing database manually.
- The SELECT claims' `EXPLAIN ANALYZE` plans are quoted, with their digests, in Query plans behind the SELECT claims.

The full claim cards below are the audit trail. Their detailed limits are separate from the teaching chapters.

Query plans behind the SELECT claims

Every SELECT cell in the Charity study stored the text of one `EXPLAIN ANALYZE` execution in the run's result file. It was captured after the cell's answer passed the correctness check and before its timed samples. This appendix quotes those plans for the claims in the Charity case's SELECT variations, from join and filter to placement. The times inside a plan come from that one instrumented execution. The ratios in the claims come from the timed samples, so the two need not agree.

Each exhibit names the run, the tag of the code that produced it, the result file and its sha256. The generator, `tools/evidence plans`, accepts a run only if it is one of the claim's registry anchors and the file's digest equals the anchor's. It also recomputes every "same node tree" and "different node tree" statement on these pages. A node tree is the plan without its numbers: node types, relations, indexes and nesting. The run shown is one of three to twenty-four behind each claim.

Where the SQL comes from. A result cell records the name of its catalogue statement and the parameters it was bound with, not the SQL text. Each plan is therefore preceded by that statement with its parameters bound, rebuilt by the study's own catalogue and binder: the code that ran it. The generator refuses the SQL unless the statement and parameters match the cell. It also requires every file the catalogue is built from to be byte-identical at the run's producing commit. Some statements carry constant switches, such as `WHERE 1 = 0 OR c.id <= 1`. These let one catalogue statement serve several arms of a run, and the plans show the planner folding the constant part away. The membership, aggregate and access SQL reads the `customers` and `orders` tables of the teaching fixture directly.

Trimmed	Node lines are kept with their actual time, rows and loops; the planner's cost estimate is removed from each node line. Of the other lines only the listed attributes are kept. Output column lists, buffer counts, YugabyteDB per-request storage metrics and estimates, and memory figures are removed. Indentation and order are unchanged.
Kept attributes	`Index Cond:`, `Recheck Cond:`, `Filter:`, `Rows Removed by Filter:`, `Rows Removed by Index Recheck:`, `Join Filter:`, `Rows Removed by Join Filter:`, `Hash Cond:`, `Merge Cond:`, `One-Time Filter:`, `Run Condition:`, `Storage Filter:`, `Storage Index Filter:`, `Partial Aggregate:`, `Sort Key:`, `Presorted Key:`, `Sort Method:`, `Group Key:`, `Cache Key:`, `Hits:`, `Heap Fetches:`, `Workers Launched:`, `Storage Table Read Requests:`, `Storage Index Read Requests:`, `Storage Table Rows Scanned:`, `Storage Index Rows Scanned:`, `SubPlan`, `InitPlan`, `CTE`, `Planning Time:`, `Execution Time:`, `Storage Read Requests:`, `Storage Rows Scanned:`
Full text	The untrimmed plan is the cell's `Plan` field in the named result file, under the axis printed beside each plan.

Reading a plan. A line starting with `->` is an input of the nearest line above it with less indentation. `actual time=a..b` is milliseconds to the first and the last row, and `rows` is the row count; both are per loop, and `loops` says how many times the node ran. `Rows Removed by Filter` counts rows read and then discarded. YugabyteDB reports `Storage ... Read Requests` only when the plan was captured with its `DIST` option: the join, CTE and placement runs. The access, aggregate and membership runs used the plain options, so their YugabyteDB plans carry no request counts.

Join spellings and CTE fences

The join, CTE and placement SQL reads two views. `customers` joins `person` to `charity`, and `facts` reads `donation`. The planner expands both views, so the plans name the three tables.

Filtered join spellings

`v6-sel-01-inner-join-spellings-same-plan` · **replicated** · card, p. 100

The planner flattens every spelling of the filtered join into the same tree. On PostgreSQL the 1% rank filter reaches the rank index (`Bitmap Index Scan on donation_rank_idx`) below both hash joins. On YugabyteDB it reaches the range-sharded rank index (`Index Scan using donation_rank_idx`). The generator compared the other five spellings with these plans: each has the same node tree, on both engines.

PostgreSQL, single node · run `select-join-v3b-domain-pg-i1-20261005` · tag `run/study-01/select-join-v3b-domain-pg-i1-20261005` `studies/01-charity-tree/results/select-join-v3b-domain-pg-i1-20261005/join.json` · sha256 `724642ee2a9420bed8795540dfb5132831794dbb5f4c0588ace27cf0707614da`

JOIN ... WHERE, rank <= 2,000 (1% of donations)

SQL `j01_join_where` · parameters `threshold=2000`

```
SELECT c.id AS customer_id, c.name AS customer_name, f.id AS fact_id, f.amount
FROM customers c JOIN facts f ON f.customer_id = c.id
WHERE f.rank <= 2000;
```

Plan · `family=bag;stmt=join_where;threshold=2000` · 14 of 36 lines kept

```
Hash Join (actual time=0.385..1.231 rows=2000 loops=1)
  Hash Cond: (p.charity_id = ch.charity_id)
  ->  Hash Join (actual time=0.374..1.070 rows=2000 loops=1)
        Hash Cond: (d.person_id = p.person_id)
        ->  Bitmap Heap Scan on selectjoin_domain.donation d (actual time=0.166..0.660 rows=2000 loops=1)
              Recheck Cond: (d.rank <= 2000)
              ->  Bitmap Index Scan on donation_rank_idx (actual time=0.081..0.081 rows=2000 loops=1)
                    Index Cond: (d.rank <= 2000)
        ->  Hash (actual time=0.206..0.206 rows=2000 loops=1)
              ->  Seq Scan on selectjoin_domain.person p (actual time=0.003..0.087 rows=2000 loops=1)
  ->  Hash (actual time=0.010..0.010 rows=50 loops=1)
        ->  Seq Scan on selectjoin_domain.charity ch (actual time=0.004..0.006 rows=50 loops=1)
Planning Time: 0.123 ms
Execution Time: 1.288 ms
```

Checked by the generator. filtered derived table (`j03_filtered_derived`), filter in JOIN ... ON (`j02_join_on`), CTE naming the filtered input (`j04_cte_filtered`), CTE naming the joined rows (`j05_cte_joined_outer`), comma join (`j06_comma_join`): same node tree as "JOIN ... WHERE, rank <= 2,000 (1% of donations)".

filtered derived table

SQL `j03_filtered_derived`

```
SELECT c.id AS customer_id, c.name AS customer_name, f.id AS fact_id, f.amount
FROM customers c
JOIN (SELECT * FROM facts WHERE rank <= 2000) f ON f.customer_id = c.id;
```

filter in JOIN ... ON

SQL `j02_join_on`

```
SELECT c.id AS customer_id, c.name AS customer_name, f.id AS fact_id, f.amount
FROM customers c JOIN facts f ON f.customer_id = c.id AND f.rank <= 2000;
```

CTE naming the filtered input

SQL `j04_cte_filtered`

```
WITH e AS (SELECT * FROM facts WHERE rank <= 2000)
SELECT c.id AS customer_id, c.name AS customer_name, e.id AS fact_id, e.amount
FROM customers c JOIN e ON e.customer_id = c.id;
```

CTE naming the joined rows

SQL `j05_cte_joined_outer`

```
WITH e AS (
  SELECT c.id AS customer_id, c.name AS customer_name, f.id AS fact_id, f.amount, f.rank
  FROM customers c JOIN facts f ON f.customer_id = c.id
)
SELECT customer_id, customer_name, fact_id, amount FROM e WHERE rank <= 2000;
```

comma join

SQL `j06_comma_join`

```
SELECT c.id AS customer_id, c.name AS customer_name, f.id AS fact_id, f.amount
FROM customers c, facts f
WHERE c.id = f.customer_id AND f.rank <= 2000;
```

YugabyteDB, single node · run `select-join-v3b-domain-yb-i1-20261005` · tag `run/study-01/select-join-v3b-domain-yb-i1-20261005` `studies/01-charity-tree/results/select-join-v3b-domain-yb-i1-20261005/join.json` · sha256 `aed6b10e8425f3871462de7a93c38b198680edb2c1ef539e7a94d85c90c8ffa6`

JOIN ... WHERE, rank <= 2,000 (1% of donations)

SQL `j01_join_where` · parameters `threshold=2000`

```
SELECT c.id AS customer_id, c.name AS customer_name, f.id AS fact_id, f.amount
FROM customers c JOIN facts f ON f.customer_id = c.id
WHERE f.rank <= 2000;
```

Plan · `family=bag;stmt=join_where;threshold=2000` · 22 of 96 lines kept

```
Hash Join (actual time=4.696..7.879 rows=2000 loops=1)
  Hash Cond: (p.charity_id = ch.charity_id)
  ->  Hash Join (actual time=4.410..7.354 rows=2000 loops=1)
        Hash Cond: (d.person_id = p.person_id)
        ->  Index Scan using donation_rank_idx on donation d (actual time=3.299..5.961 rows=2000 loops=1)
              Index Cond: (rank <= 2000)
              Storage Table Read Requests: 2
              Storage Table Rows Scanned: 2000
              Storage Index Read Requests: 2
              Storage Index Rows Scanned: 2000
        ->  Hash (actual time=1.091..1.091 rows=2000 loops=1)
              ->  Seq Scan on person p (actual time=0.534..0.889 rows=2000 loops=1)
                    Storage Table Read Requests: 2
                    Storage Table Rows Scanned: 2000
  ->  Hash (actual time=0.281..0.281 rows=50 loops=1)
        ->  Seq Scan on charity ch (actual time=0.266..0.270 rows=50 loops=1)
              Storage Table Read Requests: 1
              Storage Table Rows Scanned: 50
```

```
Planning Time: 0.201 ms
Execution Time: 8.067 ms
Storage Read Requests: 7
Storage Rows Scanned: 6050
```

Checked by the generator. filtered derived table (`j03_filtered_derived`), filter in JOIN ... ON (`j02_join_on`), CTE naming the filtered input (`j04_cte_filtered`), CTE naming the joined rows (`j05_cte_joined_outer`), comma join (`j06_comma_join`): same node tree as "JOIN ... WHERE, rank <= 2,000 (1% of donations)".

filtered derived table SQL `j03_filtered_derived`
Same SQL as in the PostgreSQL, single node exhibit above.

filter in JOIN ... ON SQL `j02_join_on`
Same SQL as in the PostgreSQL, single node exhibit above.

CTE naming the filtered input SQL `j04_cte_filtered`
Same SQL as in the PostgreSQL, single node exhibit above.

CTE naming the joined rows SQL `j05_cte_joined_outer`
Same SQL as in the PostgreSQL, single node exhibit above.

comma join SQL `j06_comma_join`
Same SQL as in the PostgreSQL, single node exhibit above.

Forced MATERIALIZED around the join

`v6-sel-02-materialized-joined-cte-penalty` · **replicated** · card, p. 101

The fence is the `CTE e` under a `CTE Scan`. Inside it the join reads all 200,000 donations sequentially, and the filter outside then discards 198,000 of them (`Rows Removed by Filter`). Left to the engine, the same CTE ran the `JOIN ... WHERE` plan above; the forced plan's node tree differs from it.

PostgreSQL, single node · run `select-join-v3b-domain-pg-i1-20261005` · tag `run/study-01/select-join-v3b-domain-pg-i1-20261005` `studies/01-charity-tree/results/select-join-v3b-domain-pg-i1-20261005/join.json` · sha256 `724642ee2a9420bed8795540dfb5132831794dbb5f4c0588ace27cf0707614da`

CTE holding the join, forced MATERIALIZED, filter outside SQL `j05m_cte_joined_outer_mat` · parameters `threshold=2000`

```
WITH e AS MATERIALIZED (
  SELECT c.id AS customer_id, c.name AS customer_name, f.id AS fact_id, f.amount, f.rank
  FROM customers c JOIN facts f ON f.customer_id = c.id
)
SELECT customer_id, customer_name, fact_id, amount FROM e WHERE rank <= 2000;
```

Plan · `family=bag;stmt=cte_joined_outer_mat;threshold=2000` · 15 of 37 lines kept

```
CTE Scan on e (actual time=0.278..75.369 rows=2000 loops=1)
  Filter: (e.rank <= 2000)
  Rows Removed by Filter: 198000
  CTE e
    ->  Hash Join (actual time=0.237..45.796 rows=200000 loops=1)
          Hash Cond: (p.charity_id = ch.charity_id)
          ->  Hash Join (actual time=0.223..30.250 rows=200000 loops=1)
                Hash Cond: (d.person_id = p.person_id)
                ->  Seq Scan on selectjoin_domain.donation d (actual time=0.003..9.698 rows=200000 loops=1)
                ->  Hash (actual time=0.218..0.219 rows=2000 loops=1)
                      ->  Seq Scan on selectjoin_domain.person p (actual time=0.003..0.093 rows=2000 loops=1)
          ->  Hash (actual time=0.012..0.013 rows=50 loops=1)
                ->  Seq Scan on selectjoin_domain.charity ch (actual time=0.005..0.008 rows=50 loops=1)
Planning Time: 0.186 ms
Execution Time: 77.915 ms
```

Checked by the generator. this plan (`j05m_cte_joined_outer_mat`): different node tree from `family=bag;stmt=cte_joined_outer;threshold=2000`.

Compared plan, not quoted here SQL `j05_cte_joined_outer`
Same SQL as in the PostgreSQL, single node exhibit above.

YugabyteDB, single node · run `select-join-v3b-domain-yb-i1-20261005` · tag `run/study-01/select-join-v3b-domain-yb-i1-20261005` `studies/01-charity-tree/results/select-join-v3b-domain-yb-i1-20261005/join.json` · sha256 `aed6b10e8425f3871462de7a93c38b198680edb2c1ef539e7a94d85c90c8ffa6`

CTE holding the join, forced MATERIALIZED, filter outside SQL `j05m_cte_joined_outer_mat` · parameters `threshold=2000`

```
WITH e AS MATERIALIZED (
  SELECT c.id AS customer_id, c.name AS customer_name, f.id AS fact_id, f.amount, f.rank
  FROM customers c JOIN facts f ON f.customer_id = c.id
)
SELECT customer_id, customer_name, fact_id, amount FROM e WHERE rank <= 2000;
```

Plan · `family=bag;stmt=cte_joined_outer_mat;threshold=2000` · 23 of 95 lines kept

```
CTE Scan on e (actual time=2.264..106.292 rows=2000 loops=1)
  Filter: (rank <= 2000)
  Rows Removed by Filter: 198000
  CTE e
    ->  Hash Join (actual time=2.253..66.380 rows=200000 loops=1)
          Hash Cond: (d.person_id = p.person_id)
          ->  Seq Scan on donation d (actual time=0.787..29.892 rows=200000 loops=1)
```

```
               Storage Table Read Requests: 196
               Storage Table Rows Scanned: 200000
         ->  Hash (actual time=1.455..1.458 rows=2000 loops=1)
               ->  Hash Join (actual time=0.903..1.261 rows=2000 loops=1)
                     Hash Cond: (p.charity_id = ch.charity_id)
                     ->  Seq Scan on person p (actual time=0.590..0.708 rows=2000 loops=1)
                           Storage Table Read Requests: 2
                           Storage Table Rows Scanned: 2000
                     ->  Hash (actual time=0.303..0.304 rows=50 loops=1)
                           ->  Seq Scan on charity ch (actual time=0.284..0.290 rows=50 loops=1)
                                 Storage Table Read Requests: 1
                                 Storage Table Rows Scanned: 50
Planning Time: 0.187 ms
Execution Time: 107.291 ms
Storage Read Requests: 199
Storage Rows Scanned: 202050
```

Checked by the generator. this plan (`j05m_cte_joined_outer_mat`): different node tree from `family=bag;stmt=cte_joined_outer;threshold=2000`.

Compared plan, not quoted here SQL `j05_cte_joined_outer`
Same SQL as in the PostgreSQL, single node exhibit above.

One CTE read twice

`v6-sel-03-cte-reuse-crossing` · **replicated** · card, p. 101

MATERIALIZED runs the CTE body once (`CTE e`), and both references read the stored rows (`CTE Scan on e` and `e_1`). NOT MATERIALIZED copies the body into each reference, so the index scan runs twice: 8 storage read requests on YugabyteDB against 4. At two references each engine's default has MATERIALIZED's node tree.

YugabyteDB, single node · run `select-join-v3b-domain-yb-i1-20261005` · tag `run/study-01/select-join-v3b-domain-yb-i1-20261005` `studies/01-charity-tree/results/select-join-v3b-domain-yb-i1-20261005/join.json` · sha256 `aed6b10e8425f3871462de7a93c38b198680edb2c1ef539e7a94d85c90c8ffa6`

two references, MATERIALIZED, unselective consumer SQL `r_mat_n2_c100` · parameters `consumer_max=200000,threshold=2000`

```
WITH e AS MATERIALIZED (
  SELECT f.id AS consumer_id, ascii(substr(md5(repeat(f.id::text, 32)), 1, 1)) AS cksum
  FROM facts f WHERE f.rank <= 2000
)
SELECT 1 AS ref, COALESCE(sum(cksum), 0)::bigint AS checksum FROM e WHERE e.consumer_id <= 200000
UNION ALL
SELECT 2 AS ref, COALESCE(sum(cksum), 0)::bigint AS checksum FROM e WHERE e.consumer_id <= 200000
ORDER BY ref
```

Plan · `family=reuse;policy=mat;refs=2;consumer=100%` · 21 of 53 lines kept

```
Sort (actual time=7.370..7.371 rows=2 loops=1)
  Sort Key: (1)
  Sort Method: quicksort  Memory: 25kB
  CTE e
    ->  Index Scan using donation_rank_idx on donation d (actual time=3.129..6.672 rows=2000 loops=1)
          Index Cond: (rank <= 2000)
          Storage Table Read Requests: 2
          Storage Table Rows Scanned: 2000
          Storage Index Read Requests: 2
          Storage Index Rows Scanned: 2000
  ->  Append (actual time=7.125..7.362 rows=2 loops=1)
        ->  Aggregate (actual time=7.125..7.125 rows=1 loops=1)
              ->  CTE Scan on e (actual time=3.131..6.962 rows=2000 loops=1)
                    Filter: (consumer_id <= 200000)
        ->  Aggregate (actual time=0.235..0.235 rows=1 loops=1)
              ->  CTE Scan on e e_1 (actual time=0.001..0.081 rows=2000 loops=1)
                    Filter: (consumer_id <= 200000)
Planning Time: 0.143 ms
Execution Time: 7.424 ms
Storage Read Requests: 4
Storage Rows Scanned: 4000
```

two references, NOT MATERIALIZED, unselective consumer SQL `r_notmat_n2_c100` · parameters `consumer_max=200000,threshold=2000`

```
WITH e AS NOT MATERIALIZED (
  SELECT f.id AS consumer_id, ascii(substr(md5(repeat(f.id::text, 32)), 1, 1)) AS cksum
  FROM facts f WHERE f.rank <= 2000
)
SELECT 1 AS ref, COALESCE(sum(cksum), 0)::bigint AS checksum FROM e WHERE e.consumer_id <= 200000
UNION ALL
SELECT 2 AS ref, COALESCE(sum(cksum), 0)::bigint AS checksum FROM e WHERE e.consumer_id <= 200000
ORDER BY ref
```

Plan · `family=reuse;policy=notmat;refs=2;consumer=100%` · 24 of 72 lines kept

```
Sort (actual time=14.889..14.891 rows=2 loops=1)
  Sort Key: (1)
  Sort Method: quicksort  Memory: 25kB
  ->  Append (actual time=7.736..14.880 rows=2 loops=1)
        ->  Aggregate (actual time=7.735..7.736 rows=1 loops=1)
```

```
              ->  Index Scan using donation_rank_idx on donation d (actual time=3.863..6.473 rows=2000 loops=1)
                    Index Cond: (rank <= 2000)
                    Storage Filter: (donation_id <= 200000)
                    Storage Table Read Requests: 2
                    Storage Table Rows Scanned: 2000
                    Storage Index Read Requests: 2
                    Storage Index Rows Scanned: 2000
        ->  Aggregate (actual time=7.141..7.141 rows=1 loops=1)
              ->  Index Scan using donation_rank_idx on donation d_1 (actual time=3.075..5.884 rows=2000 loops=1)
                    Index Cond: (rank <= 2000)
                    Storage Filter: (donation_id <= 200000)
                    Storage Table Read Requests: 2
                    Storage Table Rows Scanned: 2000
                    Storage Index Read Requests: 2
                    Storage Index Rows Scanned: 2000
Planning Time: 0.171 ms
Execution Time: 14.950 ms
Storage Read Requests: 8
Storage Rows Scanned: 8000
```

Checked by the generator. engine default, two references (`r_default_n2_c100`): same node tree as “two references, MATERIALIZED, unselective consumer”.

engine default, two references SQL `r_default_n2_c100`

```
WITH e AS (
  SELECT f.id AS consumer_id, ascii(substr(md5(repeat(f.id::text, 32)), 1, 1)) AS cksum
  FROM facts f WHERE f.rank <= 2000
)
SELECT 1 AS ref, COALESCE(sum(cksum), 0)::bigint AS checksum FROM e WHERE e.consumer_id <= 200000
UNION ALL
SELECT 2 AS ref, COALESCE(sum(cksum), 0)::bigint AS checksum FROM e WHERE e.consumer_id <= 200000
ORDER BY ref
```

PostgreSQL, single node · run `select-join-v3b-domain-pg-i1-20261005` · tag `run/study-01/select-join-v3b-domain-pg-i1-20261005` `studies/01-charity-tree/results/select-join-v3b-domain-pg-i1-20261005/join.json` · sha256 `724642ee2a9420bed8795540dfb5132831794dbb5f4c0588ace27cf0707614da`

two references, MATERIALIZED, unselective consumer SQL `r_mat_n2_c100` · parameters `consumer_max=200000,threshold=2000`

```
WITH e AS MATERIALIZED (
  SELECT f.id AS consumer_id, ascii(substr(md5(repeat(f.id::text, 32)), 1, 1)) AS cksum
  FROM facts f WHERE f.rank <= 2000
)
SELECT 1 AS ref, COALESCE(sum(cksum), 0)::bigint AS checksum FROM e WHERE e.consumer_id <= 200000
UNION ALL
SELECT 2 AS ref, COALESCE(sum(cksum), 0)::bigint AS checksum FROM e WHERE e.consumer_id <= 200000
ORDER BY ref
```

Plan · `family=reuse;policy=mat;refs=2;consumer=100%` · 17 of 30 lines kept

```
Sort (actual time=2.747..2.748 rows=2 loops=1)
  Sort Key: (1)
  Sort Method: quicksort  Memory: 25kB
  CTE e
    ->  Bitmap Heap Scan on selectjoin_domain.donation d (actual time=0.205..2.198 rows=2000 loops=1)
          Recheck Cond: (d.rank <= 2000)
          ->  Bitmap Index Scan on donation_rank_idx (actual time=0.103..0.103 rows=2000 loops=1)
                Index Cond: (d.rank <= 2000)
  ->  Append (actual time=2.580..2.743 rows=2 loops=1)
        ->  Aggregate (actual time=2.579..2.580 rows=1 loops=1)
              ->  CTE Scan on e (actual time=0.207..2.455 rows=2000 loops=1)
                    Filter: (e.consumer_id <= 200000)
        ->  Aggregate (actual time=0.162..0.162 rows=1 loops=1)
              ->  CTE Scan on e e_1 (actual time=0.001..0.090 rows=2000 loops=1)
                    Filter: (e_1.consumer_id <= 200000)
Planning Time: 0.077 ms
Execution Time: 2.770 ms
```

Checked by the generator. engine default, two references (`r_default_n2_c100`): same node tree as “two references, MATERIALIZED, unselective consumer”.
NOT MATERIALIZED, two references (`r_notmat_n2_c100`): different node tree from “two references, MATERIALIZED, unselective consumer”.

engine default, two references SQL `r_default_n2_c100`
Same SQL as in the YugabyteDB, single node exhibit above.

NOT MATERIALIZED, two references SQL `r_notmat_n2_c100`
Same SQL as in the YugabyteDB, single node exhibit above.

Inlining for a selective consumer on PostgreSQL

`v6-sel-04-cte-reuse-postgresql-selective-no-crossing` · **replicated** · card, p. 101

With one reference and a 1% consumer, MATERIALIZED builds all 2,000 CTE rows and filters 1,983 of them away afterwards. Inlined, both predicates reach one scan. The planner uses the primary-key index for the consumer’s `donation_id` range and tests the rank as a filter, with no stored intermediate result.

PostgreSQL, single node · run `select-join-v3b-domain-pg-i1-20261005` · tag `run/study-01/select-join-v3b-domain-pg-i1-20261005` `studies/01-charity-tree/results/select-join-v3b-domain-pg-i1-20261005/join.json` · sha256 `724642ee2a9420bed8795540dfb5132831794dbb5f4c0588ace27cf0707614da`

one reference, MATERIALIZED, 1% consumer SQL `r_mat_n1_c1` · parameters `consumer_max=2000,threshold=2000`

```
WITH e AS MATERIALIZED (
  SELECT f.id AS consumer_id, ascii(substr(md5(repeat(f.id::text, 32)), 1, 1)) AS cksum
  FROM facts f WHERE f.rank <= 2000
)
SELECT 1 AS ref, COALESCE(sum(cksum), 0)::bigint AS checksum FROM e WHERE e.consumer_id <= 2000
ORDER BY ref
```

Plan · `family=reuse;policy=mat;refs=1;consumer=1%` · 11 of 19 lines kept

```
Aggregate (actual time=2.196..2.197 rows=1 loops=1)
  CTE e
    ->  Bitmap Heap Scan on selectjoin_domain.donation d (actual time=0.194..1.960 rows=2000 loops=1)
          Recheck Cond: (d.rank <= 2000)
          ->  Bitmap Index Scan on donation_rank_idx (actual time=0.100..0.100 rows=2000 loops=1)
                Index Cond: (d.rank <= 2000)
  ->  CTE Scan on e (actual time=0.195..2.194 rows=17 loops=1)
        Filter: (e.consumer_id <= 2000)
        Rows Removed by Filter: 1983
Planning Time: 0.050 ms
Execution Time: 2.214 ms
```

one reference, NOT MATERIALIZED, 1% consumer SQL `r_notmat_n1_c1` · parameters `consumer_max=2000,threshold=2000`

```
WITH e AS NOT MATERIALIZED (
  SELECT f.id AS consumer_id, ascii(substr(md5(repeat(f.id::text, 32)), 1, 1)) AS cksum
  FROM facts f WHERE f.rank <= 2000
)
SELECT 1 AS ref, COALESCE(sum(cksum), 0)::bigint AS checksum FROM e WHERE e.consumer_id <= 2000
ORDER BY ref
```

Plan · `family=reuse;policy=notmat;refs=1;consumer=1%` · 7 of 11 lines kept

```
Aggregate (actual time=0.143..0.144 rows=1 loops=1)
  ->  Index Scan using donation_pkey on selectjoin_domain.donation d (actual time=0.015..0.121 rows=17 loops=1)
        Index Cond: (d.donation_id <= 2000)
        Filter: (d.rank <= 2000)
        Rows Removed by Filter: 1983
Planning Time: 0.050 ms
Execution Time: 0.153 ms
```

Checked by the generator. engine default, two references (`r_default_n2_c1`): same node tree as `family=reuse;policy=mat;refs=2;consumer=1%`.

engine default, two references SQL `r_default_n2_c1`

```
WITH e AS (
  SELECT f.id AS consumer_id, ascii(substr(md5(repeat(f.id::text, 32)), 1, 1)) AS cksum
  FROM facts f WHERE f.rank <= 2000
)
SELECT 1 AS ref, COALESCE(sum(cksum), 0)::bigint AS checksum FROM e WHERE e.consumer_id <= 2000
UNION ALL
SELECT 2 AS ref, COALESCE(sum(cksum), 0)::bigint AS checksum FROM e WHERE e.consumer_id <= 2000
ORDER BY ref
```

Compared plan, not quoted here SQL `r_mat_n2_c1`

```
WITH e AS MATERIALIZED (
  SELECT f.id AS consumer_id, ascii(substr(md5(repeat(f.id::text, 32)), 1, 1)) AS cksum
  FROM facts f WHERE f.rank <= 2000
)
SELECT 1 AS ref, COALESCE(sum(cksum), 0)::bigint AS checksum FROM e WHERE e.consumer_id <= 2000
UNION ALL
SELECT 2 AS ref, COALESCE(sum(cksum), 0)::bigint AS checksum FROM e WHERE e.consumer_id <= 2000
ORDER BY ref
```

Membership and absence

JOIN + DISTINCT against EXISTS

`v6-sel-05-membership-join-distinct-penalty` · **replicated** · card, p. 102

On PostgreSQL, EXISTS is a `Nested Loop Semi Join`: one index probe per parent, which stops at the first qualifying order. JOIN + DISTINCT produces 7,240 joined rows and then removes duplicates in a `HashAggregate`. On YugabyteDB, EXISTS was planned differently. It deduplicates the qualifying orders by `customer_id` first and then hash-joins, so both YugabyteDB plans scan all 7,314 qualifying orders. No YugabyteDB plan in these runs used an index.

PostgreSQL, single node · run `select-order-existence-postgres-1-20261004` · tag `run/study-01/select-order-existence-postgres-1-20261004` `studies/01-charity-tree/results/select-order-existence-postgres-1-20261004/existence.json` · sha256 `a1cd2e20e7fd50961ed5562a099cd46e5b993e69847f8822daf4e3696d74f042`

EXISTS, fanout 100

SQL `s06_exists_parent` · parameters `pred=amount > 100`

```
SELECT c.id, c.name FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id AND (amount > 100));
```

Plan · `family=membership;stmt=exists;fanout=100` · 8 of 13 lines kept

```
Nested Loop Semi Join (actual time=0.007..0.079 rows=99 loops=1)
  ->  Seq Scan on customers c (actual time=0.003..0.006 rows=100 loops=1)
  ->  Index Scan using orders_customer_idx on orders o (actual time=0.001..0.001 rows=1 loops=100)
        Index Cond: (customer_id = c.id)
        Filter: (amount > 100)
        Rows Removed by Filter: 0
Planning Time: 0.070 ms
Execution Time: 0.087 ms
```

JOIN + DISTINCT, fanout 100

SQL `s08_join_distinct_parent` · parameters `pred=amount > 100`

```
SELECT DISTINCT c.id, c.name FROM customers c
JOIN orders o ON o.customer_id = c.id WHERE (amount > 100);
```

Plan · `family=membership;stmt=join_distinct;fanout=100` · 11 of 20 lines kept

```
HashAggregate (actual time=1.429..1.435 rows=99 loops=1)
  Group Key: c.id, c.name
  ->  Hash Join (actual time=0.015..0.923 rows=7240 loops=1)
        Hash Cond: (o.customer_id = c.id)
        ->  Seq Scan on orders o (actual time=0.003..0.394 rows=7314 loops=1)
              Filter: (amount > 100)
              Rows Removed by Filter: 2686
        ->  Hash (actual time=0.010..0.010 rows=100 loops=1)
              ->  Seq Scan on customers c (actual time=0.001..0.005 rows=100 loops=1)
Planning Time: 0.057 ms
Execution Time: 1.446 ms
```

YugabyteDB, single node · run `select-order-existence-yugabyte-1-20261004` · tag `run/study-01/select-order-existence-yugabyte-1-20261004` `studies/01-charity-tree/results/select-order-existence-yugabyte-1-20261004/existence.json` · sha256 `b8e15f58e4e81a72c956ddd1e23bb656a932cf3a33ce8b41f59111be6fe9a82c`

EXISTS, fanout 100

SQL `s06_exists_parent` · parameters `pred=amount > 100`

```
SELECT c.id, c.name FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id AND (amount > 100));
```

Plan · `family=membership;stmt=exists;fanout=100` · 10 of 13 lines kept

```
Hash Join (actual time=4.554..4.580 rows=99 loops=1)
  Hash Cond: (o.customer_id = c.id)
  ->  HashAggregate (actual time=4.240..4.246 rows=100 loops=1)
        Group Key: o.customer_id
        ->  Seq Scan on orders o (actual time=0.623..2.969 rows=7314 loops=1)
              Storage Filter: (amount > 100)
  ->  Hash (actual time=0.308..0.309 rows=100 loops=1)
        ->  Seq Scan on customers c (actual time=0.267..0.279 rows=100 loops=1)
Planning Time: 0.156 ms
Execution Time: 4.632 ms
```

JOIN + DISTINCT, fanout 100

SQL `s08_join_distinct_parent` · parameters `pred=amount > 100`

```
SELECT DISTINCT c.id, c.name FROM customers c
JOIN orders o ON o.customer_id = c.id WHERE (amount > 100);
```

Plan · `family=membership;stmt=join_distinct;fanout=100` · 10 of 13 lines kept

```
HashAggregate (actual time=4.977..4.985 rows=99 loops=1)
  Group Key: c.id, c.name
  ->  Hash Join (actual time=1.248..3.518 rows=7240 loops=1)
        Hash Cond: (o.customer_id = c.id)
        ->  Seq Scan on orders o (actual time=0.741..1.649 rows=7314 loops=1)
              Storage Filter: (amount > 100)
        ->  Hash (actual time=0.495..0.496 rows=100 loops=1)
              ->  Seq Scan on customers c (actual time=0.457..0.468 rows=100 loops=1)
Planning Time: 0.168 ms
Execution Time: 5.042 ms
```

IN against EXISTS

`v6-sel-06-membership-in-exists-no-difference` · **repeated — no prior prediction** · card, p. 102

PostgreSQL turned IN into the same semi-join as EXISTS: the two plans have the same node tree.

PostgreSQL, single node · run `select-order-existence-postgres-1-20261004` · tag `run/study-01/select-order-existence-postgres-1-20261004` `studies/01-charity-tree/results/select-order-existence-postgres-1-20261004/existence.json` · sha256 `a1cd2e20e7fd50961ed5562a099cd46e5b993e69847f8822daf4e3696d74f042`

IN, fanout 100 SQL `s07_in_parent` · parameters `pred=amount > 100`

```
SELECT c.id, c.name FROM customers c
WHERE c.id IN (SELECT o.customer_id FROM orders o WHERE (amount > 100));
```

Plan · `family=membership;stmt=in;fanout=100` · 8 of 13 lines kept

```
Nested Loop Semi Join (actual time=0.006..0.072 rows=99 loops=1)
  ->  Seq Scan on customers c (actual time=0.002..0.006 rows=100 loops=1)
  ->  Index Scan using orders_customer_idx on orders o (actual time=0.001..0.001 rows=1 loops=100)
        Index Cond: (customer_id = c.id)
        Filter: (amount > 100)
        Rows Removed by Filter: 0
Planning Time: 0.066 ms
Execution Time: 0.080 ms
```

Checked by the generator. IN (`s07_in_parent`): same node tree as "EXISTS, fanout 100".

IN SQL `s07_in_parent`
Same SQL as in the PostgreSQL, single node exhibit above.

NOT IN over a nullable key

`v6-sel-07-nullable-not-in-empty` · **replicated** · card, p. 102

No exhibit. The NOT IN over a nullable child key is a negative control: the harness records the rows it returned (none, at every fanout), not its plan. NOT IN over a NOT NULL subquery is shown in the next exhibit.

Anti-join spellings on PostgreSQL

`v6-sel-08-anti-join-not-exists-postgresql` · **repeated — no prior prediction** · card, p. 103

NOT EXISTS becomes a `Nested Loop Anti Join` that probes the order index once for each of the 100 parents. LEFT JOIN ... IS NULL hash-joins all 7,314 qualifying orders and then discards the 7,240 joined rows. NOT IN over a NOT NULL subquery loads the qualifying orders into a hashed subplan. Both alternatives read every qualifying order.

PostgreSQL, single node · run `select-order-existence-postgres-1-20261004` · tag `run/study-01/select-order-existence-postgres-1-20261004` `studies/01-charity-tree/results/select-order-existence-postgres-1-20261004/existence.json` · sha256 `a1cd2e20e7fd50961ed5562a099cd46e5b993e69847f8822daf4e3696d74f042`

NOT EXISTS, fanout 100 SQL `s09_not_exists` · parameters `pred=amount > 100`

```
SELECT c.id, c.name FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id AND (amount > 100));
```

Plan · `family=absence;stmt=not_exists;fanout=100` · 8 of 13 lines kept

```
Nested Loop Anti Join (actual time=0.068..0.068 rows=1 loops=1)
  ->  Seq Scan on customers c (actual time=0.002..0.005 rows=100 loops=1)
  ->  Index Scan using orders_customer_idx on orders o (actual time=0.001..0.001 rows=1 loops=100)
        Index Cond: (customer_id = c.id)
        Filter: (amount > 100)
        Rows Removed by Filter: 0
Planning Time: 0.052 ms
Execution Time: 0.073 ms
```

LEFT JOIN ... IS NULL, fanout 100 SQL `s10_left_anti` · parameters `pred=amount > 100`

```
SELECT c.id, c.name FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id AND (amount > 100)
WHERE o.id IS NULL;
```

Plan · `family=absence;stmt=left_is_null;fanout=100` · 11 of 18 lines kept

```
Hash Right Join (actual time=0.734..0.735 rows=1 loops=1)
  Hash Cond: (o.customer_id = c.id)
  Filter: (o.id IS NULL)
  Rows Removed by Filter: 7240
  ->  Seq Scan on orders o (actual time=0.002..0.372 rows=7314 loops=1)
        Filter: (amount > 100)
        Rows Removed by Filter: 2686
  ->  Hash (actual time=0.012..0.012 rows=100 loops=1)
        ->  Seq Scan on customers c (actual time=0.004..0.007 rows=100 loops=1)
Planning Time: 0.063 ms
Execution Time: 0.742 ms
```

NOT IN over a NOT NULL subquery, fanout 100 SQL `e13_not_in_notnull` · parameters `pred=amount > 100`

```
SELECT c.id, c.name FROM customers c
WHERE c.id NOT IN (SELECT o.customer_id FROM orders o
                   WHERE o.customer_id IS NOT NULL AND (amount > 100));
```

Plan · `family=absence;stmt=not_in_notnull;fanout=100` · 9 of 11 lines kept

```
Seq Scan on customers c (actual time=0.852..0.852 rows=1 loops=1)
  Filter: (NOT (ANY (id = (hashed SubPlan 1).col1)))
  Rows Removed by Filter: 99
  SubPlan 1
    ->  Seq Scan on orders o (actual time=0.003..0.437 rows=7240 loops=1)
          Filter: ((customer_id IS NOT NULL) AND (amount > 100))
          Rows Removed by Filter: 2760
Planning Time: 0.035 ms
Execution Time: 0.860 ms
```

Totals and the top row per parent

Correlated SUM on PostgreSQL

`v6-sel-09-aggregate-correlated-postgresql-crossover` · **replicated** · card, p. 103

The correlated spelling runs two scalar subqueries per parent (`SubPlan 1` and `SubPlan 2`), each an index probe. For one parent that is two probes. For 10,000 parents each subplan runs 10,000 times (`loops=10000`). Join-then-GROUP-BY instead reads the 20,000 orders once and aggregates them in a `HashAggregate`.

PostgreSQL, single node · run `select-order-aggregate-postgres-1-20261004` · tag `run/study-01/select-order-aggregate-postgres-1-20261004` `studies/01-charity-tree/results/select-order-aggregate-postgres-1-20261004/aggregate.json` · sha256 `76268c647e965cd579b85330b32ccdf716c959230c56fe0d71216ce23ab39dc2`

correlated scalar SUM, one parent SQL `a03_aggregate_correlated_sel` · parameters `parent_max=1`

```
SELECT c.id, c.name,
       COALESCE((SELECT SUM(o.amount) FROM orders o WHERE o.customer_id = c.id), 0) AS total,
       (SELECT COUNT(*) FROM orders o WHERE o.customer_id = c.id) AS children
FROM customers c
WHERE 1 = 0 OR c.id <= 1;
```

Plan · `family=aggregate;stmt=correlated;parent_max=1` · 13 of 20 lines kept

```
Index Scan using customers_pkey on customers c (actual time=0.006..0.007 rows=1 loops=1)
  Index Cond: (id <= 1)
  SubPlan 1
    ->  Aggregate (actual time=0.002..0.003 rows=1 loops=1)
          ->  Index Scan using orders_customer_idx on orders o (actual time=0.001..0.001 rows=2 loops=1)
                Index Cond: (customer_id = c.id)
  SubPlan 2
    ->  Aggregate (actual time=0.002..0.002 rows=1 loops=1)
          ->  Index Only Scan using orders_customer_idx on orders o_1 (actual time=0.001..0.001 rows=2 loops=1)
                Index Cond: (customer_id = c.id)
                Heap Fetches: 2
Planning Time: 0.042 ms
Execution Time: 0.015 ms
```

join then GROUP BY, 10,000 parents SQL `a01_aggregate_join_sel` · parameters `parent_max=10000`

```
SELECT c.id, c.name, COALESCE(SUM(o.amount), 0) AS total, COUNT(o.id) AS children
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
WHERE 10000 = 0 OR c.id <= 10000
GROUP BY c.id, c.name;
```

Plan · `family=aggregate;stmt=post_join;parent_max=10000` · 10 of 19 lines kept

```
HashAggregate (actual time=4.552..5.101 rows=10000 loops=1)
  Group Key: c.id
  ->  Hash Right Join (actual time=0.782..3.088 rows=19936 loops=1)
        Hash Cond: (o.customer_id = c.id)
        ->  Seq Scan on orders o (actual time=0.002..0.629 rows=20000 loops=1)
        ->  Hash (actual time=0.773..0.774 rows=10000 loops=1)
              ->  Seq Scan on customers c (actual time=0.005..0.415 rows=10000 loops=1)
                    Filter: (id <= 10000)
Planning Time: 0.221 ms
Execution Time: 5.316 ms
```

correlated scalar SUM, 10,000 parents SQL `a03_aggregate_correlated_sel` · parameters `parent_max=10000`

```
SELECT c.id, c.name,
       COALESCE((SELECT SUM(o.amount) FROM orders o WHERE o.customer_id = c.id), 0) AS total,
       (SELECT COUNT(*) FROM orders o WHERE o.customer_id = c.id) AS children
FROM customers c
WHERE 10000 = 0 OR c.id <= 10000;
```

Plan · `family=aggregate;stmt=correlated;parent_max=10000` · 13 of 20 lines kept

```
Seq Scan on customers c (actual time=0.020..16.428 rows=10000 loops=1)
  Filter: (id <= 10000)
  SubPlan 1
    ->  Aggregate (actual time=0.001..0.001 rows=1 loops=10000)
          ->  Index Scan using orders_customer_idx on orders o (actual time=0.000..0.000 rows=2 loops=10000)
```

```
               Index Cond: (customer_id = c.id)
  SubPlan 2
    ->  Aggregate (actual time=0.001..0.001 rows=1 loops=10000)
          ->  Index Only Scan using orders_customer_idx on orders o_1 (actual time=0.000..0.000 rows=2 loops=10000)
                Index Cond: (customer_id = c.id)
                Heap Fetches: 19936
Planning Time: 0.119 ms
Execution Time: 16.657 ms
```

Correlated SUM on YugabyteDB

v6-sel-10-aggregate-yugabytedb-no-reversal · **replicated** · card, p. 103

The structure is PostgreSQL's, with each aggregate split into a partial aggregate in storage and a `Finalize Aggregate`. In this execution each probe took about 0.44 ms (`SubPlan 1`) and 0.22 ms (`SubPlan 2`), against below 0.001 ms on PostgreSQL, and the 20,000 probes took 6.8 s in all. The plan was captured without `DIST`, so it shows no storage requests, and the claim's per-request explanation remains a hypothesis.

YugabyteDB, single node · run `select-order-aggregate-yugabyte-1-20261004` · tag `run/study-01/select-order-aggregate-yugabyte-1-20261004` `studies/01-charity-tree/results/select-order-aggregate-yugabyte-1-20261004/aggregate.json` · sha256 `21ad333d1e4ff8a9da998c1a748ec21beca168a49d122da6c11658dcb9ced477`

join then GROUP BY, 10,000 parents SQL `a01_aggregate_join_sel` · parameters `parent_max=10000`

```
SELECT c.id, c.name, COALESCE(SUM(o.amount), 0) AS total, COUNT(o.id) AS children
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
WHERE 10000 = 0 OR c.id <= 10000
GROUP BY c.id, c.name;
```

Plan · `family=aggregate;stmt=post_join;parent_max=10000` · 10 of 13 lines kept

```
HashAggregate (actual time=12.898..13.552 rows=10000 loops=1)
  Group Key: c.id
  ->  Hash Right Join (actual time=4.527..9.567 rows=19936 loops=1)
        Hash Cond: (o.customer_id = c.id)
        ->  Seq Scan on orders o (actual time=0.372..2.095 rows=20000 loops=1)
        ->  Hash (actual time=4.142..4.143 rows=10000 loops=1)
              ->  Seq Scan on customers c (actual time=0.775..2.882 rows=10000 loops=1)
                    Storage Filter: (id <= 10000)
Planning Time: 0.189 ms
Execution Time: 14.456 ms
```

correlated scalar SUM, 10,000 parents SQL `a03_aggregate_correlated_sel` · parameters `parent_max=10000`

```
SELECT c.id, c.name,
       COALESCE((SELECT SUM(o.amount) FROM orders o WHERE o.customer_id = c.id), 0) AS total,
       (SELECT COUNT(*) FROM orders o WHERE o.customer_id = c.id) AS children
FROM customers c
WHERE 10000 = 0 OR c.id <= 10000;
```

Plan · `family=aggregate;stmt=correlated;parent_max=10000` · 15 of 16 lines kept

```
Seq Scan on customers c (actual time=1.676..6845.886 rows=10000 loops=1)
  Storage Filter: (id <= 10000)
  SubPlan 1
    ->  Finalize Aggregate (actual time=0.449..0.449 rows=1 loops=10000)
          ->  Index Scan using orders_customer_idx on orders o (actual time=0.435..0.437 rows=1 loops=10000)
                Index Cond: (customer_id = c.id)
                Partial Aggregate: true
  SubPlan 2
    ->  Finalize Aggregate (actual time=0.232..0.232 rows=1 loops=10000)
          ->  Index Only Scan using orders_customer_idx on orders o_1 (actual time=0.220..0.221 rows=1 loops=10000)
                Index Cond: (customer_id = c.id)
                Heap Fetches: 0
                Partial Aggregate: true
Planning Time: 0.121 ms
Execution Time: 6848.900 ms
```

Pre-aggregating derived table

v6-sel-18-aggregate-preagg-derived-table · **replicated** · card, p. 106

For one parent, join-then-GROUP-BY reads that parent's two orders through the index. The pre-aggregating derived table first groups all 20,000 orders into 10,001 groups (`HashAggregate`) and only then joins the one parent: the parent filter does not reach inside the derived table. At 10,000 parents both forms read every order once. YugabyteDB's plan at 10,000 parents also aggregates all orders before the join.

PostgreSQL, single node · run `select-order-aggregate-postgres-1-20261004` · tag `run/study-01/select-order-aggregate-postgres-1-20261004` `studies/01-charity-tree/results/select-order-aggregate-postgres-1-20261004/aggregate.json` · sha256 `76268c647e965cd579b85330b32ccdf716c959230c56fe0d71216ce23ab39dc2`

join then GROUP BY, one parent

SQL `a01_aggregate_join_sel` · parameters `parent_max=1`

```
SELECT c.id, c.name, COALESCE(SUM(o.amount), 0) AS total, COUNT(o.id) AS children
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
WHERE 1 = 0 OR c.id <= 1
GROUP BY c.id, c.name;
```

Plan · `family=aggregate;stmt=post_join;parent_max=1` · 9 of 15 lines kept

```
GroupAggregate (actual time=0.006..0.007 rows=1 loops=1)
  Group Key: c.id
  ->  Nested Loop Left Join (actual time=0.003..0.004 rows=2 loops=1)
        ->  Index Scan using customers_pkey on customers c (actual time=0.001..0.001 rows=1 loops=1)
              Index Cond: (id <= 1)
        ->  Index Scan using orders_customer_idx on orders o (actual time=0.001..0.002 rows=2 loops=1)
              Index Cond: (customer_id = c.id)
Planning Time: 0.092 ms
Execution Time: 0.016 ms
```

pre-aggregating derived table, one parent

SQL `a02_aggregate_preagg_sel` · parameters `parent_max=1`

```
SELECT c.id, c.name, COALESCE(a.total, 0) AS total, COALESCE(a.children, 0) AS children
FROM customers c
LEFT JOIN (SELECT customer_id, SUM(amount) AS total, COUNT(*) AS children
           FROM orders GROUP BY customer_id) a ON a.customer_id = c.id
WHERE 1 = 0 OR c.id <= 1;
```

Plan · `family=aggregate;stmt=pre_agg;parent_max=1` · 10 of 19 lines kept

```
Hash Right Join (actual time=3.299..3.563 rows=1 loops=1)
  Hash Cond: (orders.customer_id = c.id)
  ->  HashAggregate (actual time=2.570..3.205 rows=10001 loops=1)
        Group Key: orders.customer_id
        ->  Seq Scan on orders (actual time=0.003..0.737 rows=20000 loops=1)
  ->  Hash (actual time=0.002..0.003 rows=1 loops=1)
        ->  Index Scan using customers_pkey on customers c (actual time=0.001..0.002 rows=1 loops=1)
              Index Cond: (id <= 1)
Planning Time: 0.067 ms
Execution Time: 3.582 ms
```

pre-aggregating derived table, 10,000 parents

SQL `a02_aggregate_preagg_sel` · parameters `parent_max=10000`

```
SELECT c.id, c.name, COALESCE(a.total, 0) AS total, COALESCE(a.children, 0) AS children
FROM customers c
LEFT JOIN (SELECT customer_id, SUM(amount) AS total, COUNT(*) AS children
           FROM orders GROUP BY customer_id) a ON a.customer_id = c.id
WHERE 10000 = 0 OR c.id <= 10000;
```

Plan · `family=aggregate;stmt=pre_agg;parent_max=10000` · 10 of 19 lines kept

```
Hash Right Join (actual time=2.903..4.406 rows=10000 loops=1)
  Hash Cond: (orders.customer_id = c.id)
  ->  HashAggregate (actual time=2.137..2.722 rows=10001 loops=1)
        Group Key: orders.customer_id
        ->  Seq Scan on orders (actual time=0.004..0.644 rows=20000 loops=1)
  ->  Hash (actual time=0.760..0.760 rows=10000 loops=1)
        ->  Seq Scan on customers c (actual time=0.007..0.392 rows=10000 loops=1)
              Filter: (id <= 10000)
Planning Time: 0.113 ms
Execution Time: 4.630 ms
```

YugabyteDB, single node · run `select-order-aggregate-yugabyte-1-20261004` · tag `run/study-01/select-order-aggregate-yugabyte-1-20261004` `studies/01-charity-tree/results/select-order-aggregate-yugabyte-1-20261004/aggregate.json` · sha256 `21ad333d1e4ff8a9da998c1a748ec21beca168a49d122da6c11658dcb9ced477`

pre-aggregating derived table, 10,000 parents

SQL `a02_aggregate_preagg_sel` · parameters `parent_max=10000`

```
SELECT c.id, c.name, COALESCE(a.total, 0) AS total, COALESCE(a.children, 0) AS children
FROM customers c
LEFT JOIN (SELECT customer_id, SUM(amount) AS total, COUNT(*) AS children
           FROM orders GROUP BY customer_id) a ON a.customer_id = c.id
WHERE 10000 = 0 OR c.id <= 10000;
```

Plan · `family=aggregate;stmt=pre_agg;parent_max=10000` · 11 of 14 lines kept

```
Hash Left Join (actual time=11.732..15.070 rows=10000 loops=1)
  Hash Cond: (c.id = a.customer_id)
  ->  Seq Scan on customers c (actual time=0.783..2.511 rows=10000 loops=1)
        Storage Filter: (id <= 10000)
  ->  Hash (actual time=10.931..10.932 rows=10000 loops=1)
        ->  Subquery Scan on a (actual time=8.120..9.768 rows=10001 loops=1)
              ->  HashAggregate (actual time=8.118..8.732 rows=10001 loops=1)
                    Group Key: orders.customer_id
                    ->  Seq Scan on orders (actual time=0.691..4.845 rows=20000 loops=1)
Planning Time: 0.129 ms
Execution Time: 15.994 ms
```

LATERAL probe against ROW_NUMBER()

v6-sel-11-topn-lateral-probe-crossover · **replicated** · card, p. 104

LATERAL runs an index probe, a sort and a limit for each parent: once for one parent, and 10,000 times for 10,000 parents (`loops=10000`, about 0.47 ms each on YugabyteDB). ROW_NUMBER() reads the 19,936 orders once, sorts them, and keeps the first row of each parent (`Run Condition`).

YugabyteDB, single node · run `select-order-aggregate-yugabyte-1-20261004` · tag `run/study-01/select-order-aggregate-yugabyte-1-20261004` `studies/01-charity-tree/results/select-order-aggregate-yugabyte-1-20261004/aggregate.json` · sha256 `21ad333d1e4ff8a9da998c1a748ec21beca168a49d122da6c11658dcb9ced477`

LATERAL probe, one parent

SQL `a05_topn_lateral_sel` · parameters `n=1,parent_max=1`

```
SELECT c.id, o.id, o.amount
FROM customers c
CROSS JOIN LATERAL (
  SELECT o.id, o.amount FROM orders o
  WHERE o.customer_id = c.id
  ORDER BY o.amount DESC NULLS LAST, o.id ASC
  LIMIT 1) o
WHERE 1 = 0 OR c.id <= 1
ORDER BY c.id ASC, o.amount DESC NULLS LAST, o.id ASC;
```

Plan · `family=topn;stmt=lateral;parent_max=1` · 14 of 15 lines kept

```
Sort (actual time=2.367..2.368 rows=1 loops=1)
  Sort Key: c.id, o_1.amount DESC NULLS LAST, o_1.id
  Sort Method: quicksort  Memory: 25kB
  ->  Nested Loop (actual time=2.357..2.361 rows=1 loops=1)
        ->  Seq Scan on customers c (actual time=1.901..1.903 rows=1 loops=1)
              Storage Filter: (id <= 1)
        ->  Limit (actual time=0.452..0.454 rows=1 loops=1)
              ->  Sort (actual time=0.451..0.452 rows=1 loops=1)
                    Sort Key: o_1.amount DESC NULLS LAST, o_1.id
                    Sort Method: quicksort  Memory: 25kB
                    ->  Index Scan using orders_customer_idx on orders o_1 (actual time=0.426..0.436 rows=2 loops=1)
                          Index Cond: (customer_id = c.id)
Planning Time: 0.099 ms
Execution Time: 2.422 ms
```

ROW_NUMBER(), 10,000 parents

SQL `a04_topn_rownumber_sel` · parameters `n=1,parent_max=10000`

```
SELECT customer_id, id, amount FROM (
  SELECT o.customer_id, o.id, o.amount,
         ROW_NUMBER() OVER (PARTITION BY o.customer_id
                            ORDER BY o.amount DESC NULLS LAST, o.id ASC) AS rn
  FROM orders o
  WHERE o.customer_id IS NOT NULL
    AND (10000 = 0 OR o.customer_id <= 10000)) t
WHERE rn <= 1
ORDER BY customer_id ASC, amount DESC NULLS LAST, id ASC;
```

Plan · `family=topn;stmt=rownumber;parent_max=10000` · 10 of 11 lines kept

```
Subquery Scan on t (actual time=9.878..33.715 rows=10000 loops=1)
  ->  WindowAgg (actual time=9.876..32.501 rows=10000 loops=1)
        Run Condition: (row_number() OVER (?) <= 1)
        ->  Sort (actual time=9.859..10.919 rows=19936 loops=1)
              Sort Key: o.customer_id, o.amount DESC NULLS LAST, o.id
              Sort Method: quicksort  Memory: 1853kB
              ->  Seq Scan on orders o (actual time=0.955..5.825 rows=19936 loops=1)
                    Storage Filter: ((customer_id IS NOT NULL) AND (customer_id <= 10000))
Planning Time: 0.110 ms
Execution Time: 34.784 ms
```

LATERAL probe, 10,000 parents

SQL `a05_topn_lateral_sel` · parameters `n=1,parent_max=10000`

```
SELECT c.id, o.id, o.amount
FROM customers c
CROSS JOIN LATERAL (
  SELECT o.id, o.amount FROM orders o
  WHERE o.customer_id = c.id
  ORDER BY o.amount DESC NULLS LAST, o.id ASC
  LIMIT 1) o
WHERE 10000 = 0 OR c.id <= 10000
ORDER BY c.id ASC, o.amount DESC NULLS LAST, o.id ASC;
```

Plan · `family=topn;stmt=lateral;parent_max=10000` · 14 of 15 lines kept

```
Sort (actual time=4757.319..4758.330 rows=10000 loops=1)
  Sort Key: c.id, o_1.amount DESC NULLS LAST, o_1.id
  Sort Method: quicksort  Memory: 931kB
  ->  Nested Loop (actual time=1.650..4752.536 rows=10000 loops=1)
        ->  Seq Scan on customers c (actual time=0.934..6.951 rows=10000 loops=1)
              Storage Filter: (id <= 10000)
        ->  Limit (actual time=0.473..0.473 rows=1 loops=10000)
              ->  Sort (actual time=0.472..0.472 rows=1 loops=10000)
                    Sort Key: o_1.amount DESC NULLS LAST, o_1.id
                    Sort Method: quicksort  Memory: 25kB
                    ->  Index Scan using orders_customer_idx on orders o_1 (actual time=0.453..0.455 rows=2 loops=10000)
                          Index Cond: (customer_id = c.id)
```

```
Planning Time: 0.226 ms
Execution Time: 4759.134 ms
```

DISTINCT ON on PostgreSQL

v6-sel-12-topn-distinct-on-postgresql · **replicated** · card, p. 104

Both spellings read the same 19,936 orders through `orders_customer_idx`, already in customer order, and add an `Incremental Sort`. DISTINCT ON then keeps the first row per customer with `Unique`. ROW_NUMBER() computes a window aggregate and filters it in a subquery.

PostgreSQL, single node · run `select-order-aggregate-postgres-1-20261004` · tag `run/study-01/select-order-aggregate-postgres-1-20261004`
`studies/01-charity-tree/results/select-order-aggregate-postgres-1-20261004/aggregate.json` · sha256 `76268c647e965cd579b85330b32ccdf716c959230c56fe0d71216ce23ab39dc2`

ROW_NUMBER(), 10,000 parents SQL `a04_topn_rownumber_sel` · parameters `n=1,parent_max=10000`

```
SELECT customer_id, id, amount FROM (
  SELECT o.customer_id, o.id, o.amount,
         ROW_NUMBER() OVER (PARTITION BY o.customer_id
                            ORDER BY o.amount DESC NULLS LAST, o.id ASC) AS rn
  FROM orders o
  WHERE o.customer_id IS NOT NULL
    AND (10000 = 0 OR o.customer_id <= 10000)) t
WHERE rn <= 1
ORDER BY customer_id ASC, amount DESC NULLS LAST, id ASC;
```

Plan · `family=topn;stmt=rownumber;parent_max=10000` · 10 of 17 lines kept

```
Subquery Scan on t (actual time=0.023..6.054 rows=10000 loops=1)
  ->  WindowAgg (actual time=0.023..5.624 rows=10000 loops=1)
        Run Condition: (row_number() OVER (?) <= 1)
        ->  Incremental Sort (actual time=0.019..3.171 rows=19936 loops=1)
              Sort Key: o.customer_id, o.amount DESC NULLS LAST, o.id
              Presorted Key: o.customer_id
              ->  Index Scan using orders_customer_idx on orders o (actual time=0.007..1.373 rows=19936 loops=1)
                    Index Cond: ((customer_id IS NOT NULL) AND (customer_id <= 10000))
Planning Time: 0.123 ms
Execution Time: 6.264 ms
```

DISTINCT ON, 10,000 parents SQL `a06_topn_distinct_on_sel` · parameters `parent_max=10000`

```
SELECT DISTINCT ON (o.customer_id) o.customer_id, o.id, o.amount
FROM orders o
WHERE o.customer_id IS NOT NULL
  AND (10000 = 0 OR o.customer_id <= 10000)
ORDER BY o.customer_id ASC, o.amount DESC NULLS LAST, o.id ASC;
```

Plan · `family=topn;stmt=distinct_on;parent_max=10000` · 8 of 14 lines kept

```
Unique (actual time=0.019..3.906 rows=10000 loops=1)
  ->  Incremental Sort (actual time=0.019..3.048 rows=19936 loops=1)
        Sort Key: customer_id, amount DESC NULLS LAST, id
        Presorted Key: customer_id
        ->  Index Scan using orders_customer_idx on orders o (actual time=0.007..1.418 rows=19936 loops=1)
              Index Cond: ((customer_id IS NOT NULL) AND (customer_id <= 10000))
Planning Time: 0.092 ms
Execution Time: 4.137 ms
```

Predicates and pagination

OR against UNION ALL

v6-sel-13-or-not-beaten-by-union · **replicated** · card, p. 104

With selective arms, PostgreSQL serves OR with one `BitmapOr` over two index scans. UNION ALL runs the two bitmap scans separately and appends them before the sort. With broad arms, OR reads the primary-key index in id order and filters, while UNION ALL runs two index scans and sorts their 4,652 rows. On YugabyteDB the hash-sharded amount index cannot serve a range, so OR is one sequential scan with the predicate pushed to storage (`Storage Filter`).

PostgreSQL, single node · run `select-order-access-postgres-1-20261004` · tag `run/study-01/select-order-access-postgres-1-20261004`
`studies/01-charity-tree/results/select-order-access-postgres-1-20261004/access.json` · sha256 `0783debd960ffb0e76f9cf0771a11a3c065e433a6e1efa11519f06447204a0f4`

OR, selective indexable arms SQL `s18_or_predicate` · parameters `pred_a=amount <= 5,pred_b=amount >= 496`

```
SELECT id, amount FROM orders WHERE (amount <= 5) OR (amount >= 496) ORDER BY id;
```

Plan · `shape=selective/indexable;spelling=or` · 12 of 20 lines kept

```
Sort (actual time=0.062..0.068 rows=251 loops=1)
  Sort Key: id
```

```
  Sort Method: quicksort  Memory: 30kB
  ->  Bitmap Heap Scan on orders (actual time=0.013..0.047 rows=251 loops=1)
        Recheck Cond: ((amount <= 5) OR (amount >= 496))
        ->  BitmapOr (actual time=0.006..0.007 rows=0 loops=1)
              ->  Bitmap Index Scan on orders_amount_idx (actual time=0.004..0.004 rows=114 loops=1)
                    Index Cond: (amount <= 5)
              ->  Bitmap Index Scan on orders_amount_idx (actual time=0.003..0.003 rows=137 loops=1)
                    Index Cond: (amount >= 496)
Planning Time: 0.043 ms
Execution Time: 0.080 ms
```

UNION ALL, selective indexable arms SQL `s19_union_all_predicate` · parameters `pred_a=amount <= 5,pred_b=amount >= 496`

```
SELECT id, amount FROM orders WHERE (amount <= 5)
UNION ALL
SELECT id, amount FROM orders WHERE (amount >= 496)
ORDER BY id;
```

Plan · `shape=selective/indexable;spelling=union_all` · 14 of 24 lines kept

```
Sort (actual time=0.098..0.109 rows=251 loops=1)
  Sort Key: orders.id
  Sort Method: quicksort  Memory: 30kB
  ->  Append (actual time=0.011..0.076 rows=251 loops=1)
        ->  Bitmap Heap Scan on orders (actual time=0.011..0.034 rows=114 loops=1)
              Recheck Cond: (amount <= 5)
              ->  Bitmap Index Scan on orders_amount_idx (actual time=0.004..0.005 rows=114 loops=1)
                    Index Cond: (amount <= 5)
        ->  Bitmap Heap Scan on orders orders_1 (actual time=0.008..0.030 rows=137 loops=1)
              Recheck Cond: (amount >= 496)
              ->  Bitmap Index Scan on orders_amount_idx (actual time=0.004..0.004 rows=137 loops=1)
                    Index Cond: (amount >= 496)
Planning Time: 0.063 ms
Execution Time: 0.124 ms
```

OR, broad arms SQL `s18_or_predicate` · parameters `pred_a=amount <= 100,pred_b=amount >= 400`

```
SELECT id, amount FROM orders WHERE (amount <= 100) OR (amount >= 400) ORDER BY id;
```

Plan · `shape=broad/indexable;spelling=or` · 5 of 6 lines kept

```
Index Scan using orders_pkey on orders (actual time=0.011..0.773 rows=4652 loops=1)
  Filter: ((amount <= 100) OR (amount >= 400))
  Rows Removed by Filter: 7348
Planning Time: 0.053 ms
Execution Time: 0.883 ms
```

UNION ALL, broad arms SQL `s19_union_all_predicate` · parameters `pred_a=amount <= 100,pred_b=amount >= 400`

```
SELECT id, amount FROM orders WHERE (amount <= 100)
UNION ALL
SELECT id, amount FROM orders WHERE (amount >= 400)
ORDER BY id;
```

Plan · `shape=broad/indexable;spelling=union_all` · 10 of 14 lines kept

```
Sort (actual time=1.233..1.345 rows=4652 loops=1)
  Sort Key: orders.id
  Sort Method: quicksort  Memory: 302kB
  ->  Append (actual time=0.008..0.767 rows=4652 loops=1)
        ->  Index Scan using orders_amount_idx on orders (actual time=0.008..0.306 rows=2273 loops=1)
              Index Cond: (amount <= 100)
        ->  Index Scan using orders_amount_idx on orders orders_1 (actual time=0.004..0.272 rows=2379 loops=1)
              Index Cond: (amount >= 400)
Planning Time: 0.053 ms
Execution Time: 1.451 ms
```

YugabyteDB, single node · run `select-order-access-yugabyte-1-20261004` · tag `run/study-01/select-order-access-yugabyte-1-20261004` `studies/01-charity-tree/results/select-order-access-yugabyte-1-20261004/access.json` · sha256 `9a3b8b777b99388a1cb983808a53bb0f976aad693cb14e7dd4f4fe8da8b99d7f`

OR, selective indexable arms SQL `s18_or_predicate` · parameters `pred_a=amount <= 5,pred_b=amount >= 496`

```
SELECT id, amount FROM orders WHERE (amount <= 5) OR (amount >= 496) ORDER BY id;
```

Plan · `shape=selective/indexable;spelling=or` · 7 of 8 lines kept

```
Sort (actual time=3.625..3.636 rows=251 loops=1)
  Sort Key: id
  Sort Method: quicksort  Memory: 36kB
  ->  Seq Scan on orders (actual time=3.465..3.544 rows=251 loops=1)
        Storage Filter: ((amount <= 5) OR (amount >= 496))
Planning Time: 0.057 ms
Execution Time: 3.704 ms
```

Expression index

`v6-sel-14-expression-index-postgresql` · **repeated — no prior prediction** · card, p. 105

Without a matching index, the predicate on `amount + 0` is a filter over the primary-key scan, which removes 11,749 rows. With the expression index present, the same SQL becomes a `BitmapOr` over

`orders_amount_expr_idx`. For the broad shape the index changes nothing: its node tree is the same with or without the index.

PostgreSQL, single node · run `select-order-access-postgres-1-20261004` · tag `run/study-01/select-order-access-postgres-1-20261004` `studies/01-charity-tree/results/select-order-access-postgres-1-20261004/access.json` · sha256 `0783debd960ffb0e76f9cf0771a11a3c065e433a6e1efa11519f06447204a0f4`

selective OR on (amount + 0), no expression index SQL `s18_or_predicate` · parameters `pred_a=(amount + 0) <= 5,pred_b=(amount + 0) >= 496`

```
SELECT id, amount FROM orders WHERE ((amount + 0) <= 5) OR ((amount + 0) >= 496) ORDER BY id;
```

Plan · `shape=selective/expression;spelling=or` · 5 of 6 lines kept

```
Index Scan using orders_pkey on orders (actual time=0.008..0.617 rows=251 loops=1)
  Filter: (((amount + 0) <= 5) OR ((amount + 0) >= 496))
  Rows Removed by Filter: 11749
Planning Time: 0.025 ms
Execution Time: 0.628 ms
```

the same SQL with the expression index present SQL `s18_or_predicate` · parameters `pred_a=(amount + 0) <= 5,pred_b=(amount + 0) >= 496`

```
SELECT id, amount FROM orders WHERE ((amount + 0) <= 5) OR ((amount + 0) >= 496) ORDER BY id;
```

Plan · `shape=selective/expression;index=expression_index;spelling=or` · 12 of 20 lines kept

```
Sort (actual time=0.057..0.063 rows=251 loops=1)
  Sort Key: id
  Sort Method: quicksort  Memory: 30kB
  ->  Bitmap Heap Scan on orders (actual time=0.010..0.042 rows=251 loops=1)
        Recheck Cond: (((amount + 0) <= 5) OR ((amount + 0) >= 496))
        ->  BitmapOr (actual time=0.005..0.005 rows=0 loops=1)
              ->  Bitmap Index Scan on orders_amount_expr_idx (actual time=0.003..0.003 rows=114 loops=1)
                    Index Cond: ((amount + 0) <= 5)
              ->  Bitmap Index Scan on orders_amount_expr_idx (actual time=0.002..0.002 rows=137 loops=1)
                    Index Cond: ((amount + 0) >= 496)
Planning Time: 0.035 ms
Execution Time: 0.074 ms
```

Checked by the generator. broad expression shape with the index (`s18_or_predicate`): same node tree as `shape=broad/expression;spelling=or`.

broad expression shape with the index SQL `s18_or_predicate`

```
SELECT id, amount FROM orders WHERE ((amount + 0) <= 100) OR ((amount + 0) >= 400) ORDER BY id;
```

Compared plan, not quoted here SQL `s18_or_predicate`
Same SQL as in the PostgreSQL, single node exhibit above.

Keyset against OFFSET

`v6-sel-15-keyset-offset-crossover` · **replicated** · card, p. 105

At depth 10,000, OFFSET sorts all 12,000 orders and walks 10,005 sorted rows to return five. On PostgreSQL the keyset predicate becomes a three-arm `BitmapOr`, including the `amount IS NULL` arm, followed by a top-N heapsort over 2,000 rows. On YugabyteDB, keyset is a sequential scan with the predicate as a `Storage Filter`, also followed by a top-N heapsort.

PostgreSQL, single node · run `select-order-access-postgres-1-20261004` · tag `run/study-01/select-order-access-postgres-1-20261004` `studies/01-charity-tree/results/select-order-access-postgres-1-20261004/access.json` · sha256 `0783debd960ffb0e76f9cf0771a11a3c065e433a6e1efa11519f06447204a0f4`

OFFSET, page depth 10,000 SQL `s16_page_offset` · parameters `limit=5,offset=10000`

```
SELECT id, amount FROM orders ORDER BY amount DESC NULLS LAST, id ASC LIMIT 5 OFFSET 10000;
```

Plan · `depth=10000;spelling=offset` · 7 of 10 lines kept

```
Limit (actual time=2.489..2.489 rows=5 loops=1)
  ->  Sort (actual time=2.074..2.297 rows=10005 loops=1)
        Sort Key: amount DESC NULLS LAST, id
        Sort Method: quicksort  Memory: 853kB
        ->  Seq Scan on orders (actual time=0.005..0.451 rows=12000 loops=1)
Planning Time: 0.028 ms
Execution Time: 2.495 ms
```

keyset, page depth 10,000 SQL `s17_page_keyset` · parameters `cursor_set=1,last_amount=67,last_id=4310,last_is_null=0,limit=5`

```
SELECT id, amount FROM orders
WHERE 1 = 0
   OR (0 = 1 AND amount IS NULL AND id > 4310)
   OR (0 = 0 AND
       (amount < 67 OR (amount = 67 AND id > 4310) OR amount IS NULL))
ORDER BY amount DESC NULLS LAST, id ASC LIMIT 5;
```

Plan · `depth=10000;spelling=keyset` · 17 of 25 lines kept

```
Limit (actual time=0.329..0.330 rows=5 loops=1)
  ->  Sort (actual time=0.328..0.329 rows=5 loops=1)
        Sort Key: amount DESC NULLS LAST, id
        Sort Method: top-N heapsort  Memory: 25kB
        ->  Bitmap Heap Scan on orders (actual time=0.040..0.219 rows=2000 loops=1)
              Recheck Cond: ((amount < 67) OR (amount = 67) OR (amount IS NULL))
              Filter: ((amount < 67) OR ((amount = 67) AND (id > 4310)) OR (amount IS NULL))
              Rows Removed by Filter: 6
              ->  BitmapOr (actual time=0.031..0.031 rows=0 loops=1)
                    ->  Bitmap Index Scan on orders_amount_idx (actual time=0.023..0.023 rows=1460 loops=1)
                          Index Cond: (amount < 67)
                    ->  Bitmap Index Scan on orders_amount_idx (actual time=0.001..0.001 rows=24 loops=1)
                          Index Cond: (amount = 67)
                    ->  Bitmap Index Scan on orders_amount_idx (actual time=0.006..0.006 rows=522 loops=1)
                          Index Cond: (amount IS NULL)
Planning Time: 0.050 ms
Execution Time: 0.339 ms
```

YugabyteDB, single node · run `select-order-access-yugabyte-1-20261004` · tag `run/study-01/select-order-access-yugabyte-1-20261004` `studies/01-charity-tree/results/select-order-access-yugabyte-1-20261004/access.json` · sha256 `9a3b8b777b99388a1cb983808a53bb0f976aad693cb14e7dd4f4fe8da8b99d7f`

OFFSET, page depth 10,000 SQL `s16_page_offset` · parameters `limit=5,offset=10000`

```
SELECT id, amount FROM orders ORDER BY amount DESC NULLS LAST, id ASC LIMIT 5 OFFSET 10000;
```

Plan · `depth=10000;spelling=offset` · 7 of 8 lines kept

```
Limit (actual time=7.202..7.205 rows=5 loops=1)
  ->  Sort (actual time=5.852..6.254 rows=10005 loops=1)
        Sort Key: amount DESC NULLS LAST, id
        Sort Method: quicksort  Memory: 947kB
        ->  Seq Scan on orders (actual time=0.475..2.720 rows=12000 loops=1)
Planning Time: 0.050 ms
Execution Time: 7.246 ms
```

keyset, page depth 10,000 SQL `s17_page_keyset` · parameters `cursor_set=1,last_amount=67,last_id=4310,last_is_null=0,limit=5`

```
SELECT id, amount FROM orders
WHERE 1 = 0
   OR (0 = 1 AND amount IS NULL AND id > 4310)
   OR (0 = 0 AND
       (amount < 67 OR (amount = 67 AND id > 4310) OR amount IS NULL))
ORDER BY amount DESC NULLS LAST, id ASC LIMIT 5;
```

Plan · `depth=10000;spelling=keyset` · 8 of 9 lines kept

```
Limit (actual time=4.516..4.517 rows=5 loops=1)
  ->  Sort (actual time=4.514..4.515 rows=5 loops=1)
        Sort Key: amount DESC NULLS LAST, id
        Sort Method: top-N heapsort  Memory: 25kB
        ->  Seq Scan on orders (actual time=2.191..4.234 rows=2000 loops=1)
              Storage Filter: ((amount < 67) OR ((amount = 67) AND (id > 4310)) OR (amount IS NULL))
Planning Time: 0.094 ms
Execution Time: 4.553 ms
```

Placement on three nodes

Splitting into three tablets

`v6-sel-16-placement-splitting-effect` · **replicated** · card, p. 105

Placement did not change the plan: the single-tablet and the distributed runs have the same node tree. The difference is inside the donation scan. In these two executions it made 176 storage read requests in 80 ms on one tablet, and 59 requests in 17 ms across three tablets. These are single instrumented executions; the claim's ratios come from the timed samples.

YugabyteDB, three nodes (RF3) · run `select-join-v4-topo-tablet-i1-20261005` · tag `run/study-01/select-join-v4-topo-tablet-i1-20261005` `studies/01-charity-tree/results/select-join-v4-topo-tablet-i1-20261005/join.json` · sha256 `6122526384aa46083db9172dc68a117e37915dd37a9807571f85973be1ae4a3a`

single-tablet layout, 180,000-row join SQL `j01_join_where` · parameters `threshold=180000`

```
SELECT c.id AS customer_id, c.name AS customer_name, f.id AS fact_id, f.amount
FROM customers c JOIN facts f ON f.customer_id = c.id
WHERE f.rank <= 180000;
```

Plan · `family=bag;stmt=join_where;threshold=180000` · 20 of 92 lines kept

```
Hash Join (actual time=3.591..112.546 rows=180000 loops=1)
  Hash Cond: (d.person_id = p.person_id)
  ->  Seq Scan on donation d (actual time=1.475..79.753 rows=180000 loops=1)
        Storage Filter: (rank <= 180000)
        Storage Table Read Requests: 176
        Storage Table Rows Scanned: 200000
  ->  Hash (actual time=2.082..2.086 rows=2000 loops=1)
        ->  Hash Join (actual time=1.464..1.862 rows=2000 loops=1)
              Hash Cond: (p.charity_id = ch.charity_id)
              ->  Seq Scan on person p (actual time=0.884..1.018 rows=2000 loops=1)
                    Storage Table Read Requests: 2
                    Storage Table Rows Scanned: 2000
              ->  Hash (actual time=0.564..0.566 rows=50 loops=1)
                    ->  Seq Scan on charity ch (actual time=0.543..0.549 rows=50 loops=1)
                          Storage Table Read Requests: 1
                          Storage Table Rows Scanned: 50
Planning Time: 0.303 ms
Execution Time: 127.005 ms
Storage Read Requests: 179
Storage Rows Scanned: 202050
```

YugabyteDB, three nodes (RF3) · run `select-join-v4-topo-distributed-i1-20261005` · tag `run/study-01/select-join-v4-topo-distributed-i1-20261005` `studies/01-charity-tree/results/select-join-v4-topo-distributed-i1-20261005/join.json` · sha256 `1e89ee032810798eedb2bc9d723a7bd8386f9717146a338ac038608541db9805`

three tablets, leaders on three nodes, 180,000-row join SQL `j01_join_where` · parameters `threshold=180000`

```
SELECT c.id AS customer_id, c.name AS customer_name, f.id AS fact_id, f.amount
FROM customers c JOIN facts f ON f.customer_id = c.id
WHERE f.rank <= 180000;
```

Plan · `family=bag;stmt=join_where;threshold=180000` · 20 of 92 lines kept

```
Hash Join (actual time=3.423..48.973 rows=180000 loops=1)
  Hash Cond: (d.person_id = p.person_id)
  ->  Seq Scan on donation d (actual time=1.202..17.381 rows=180000 loops=1)
        Storage Filter: (rank <= 180000)
        Storage Table Read Requests: 59
        Storage Table Rows Scanned: 200000
  ->  Hash (actual time=2.207..2.210 rows=2000 loops=1)
        ->  Hash Join (actual time=1.627..1.988 rows=2000 loops=1)
              Hash Cond: (p.charity_id = ch.charity_id)
              ->  Seq Scan on person p (actual time=1.024..1.144 rows=2000 loops=1)
                    Storage Table Read Requests: 1
                    Storage Table Rows Scanned: 2000
              ->  Hash (actual time=0.589..0.590 rows=50 loops=1)
                    ->  Seq Scan on charity ch (actual time=0.571..0.576 rows=50 loops=1)
                          Storage Table Read Requests: 1
                          Storage Table Rows Scanned: 50
Planning Time: 0.198 ms
Execution Time: 62.579 ms
Storage Read Requests: 61
Storage Rows Scanned: 202050
```

Checked by the generator. this plan (`j01_join_where`): same node tree as “single-tablet layout, 180,000-row join” in run `select-join-v4-topo-tablet-i1-20261005`.

Colocated database

`v6-sel-17-placement-colocated-vs-distributed` · **repeated — no prior prediction** · card, p. 106

The joins are the same as in the distributed run, but the colocated run read `person` and `charity` through their primary-key indexes where the distributed run scanned them, so the node trees differ. The donation table was scanned sequentially in every layout. In the colocated run that scan made 176 read requests, as on the single tablet.

YugabyteDB, three nodes (RF3) · run `select-join-v4-topo-colocated-i1-20261005` · tag `run/study-01/select-join-v4-topo-colocated-i1-20261005` `studies/01-charity-tree/results/select-join-v4-topo-colocated-i1-20261005/join.json` · sha256 `771f30fddb4e5880544603c9cc85e1424602ea23efe11e298b93cb35f0f11a38`

colocated database, 180,000-row join

SQL `j01_join_where` · parameters `threshold=180000`

```
SELECT c.id AS customer_id, c.name AS customer_name, f.id AS fact_id, f.amount
FROM customers c JOIN facts f ON f.customer_id = c.id
WHERE f.rank <= 180000;
```

Plan · `family=bag;stmt=join_where;threshold=180000` · 20 of 96 lines kept

```
Hash Join (actual time=3.408..93.237 rows=180000 loops=1)
  Hash Cond: (d.person_id = p.person_id)
  ->  Seq Scan on donation d (actual time=1.228..62.004 rows=180000 loops=1)
        Storage Filter: (rank <= 180000)
        Storage Table Read Requests: 176
        Storage Table Rows Scanned: 200000
  ->  Hash (actual time=2.157..2.163 rows=2000 loops=1)
        ->  Hash Join (actual time=1.439..1.949 rows=2000 loops=1)
              Hash Cond: (p.charity_id = ch.charity_id)
              ->  Index Scan using person_pkey on person p (actual time=0.912..1.155 rows=2000 loops=1)
                    Storage Table Read Requests: 2
                    Storage Table Rows Scanned: 2000
              ->  Hash (actual time=0.517..0.520 rows=50 loops=1)
                    ->  Index Scan using charity_pkey on charity ch (actual time=0.493..0.503 rows=50 loops=1)
                          Storage Table Read Requests: 1
                          Storage Table Rows Scanned: 50
Planning Time: 0.196 ms
Execution Time: 106.642 ms
Storage Read Requests: 179
Storage Rows Scanned: 202050
```

Checked by the generator. this plan (`j01_join_where`): different node tree from “three tablets, leaders on three nodes, 180,000-row join” in run `select-join-v4-topo-distributed-i1-20261005`.

Glossary

APPENDIX · GLOSSARY

The single definition site for the vocabulary the rest of the book uses. Where a term is also a registry field, the registry's value is named in brackets.

Term	Meaning
claim	One reviewed statement about what the corpus measured, registered in the active evidence package and identified by an id such as `v2-16`. A number may not appear in this book without one.
strength	How strongly a claim is supported: `run_replicated` (whole runs in a non-fixed order agree, and a prediction was recorded before them), `repeated_controlled` (repeated whole runs or fresh loads with a control, but no direction recorded beforehand), `single_run_directional` (one run, one trial per cell, a direction not a curve), `mechanism_supported` (the mechanism is shown, the rate is not), `gap` (an explicit absence of evidence).
direct evidence	A callout whose numbers come from the cited run of the claim beside it.
mechanism evidence	A card or callout that demonstrates how something works without establishing a rate.
analogy	A labelled transfer beyond the measured family or environment. Never a measurement.
predicate pushdown	Applying a filter within an earlier scan or input when the answer permits it. SQL text order does not prove it happened.
CTE folding	Combining an eligible WITH query with its consumer for joint optimization.
CTE materialization	Evaluating a separate per-query result for reuse; it can block outer-filter pushdown. It is not a maintained rollup.
gap	An explicit absence of evidence, of one of four kinds. A gap is never a null result.
schema limitation	The representation cannot answer the question, so no run would. The hold-funnel gap is one.
coverage gap	A relevant regime has not been measured yet, but could be.
instrument gap	The property could not be observed with the tools the study had. Physical tablet placement is one.
unstable measurement	The number moved between runs with no code change, so no value is quotable.
partially superseded	Part of the claim has been replaced by newer evidence; the card says what moved to which successor and what is still open.
retired	The claim is no longer active, is not cited, and names the successor that closes it.
run	One execution of the harness under the benchmark lock, identified by a run tag and an inputs digest. Results are comparable only within one environment.
replication	A whole-run repeat of the same cell. The cards report it as `whole_run_replications`; `Trials: 1` means exactly one.
trial	A measurement inside a run. A fresh-load trial repeats the load phase; an in-run repeat is another measurement window inside one process. Neither is an independent replication.
read score	A blended within-run composite of a family's read queries. It compares variants inside one run and nothing else: it is not the throughput of any single query and is not quotable across runs.

Term	Meaning
wrong_reads	The harness counter for reads that violated the strict comparator in force, not for corruption. A relaxed cell can record many without serving an impossible value.
strict after acknowledgement	`begin(R1) > ack(W1)` implies the read must not return a version older than `W1`, unless a later write to the key supersedes it. This is the comparator the harness counts against.
relaxed freshness	A family of named weakenings of the strict line: session/read-your-writes, bounded staleness with a declared delta, eventual freshness, and best effort. Bounded staleness is one member, not the definition.
resource framing	`db-only` counts the database's resources; `equal-total` gives the cache a share of an equal budget. The two are never pooled, and a single-node versus cluster comparison answers "add machines", not "same budget".
colocation	Placing related rows in one shared partition. A layout label states intent; verified placement needs a tablet and leader listing, which only the Charity SELECT placement runs record.
tablet and leader	A tablet is one shard of a table on YugabyteDB; its leader is the replica that serves reads and writes. Splitting a table into several tablets can spread leaders over nodes.
semi-join and anti-join	A test for whether a related row exists (`EXISTS`, `IN`) or does not (`NOT EXISTS`). Each parent appears at most once, unlike a plain join.
keyset pagination	Fetching the next page after the last row's ordering keys, instead of skipping a counted prefix with `OFFSET`.
position drift	How much one repeated cell moves between the first and last place in a run. A difference smaller than it is not a resolved effect.
rollup	A parent aggregate maintained from its children. Where it lives in the same database it is updated in the same transaction; otherwise it is published, with a named delivery and reconciliation path.
rolldown	Parent information copied onto the child to avoid a join.
embedding	A child collection stored inside its parent row, as JSON or an array. In this corpus that means a relational JSON column, not a native document database.
source correctness	A mechanism that makes the authoritative mutation itself correct, such as a lock plus invariant recheck or a conditional update.
copy correctness	A mechanism that orders what leaves the transaction: invalidation, a publication fence, version validation at the hit boundary, a fill lease, a TTL, or an outbox.
derived state	A stored structure that answers from current facts and owes its maintainer a correctness contract on every contributing write: an index, a rollup, a rolldown or an external read copy.
retained history	Information kept so a question can be answered after the event was undone: an event/ledger table, an audit trail, a temporal history or a snapshot. A current-state-only representation cannot reconstruct what it has destroyed.
invalidation	Removing or marking a cache entry stale. It orders nothing about a read or a fill already in flight.
fill lease	A bounded ownership period that suppresses duplicate concurrent fills while it is valid and honoured. It is a load control, not a freshness proof.
fencing token	A monotonically increasing token checked at a protected sink, which rejects an operation whose token is older than one the sink has already accepted. It orders operations at the sink; it does not itself consult the source.

Term	Meaning
source-generation validation	Before publishing, checking that the authoritative current generation still equals the generation the reader observed, and refusing the fill if it moved. This closes the stale-fill race.
cache CAS	A conditional modification of cache state against an explicitly stated expected value or generation. It is not source-generation validation.
hit validation	Verifying a cached value against authoritative or version state at the hit boundary before serving it.
transactional outbox	Writing a change in the same transaction as the mutation so it can be published durably. It makes a committed change observable; it does not by itself make every cache hit current.
TTL / early expiry	A bound on how long an entry is eligible to serve, plus probabilistic early refill before many entries reach one deadline. It controls eligibility and load, not currentness, unless the declared contract is itself bounded by it.
publication fence	Refusing a fill whose source moved after the reader took its snapshot. This, not invalidation, is what closes the stale-fill race; source-generation validation is its strongest named form.
confound	A second change that moved with the one being measured. Registered in the package's confound register and published beside the numbers it affects.
family, variant, scenario	A family is a way of deciding where a fact lives; a variant is one design inside a family; a scenario is a workload built around one question, such as a hot drop or a refund.

Evidence registry (active v6 claims)

The active registry holds 50 claims that may be cited as evidence. This index is rendered from `book/evidence/v6/claims.json` at build time; the digest above identifies its content.

normalized facts

v2-01-normalized-index-first

normalized facts · run_replicated

In the charity -> person -> donation tree, the first large read win is the query index: D2 indexed scores about 10–11x D1 minimal's read score in two whole runs (13.3x in the first), and flattening the foreign key instead (D3 flat+FK) reaches about 14–16x (20.5x in the first). The orderings and the bands replicated in both design orders; the point figures were single-run levels.

Trials: 3 whole-run replications · 1 fresh-load trial per cell · 1 inner iteration

Confounds: conf-01-study01-d2d3-copied-key-package

Limits: PostgreSQL 1 node and YugabyteDB 1/3 nodes in one run; small scale; a single measurement per configuration, not a repeat; read score is a blended measure, not one query; the quoted point figures (13.3x, 20.5x) are the primary run's single levels: the whole-run replication of 2026–10–01 measured 9.7–15.5x for the same two relations while the absolute read scores moved 40–45% between its two runs, so the band and the ordering are what may be quoted, not the points

Source: studies/01-charity-tree/reports/20260912-small.md · run 20260912-small · tag none (legacy)

v2-06-hot-drop-designs

normalized facts · run_replicated

For a hot drop, pre-created seats taken by compare-and-set or SKIP LOCKED (P3 CAS, P2 skip-locked) sell 4–8x faster than any single counter row, but only with retry backoff in mind, and a counter on the event row also blocks unrelated edits to it. Both deliberately wrong controls fired in the same run. Two replications thirteen days later, on the same topology and with the design order reversed between them (AB/BA), reproduced the range - P3 at 4.30–7.06x and 4.64–6.99x, P2 at 3.25–5.25x and 5.78–8.71x against the two counters - while every absolute throughput fell 26–72% below the original run. The range is therefore the claim and the absolute numbers are one session's readings.

Trials: 3 whole-run replications · 3 fresh-load trials per cell · 1 inner iteration

Limits: one run of the experiment, a single measurement per configuration; the run has 42 cells of which 2 failed (C2 count (SER) on the two YugabyteDB topologies); those cells are not evidence; hot-drop workload only; every absolute throughput in the 2026–09–26 replication was 26–72% below the 2026–09–13 run for the same design and tier, so the 4–8x range reproduced while the numbers behind it did not; the range is quotable and the figures are not; the ratio moved between runs: p3 against c4 went 7.07 to 4.37 to 4.64 at 100 seats and 7.26 to 4.30 to 4.70 at 1000, and p2 against c4 went 3.97 to 3.25 to 5.78, so the range is a range, not a point, and individual ratios shift by up to 44%; the replication re-ran neither deliberately wrong control (c1_count_naive, h0_hold_naive_confirm), so it strengthens the comparison without re-firing the correctness control that the first run provided; r1_inventory_row recorded one trial of about 4x lower throughput than its siblings at each tier (122 and 117 against 468–653); whether that is a real instability or a harness artefact is unresolved, and r1 is the weakest comparison in the replication; two of the original sweep's five tiers (10 and 10000 seats) were not re-measured; p2_precreated_skip_locked moved 58–76% between the two same-day replications while the counters moved within 15% and p3 within 7%; it ran first in one and last in the other, so that design carries a position or warm-up effect as large as the comparison it takes part in, and the claim's pairing of p2 with p3 hides it - the >=3x result holds at both extremes; the anomalous r1_inventory_row trials of the first replication (122 and 117 against 468–653) did not recur in the reverse-order run, which makes a harness artefact likelier than a design property without settling it; the read phase of this claim's primary anchor 20260913T021206Z was client-CPU throttled by a design-dependent amount (0.2 s for p2_precreated_skip_locked to 29.5 s for h0_hold_naive_confirm, out of 633 CFS periods), so that report's read figures are not quotable. The hazard reproduces on the current harness (20260929T165533Z, 20260929T170353Z: r1_inventory_row loses 420 of 634 and 422 of 633 periods), and the 2026–09–30 budget calibration (20260930T000544Z, 20260930T000759Z, 20260930T001013Z) shows the client's quota is a symptom rather than the ceiling: at three client cores the client is not throttled at all and the reads are slower (52,884 -> 43,726 ops/s at 10 seats), because the third core comes out of the database's share of the machine's four CPUs (database throttled periods 238 -> 847 -> 1052). The claim's own basis is the race phase (sell rate), where one cell in fourteen designs shows throttling. The read family has since been re-measured at the study's new 2-worker read footprint (20260930T164028Z, 20260930T164834Z: four designs, client unthrottled at 0 of ~631 periods, every cell latency-bound at 2.00 operations in flight for 2 workers (with the mean latency; a median-based count reads 1.9)), so read levels are quotable only with the footprint attached. All fifteen designs now have a footprint-2 footing (the four-design tranche plus the eleven-design tranche 20260930T201732Z/20260930T203913Z, where no design throttles: 0 of ~631 periods in all 22 read phases), and this claim's primary anchor's own footprint-8 read figures remain not quotable: the inflation they carry is design-dependent, 1.09x to 2.22x, and it tracks how many periods the old client lost per design rather than anything about the designs. All fifteen designs have since been measured in a **single run**, twice and in reversed order (20261001T100948Z, 20261001T103931Z): 0 throttled periods in all 30 read phases and every one of the 270 cells store-bound at 2.00 operations in flight by the mean latency, so the family's levels are jointly comparable within that session. Two consequences for quoting: the top group (h0, h1, r1, r2, p4) differs by 4%, inside that session's own order term, so its internal ranking is not quotable - only the 1.5x gap between that group and the per-seat group is; and c1_count_naive carries a 40% position term (first of one order against last of the other), so its q01 level should not be quoted from these runs.

Source: studies/02-ticket-booking/reports/20260913T021206Z.md · run 20260913T021206Z · tag run/02-ticket-booking/20260913T021206Z

v2-10-guarded-confirm-refusals

normalized facts · run_replicated

On YugabyteDB a guarded confirmation is not enough, but the refusals are spurious rather than a design fault: at the 10,000-seat tier the correct designs refused valid holds (s1_conditional_update 2 refusals in one whole run and 3 in the reversed run, s2_lock_then_update 1 in each), and every refusal is classified transient - the identical statement re-issued inside the refusing transaction matched every seat - while the retry-once variant s1r_confirm_retry refused nothing and every retry it made succeeded (5 across the forward run). On PostgreSQL the same three designs refused nothing at any of five tiers from 10 to 100,000 seats in two whole runs, so the guarantee held everywhere there. The study's negative control s0_check_then_hold_rc refused at every tier on both engines and stole 474–599 holds per tier on YugabyteDB and 1,168–6,231 on PostgreSQL, so the audit detects both the spurious refusal and the real theft. This claim is about refusals, not throughput, and no cost conclusion follows from it.

Trials: 3 whole-run replications · 1 fresh-load trial per cell · 1 inner iteration

Limits: v1 matrix run, a single measurement per configuration; the claim is about refusals, not throughput; retry-once is a design variant (S1r), not a tuning flag; the replicated refusals are few and tier-bound: 2, 3 and 1 refusal events in the correct designs across two whole runs, all at the 10,000-seat tier except s2's single 1,000-seat refusal in the reversed order, so the claim is that refusals happen and are transient - not a rate, and not a tier law; the YugabyteDB arm measured four tiers (10 to 10,000 seats) while the PostgreSQL control measured five (10 to 100,000), because that is how the study configures its tiers per engine; 'the guarantee held everywhere' therefore means everywhere the PostgreSQL arm ran; the refusals are the harness's own classification of its own designs, and the transient sub-class is decided by re-issuing the statement inside the refusing transaction (AM-01), so this replicates the mechanism and its detection, not an independent definition of a valid hold; the 2 GiB podman-machine blocker recorded in CONTEXT.md is a multi-node limit: a single-node YugabyteDB arm fits and completes here, which is what makes this replication possible, while yb-cluster3 remains unmeasurable on this machine; the spread figure is the refusal count's own movement, not a throughput spread: s1_conditional_update refused 2 times in the forward run and 3 in the reversed one, so the count moves 40% across the two runs and the phenomenon is established, not quantified

Source: studies/03-reserved-seating/reports/20260915T002411Z.md · run 20260915T002411Z · tag run/03-reserved-seating/20260915T002411Z

v2-12-lock-vs-version-upper-bound

normalized facts · run_replicated

Under one hot key at 16 writers a lock-before-update (c2_pessimistic_lock) delivered about 1.6–1.8x the throughput of a version check (c1_optimistic_version): 1.76x in the primary run and 1.56x, 1.57x, 1.64x and 1.69x in four whole runs on 2026-10-02 that crossed the two design orders with the retry budget at its default and at zero. The retry budget does not explain the advantage - at a matched design order removing it moves the ratio by 1.1% to 3.2% - so the earlier wording calling the figure an upper bound because of per-retry harness work is retired by its own control. An unchecked read-modify-write (x1_lost_update_control) lost 93.7% of acknowledged updates in the primary run and 93.7% and 93.9% in the replications, while both correct designs lost none.

Trials: 5 whole-run replications · 1 fresh-load trial per cell · 1 inner iteration

Limits: one key, one installed product, 16 writers; PostgreSQL single node, small scale; one fresh-load trial per run; five whole runs now support the ratio, but no run repeats the cell, and the -retries 1 setting was not among the controls (the budget controls used the default 8 against -retries 0); 1.76x is one point on a writer curve, not a curve; the upper-bound hedge is retired: the retry budget was re-run on 2026-10-02 at 16 writers in both design orders, default against -retries 0 (1.64x against 1.69x with the lock first, 1.57x against 1.56x with the version check first), a 1.1% to 3.2% difference, which agrees with the primary's own retry control (1.60x against 1.61x) and means the advantage is not per-retry harness work; the re-scoped figure is a band of five whole runs on two days (1.56x, 1.57x, 1.64x, 1.69x, 1.76x); absolute throughput is day-specific, so only the within-run ratio travels

Source: studies/04-configuration-portal/reports/20260921T1215Z-small.md · run 20260921T1215Z-small · tag run/04-configuration-portal/20260921T1215Z-small

v2-gap-03-study04-missing-designs

normalized facts · gap · coverage gap

Eight of the eighteen planned Study 04 configuration-portal designs are not implemented, and cardinality, cadence/churn, repeated randomized trials, equal-total resources, placement and YugabyteDB are unmeasured. These are coverage gaps, not null results.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Limits: no conclusion may read 'the missing designs would not change this'; owned by task-20260922T025912Z-study04-v2-protocol

Source: studies/04-configuration-portal/reports/20260921T1215Z-small.md · run 20260921T1215Z-small · tag run/04-configuration-portal/20260921T1215Z-small

v4-gap-02-endpoint-distribution-partial

normalized facts · gap · partially superseded by v6-sel-16-placement-splitting-effect, v6-sel-17-placement-colocated-vs-distributed · coverage gap + instrument gap

Moved to successor claims: v6-sel-16-placement-splitting-effect, v6-sel-17-placement-colocated-vs-distributed.

Still open in this claim: study 01 design runs and study 02 yb-cluster3 runs connect through one endpoint; the Study 05 three-node spread is recorded as a node count, not a per-endpoint balance; no real network between nodes.

Statement below is unchanged from its source registry; the moved dimensions are not restated as open here.

Client endpoint distribution is measured for the 2026–09–23 Study 05 three-node cells, where the client spreads operations round-robin over all three endpoints (v3–05). It is also measured per endpoint for Study 01's SELECT join cluster runs of 2026–10–05, where reader and writer operations were balanced across the three nodes (for example 168/168/168 reads; v6-sel-16, v6-sel-17). The earlier multi-node cells still reach the cluster through a single endpoint: the study 01 design runs and the study 02 yb-cluster3 runs each set one DSN against yb-n1. Their cluster ratios therefore still stand in for cluster access through one endpoint, the Study 05 spread is recorded as a node count rather than a per-endpoint balance, and no run has a real network between nodes.

Trials: 0 whole-run replications · 0 fresh-load trials per cell · 0 inner iterations

Limits: no real network between nodes; endpoint distribution is unmeasured for the studies 01–03 multi-node cells, which connect through a single endpoint; the Study 05 three-node spread is round-robin by construction and is recorded as a node count, not as a measured per-endpoint balance

Source: studies/05-cache-consistency/reports/20260923T1157Z-res2-yb3.md · run 20260923T1157Z-res2-yb3 · tag run/05-cache-consistency/20260923T1157Z-res2-yb3

v2-17-optimistic-pessimistic-crossover

normalized facts · run_replicated

Under one hot key a bounded-retry version check (c1_optimistic_version) and a lock-before-update (c2_pessimistic_lock) cross over between one and two writers: in the primary sweep the lock delivered 0.89x the version check's throughput at 1 writer (1,177 against 1,329 ops/s) and 1.10x at 2 (1,285 against 1,167), rising to 1.68x at 32 and 1.60x at 64, while the version check's throughput fell 2.9x across the sweep and its conflict count rose from 0 to 21,301. Three whole runs on 2026–10–02 reproduced the crossing (0.905x, 0.939x and 0.948x at 1 writer; 1.035x, 1.106x and 1.166x at 2) and the divergence above it (1.56x-1.69x at 16 writers and 1.73x-1.96x at 64, in both design orders), with the version check's conflicts rising from 0 to about 25,000 and its throughput falling from 1,286 to 563 ops/s. One further 1-writer run measured 3.85x because c1 itself read 306 ops/s there while another run at the same level and design order read 1,286; no client or database throttling was recorded, and the added version-check-first runs read 0.905x and 0.948x at that level, so the excursion is reported as unexplained rather than as an order effect.

Trials: 23 whole-run replications · 1 fresh-load trial per cell · 1 inner iteration

Limits: PostgreSQL single node, small scale, one hot key; one fresh-load trial per run; the 2026–10–02 tranche measured the pair at 1, 2, 16 and 64 writers in both design orders, and at 16 writers with the retry budget at zero, but the primary's 4, 8 and 32-writer levels were not re-measured, so the claim's 32-writer point (1.68x) rests on the primary alone; we tested the obvious explanation for the lock's advantage and it failed: with the retry budget set to zero the c2/c1 ratio at 16 writers is unchanged within 1.1% to 3.2% in both design orders, and c2_pessimistic_lock, which never retries either way, moved by the same amount as c1, so the advantage cannot be put down to retry work; the primary's seven runs used two phase sets (the first four also ran read and write phases); the contention metric should not depend on that, but it is an assumption rather than a measurement; at 1 writer the unsafe control x1_lost_update_control cannot lose an update because there is no concurrency, so its gate cannot fire: three runs that included it at that level (20261002T162400Z-w1, 20261002T162737Z-w1 and 20261002T165329Z) record a failed x1 cell, and the claim's control_coverage comes from the levels at two writers and above, where x1 lost 50.0% and 93.7%-98.0% of acknowledged updates; one 1-writer run (20261002T162737Z-w1) measured 3.85x because c1 read 306 ops/s there against 1,286 in the other run with the same design order; the version-check-first runs gave 0.905x and 0.948x at that level and no client or database throttling was recorded in any of them, so the excursion is reported as unexplained; absolute throughput is day-specific: c2 measured 869 ops/s on 2026–09–23 and 1,056 on 2026–09–26 at identical settings, phases and writer count, so only the within-run ratio is carried across days

Source: studies/04-configuration-portal/reports/20260923T0011Z-ctlC-w1.md · run 20260923T0011Z-ctlC-w1 · tag run/04-configuration-portal/20260923T0011Z-ctlC-w1

v6-sel-01-inner-join-spellings-same-plan

normalized facts · run_replicated

For a filtered inner join over Study 01's 200,000-donation fixture, five alternative spellings (JOIN ... ON with the filter, a filtered derived table, a CTE naming the filtered input, a CTE naming the joined rows, and a comma join) ran the same plan as JOIN ... WHERE in all 120 spelling, selectivity, engine and phase pairs: selectivity 0.01% to 90%, PostgreSQL btree and YugabyteDB range index, isolated and blended phases, three runs each. The 27 pairs whose sign agreed across runs had identical plans, magnitudes of 0.2–40% and directions that changed between phases, so no difference is attributable to the spelling.

Trials: 12 whole-run replications · 1 fresh-load trial per cell · 5 inner iterations

Limits: one laptop, single-node engines and one sequential reader: latency of a drained result, not throughput; a no-resolved-difference statement bounded by this fixture, six selectivity levels and two engines, not an equivalence theorem: the domain position control moved one cell by up to 42% (0.868–1.419), more than any stable spelling sign; YugabyteDB used the range-sharded rank index (rank ASC); with the engine's default hash-sharded index no spelling had a selective path to use; plan equality compares normalized plan trees (node types, relations and indexes), not costs or row estimates

Source: studies/01-charity-tree/reports/20261005T-select-synthesis-evidence.md · run select-join-v3b-domain-pg-i1-20261005 · tag run/study-01/select-join-v3b-domain-pg-i1-20261005

v6-sel-02-materialized-joined-cte-penalty

normalized facts · run_replicated

Forcing MATERIALIZED on a CTE that contains the join, with the selective filter applied outside it, multiplied the query time against the same CTE left to the engine. On PostgreSQL the factor was 84–101x at 0.01% selectivity, 51–64x at 0.1% and 23–27x at 1%, falling to 1.02–1.44x at 90%. On YugabyteDB with a range index it was 34–49x, 20–29x and 8.1–9.5x, falling to 0.93–1.03x at 90%. Three runs per engine and phase. With the filter inside the CTE, forcing MATERIALIZED cost at most 1.61x on PostgreSQL and 0.71–1.25x on YugabyteDB.

Trials: 12 whole-run replications · 1 fresh-load trial per cell · 5 inner iterations

Limits: the penalty is plan-explained on both engines (the materialization boundary keeps the outside filter from reaching the join), measured as end-to-end latency of one reader; YugabyteDB at 90% is inside its position drift (0.93–1.03x), so no reversal is claimed; with a hash-sharded YugabyteDB index the same hint looked nearly free (earlier runs: 1.4–1.7x) because there was no selective path to lose; this claim holds for the range-index arm

Source: studies/01-charity-tree/reports/20261005T-select-synthesis-evidence.md · run select-join-v3b-domain-pg-i1-20261005 · tag run/study-01/select-join-v3b-domain-pg-i1-20261005

v6-sel-03-cte-reuse-crossing

normalized facts · run_replicated

For one expensive CTE referenced n times, NOT MATERIALIZED against MATERIALIZED crossed between one and two references on PostgreSQL at the unselective (100%) consumer and on YugabyteDB at both consumers. At n=2 NOT MATERIALIZED took 1.43–2.06x the MATERIALIZED time and at n=16 3.9–12.1x, in every run of both phases. At n=1 the ratio was 0.81–0.94 on YugabyteDB at the 1% consumer (6 of 6 runs) and 0.87–1.07 at the 100% consumers. Both engines' default policy ran MATERIALIZED's plan from two references.

Trials: 12 whole-run replications · 1 fresh-load trial per cell · 5 inner iterations

Limits: the YugabyteDB one-reference advantage at the 1% consumer is small (6–19%) but has the same sign in all six runs; the n=1 point at the 100% consumers is mixed in sign, so the crossing is bracketed between 1 and 2 references, not located; one CTE body and two consumer selectivities on one fixture

Source: studies/01-charity-tree/reports/20261005T-select-synthesis-evidence.md · run select-join-v3b-domain-yb-i1-20261005 · tag run/study-01/select-join-v3b-domain-yb-i1-20261005

v6-sel-04-cte-reuse-postgresql-selective-no-crossing

normalized facts · run_replicated

On PostgreSQL with a selective (1%) consumer, NOT MATERIALIZED stayed faster than MATERIALIZED at every reuse count from 1 to 16: 0.17–0.27x at one reference and 0.68–0.81x at sixteen, in all six runs. The default policy materialized from two references, so forcing NOT MATERIALIZED was 1.2–4.8x faster at 2–16 references.

Trials: 6 whole-run replications · 1 fresh-load trial per cell · 5 inner iterations

Limits: the ratio rises with the reference count (0.17–0.27 at 1 to 0.68–0.81 at 16), so a crossing beyond 16 references is possible and untested; one CTE body and one consumer selectivity on one fixture

Source: studies/01-charity-tree/reports/20261005T-select-synthesis-evidence.md · run select-join-v3b-domain-pg-i1-20261005 · tag run/study-01/select-join-v3b-domain-pg-i1-20261005

v6-sel-05-membership-join-distinct-penalty

normalized facts · run_replicated

For the set of parents with a qualifying child, JOIN + DISTINCT took 1.40–1.69x the EXISTS time at child fanout 1, 1.38–1.44x at fanout 10 and 3.64–4.23x at fanout 100 on PostgreSQL, in all three randomized-order runs. On YugabyteDB it was 1.10–1.16x at fanout 1 and 0.95–1.01x at fanouts 10 and 100, with no growth.

Trials: 6 whole-run replications · 1 fresh-load trial per cell · 5 inner iterations

Limits: fanout and parent count change together (10,000 child rows over 10,000, 1,000 and 100 parents), so the growth cannot be attributed to fanout alone; the YugabyteDB runs used the default hash-sharded indexes and none of their 64 plans used an index; single-node engines, one sequential reader; position drift 22% (PostgreSQL) and 13% (YugabyteDB); the run environment page is xiles-podman-select-order-correction-20261004, which documents this same host and measurement slot

Source: studies/01-charity-tree/reports/20261005T-select-synthesis-evidence.md · run select-order-existence-postgres-1-20261004 · tag run/study-01/select-order-existence-postgres-1-20261004

v6-sel-06-membership-in-exists-no-difference

normalized facts · repeated_controlled

IN and EXISTS over the same membership question had no resolved latency difference on either engine. IN/EXISTS was 0.86–1.30 on PostgreSQL with mixed signs at every fanout, and 0.94–1.07 on YugabyteDB, inside each engine's measured position drift (22% and 13%). Every cell returned the oracle's parent set.

Trials: 6 whole-run replications · 1 fresh-load trial per cell · 5 inner iterations

Limits: no resolved difference on these fixtures, not an equivalence result: no equivalence bound was set; the YugabyteDB runs used the default hash-sharded indexes and none of their plans used an index; the run environment page is xiles-podman-select-order-correction-20261004, which documents this same host and measurement slot

Source: studies/01-charity-tree/reports/20261005T-select-synthesis-evidence.md · run select-order-existence-postgres-1-20261004 · tag run/study-01/select-order-existence-postgres-1-20261004

v6-sel-07-nullable-not-in-empty

normalized facts · run_replicated

A NOT IN over a child key that contains a NULL returned no rows at every fanout level in all six runs (18 of 18 checks, PostgreSQL and YugabyteDB), although the oracle's set of parents without a qualifying child was non-empty. NOT EXISTS, LEFT JOIN ... IS NULL and NOT IN over a NOT NULL subquery returned the oracle's set.

Trials: 6 whole-run replications · 1 fresh-load trial per cell · 1 inner iteration

Limits: an answer-contract result: for every outer row the NOT IN predicate is FALSE or UNKNOWN once the subquery holds a NULL, so no row qualifies; it is not UNKNOWN for every row; one fixture with NULL child keys at three fanout levels; the run environment page is xiles-podman-select-order-correction-20261004, which documents this same host and measurement slot

Source: studies/01-charity-tree/reports/20261005T-select-synthesis-evidence.md · run select-order-existence-yugabyte-1-20261004 · tag run/study-01/select-order-existence-yugabyte-1-20261004

v6-sel-08-anti-join-not-exists-postgresql

normalized facts · repeated_controlled

For parents without a qualifying child at fanout 100, NOT EXISTS on PostgreSQL took 0.21–0.24 ms against 0.64–0.69 ms for LEFT JOIN ... IS NULL (2.71–3.27x) and 0.67–0.72 ms for NOT IN over a NOT NULL subquery (2.85–3.16x), in all three runs. At fanouts 1 and 10 the NOT-NULL NOT IN was 1.09–1.19x and LEFT JOIN ... IS NULL 0.88–1.08x. On YugabyteDB the three spellings were within 0.997–1.097x of each other.

Trials: 6 whole-run replications · 1 fresh-load trial per cell · 5 inner iterations

Limits: fanout and parent count change together (100 parents at fanout 100); the YugabyteDB runs used the default hash-sharded indexes and none of their plans used an index; the run environment page is xiles-podman-select-order-correction-20261004, which documents this same host and measurement slot

Source: studies/01-charity-tree/reports/20261005T-select-synthesis-evidence.md · run select-order-existence-postgres-1-20261004 · tag run/study-01/select-order-existence-postgres-1-20261004

v6-sel-09-aggregate-correlated-postgresql-crossover

normalized facts · run_replicated

On PostgreSQL a correlated scalar SUM per parent was 0.47–0.90x the join-then-GROUP-BY time for one parent, mixed in sign at 10 parents, and 1.26–1.72x at 100, 1.08–1.27x at 1,000 and 2.49–2.70x at 10,000 parents, in all three randomized-order runs.

Trials: 3 whole-run replications · 1 fresh-load trial per cell · 5 inner iterations

Limits: the aggregate family's position drift is 29% on PostgreSQL (one cell 0.710 last/first), larger than the 1,000-parent effect; pre-aggregation overtook post-join only at 10,000 parents (0.77–0.80x), inside that drift; it is not claimed; one fixture of 10,000 parents with two children each; the run environment page is xiles-podman-select-order-correction-20261004, which documents this same host and measurement slot

Source: studies/01-charity-tree/reports/20261005T-select-synthesis-evidence.md · run select-order-aggregate-postgres-1-20261004 · tag run/study-01/select-order-aggregate-postgres-1-20261004

v6-sel-10-aggregate-yugabytedb-no-reversal

normalized facts · run_replicated

On single-node YugabyteDB the correlated scalar SUM never beat join-then-GROUP-BY. It was 1.07–1.23x at one parent, 1.93–2.00x at 10, 8.6–11.9x at 100, 41–46x at 1,000 and 398–541x at 10,000 parents: 6.7–20.0 s per query, against 16–17 ms on PostgreSQL. Pre-aggregation never beat post-join with a stable sign. All three randomized-order runs.

Trials: 3 whole-run replications · 1 fresh-load trial per cell · 5 inner iterations

Limits: the per-probe cost (about 0.5–2 ms per parent) suggests a separate storage request per probe, but no RPC counters were parsed, so the mechanism is a hypothesis; the aggregate family's position drift on YugabyteDB is 74% (one cell 1.743 last/first); single node; the customer_id index is hash-sharded and serves the equality probes (43/91 plans used an index); the run environment page is xiles-podman-select-order-correction-20261004, which documents this same host and measurement slot

Source: studies/01-charity-tree/reports/20261005T-select-synthesis-evidence.md · run select-order-aggregate-yugabyte-1-20261004 · tag run/study-01/select-order-aggregate-yugabyte-1-20261004

v6-sel-11-topn-lateral-probe-crossover

normalized facts · run_replicated

For the top row per parent, a LATERAL index probe against ROW_NUMBER() was, on YugabyteDB, 0.54–0.70x at one parent, 1.12–1.25x at 10, 6.1–7.0x at 100, 44–49x at 1,000 and 257–289x at 10,000 parents. On PostgreSQL it was 0.56–0.98x at one parent (inside the 29% position drift), mixed at 10, and 1.19–2.20x from 100 to 10,000 parents. All three randomized-order runs per engine.

Trials: 6 whole-run replications · 1 fresh-load trial per cell · 5 inner iterations

Limits: the PostgreSQL one-parent advantage is below its family's position drift in two of three runs, so its magnitude is unresolved; position drift 29% (PostgreSQL) and 74% (YugabyteDB) in the aggregate family; N=1 only; ROW_NUMBER needs a total-order tiebreak, enforced by the x06 control; the run environment page is xiles-podman-select-order-correction-20261004, which documents this same host and measurement slot

Source: studies/01-charity-tree/reports/20261005T-select-synthesis-evidence.md · run select-order-aggregate-yugabyte-1-20261004 · tag run/study-01/select-order-aggregate-yugabyte-1-20261004

v6-sel-12-topn-distinct-on-postgresql

normalized facts · run_replicated

On PostgreSQL, DISTINCT ON for the top row per parent took 0.61–0.64x the ROW_NUMBER() time at 10,000 parents, 0.61–0.92x at 1,000 and 0.44–0.85x at one parent, in all three randomized-order runs; at 10 and 100 parents the sign was mixed. On YugabyteDB it was 0.89–0.93x at 1,000 parents and 0.70–1.00x at 10,000, with one run practically equal.

Trials: 6 whole-run replications · 1 fresh-load trial per cell · 5 inner iterations

Limits: DISTINCT ON is PostgreSQL-dialect SQL and answers N=1 only; the PostgreSQL 1,000-parent and one-parent ranges include runs inside the 29% position drift; only the 10,000-parent effect clearly exceeds it; the run environment page is xiles-podman-select-order-correction-20261004, which documents this same host and measurement slot

Source: studies/01-charity-tree/reports/20261005T-select-synthesis-evidence.md · run select-order-aggregate-postgres-1-20261004 · tag run/study-01/select-order-aggregate-postgres-1-20261004

v6-sel-13-or-not-beaten-by-union

normalized facts · run_replicated

Rewriting a two-arm disjunction as UNION ALL or UNION never beat OR by a resolved margin. On PostgreSQL, UNION ALL took 1.23–1.55x and UNION 1.25–1.50x the OR time on the broad shapes, and 0.86–1.14x (mixed in sign or within 7%) on the selective shapes, where one BitmapOr served both arms. On YugabyteDB both took 1.32–1.87x OR, and no plan used an index.

Trials: 6 whole-run replications · 1 fresh-load trial per cell · 5 inner iterations

Limits: a bounded non-observation of the OR-blocks-an-index mechanism on these shapes, not a rule that OR never loses; the YugabyteDB runs used the default hash-sharded amount index, so 0/103 of their plans used an index and their margins compare one scan against two; one selective threshold (about 0.95% per arm) on a 12,000-row fixture; the run environment page is xiles-podman-select-order-correction-20261004, which documents this same host and measurement slot

Source: studies/01-charity-tree/reports/20261005T-select-synthesis-evidence.md · run select-order-access-postgres-1-20261004 · tag run/study-01/select-order-access-postgres-1-20261004

v6-sel-14-expression-index-postgresql

normalized facts · repeated_controlled

With the query text unchanged, adding an expression index matching (amount + 0) cut PostgreSQL's selective expression OR to 0.28–0.33x of its time without the index, and the plan switched to the expression index. The broad expression shape was unchanged (0.82–1.09x, a sequential scan either way).

Trials: 3 whole-run replications · 1 fresh-load trial per cell · 5 inner iterations

Limits: the index arm runs as its own seeded phase after the default arm, so phase position is not balanced; the plan change explains the effect; on YugabyteDB the same arm read 0.75–0.89x but no YugabyteDB plan used any index, so that change is not attributable to the index; a one-batch index-maintenance probe is too noisy to price the index; the run environment page is xiles-podman-select-order-correction-20261004, which documents this same host and measurement slot

Source: studies/01-charity-tree/reports/20261005T-select-synthesis-evidence.md · run select-order-access-postgres-1-20261004 · tag run/study-01/select-order-access-postgres-1-20261004

v6-sel-15-keyset-offset-crossover

normalized facts · run_replicated

For five-row pages under a fixed snapshot, keyset pagination against OFFSET was 1.06–1.29x at page depths 1–100, 0.73–0.82x at 1,000 and 0.21–0.24x at 10,000 on PostgreSQL. On single-node YugabyteDB it was 1.20–1.36x at depths 1–1,000 and 0.69–0.72x at 10,000. All three randomized-order runs; both spellings returned the same page at every depth.

Trials: 6 whole-run replications · 1 fresh-load trial per cell · 5 inner iterations

Limits: the keyset spelling orders by (amount DESC NULLS LAST, id) with a NULL-aware three-way OR and no (amount, id) composite index; PostgreSQL served it with a BitmapOr and a top-N sort, and its cost fell with depth (0.90–1.01 to 0.45–0.49 ms), so the flat-cost component of the prediction is untested; the YugabyteDB runs used the default hash-sharded amount index and no plan used an index, so their crossing describes scan plans; one 12,000-row fixture; the run environment page is xiles-podman-select-order-correction-20261004, which documents this same host and measurement slot

Source: studies/01-charity-tree/reports/20261005T-select-synthesis-evidence.md · run select-order-access-postgres-1-20261004 · tag run/study-01/select-order-access-postgres-1-20261004

v6-sel-16-placement-splitting-effect

normalized facts · run_replicated

On a 3-node YugabyteDB cluster (RF3, 2 CPU/3 GiB per node), pre-splitting the joined relations into three tablets, with leaders gated onto three nodes, changed join times against the single-tablet layout in every run. A 2,000-row join took 0.73x (isolated) and 0.71x (blended), and a 180,000-row join 0.54x and 0.71x. The 20-row join was 1.22x slower in the isolated phase and unresolved in the blended phase. At 180,000 rows the split layout was also 0.71–0.76x the single node and the equal-total single node.

Trials: 24 whole-run replications · 1 fresh-load trial per cell · 5 inner iterations

Limits: the nodes are containers on one laptop, so this is data placement without a network; tablet count and leader spread change together in this pair; a split single-node arm was not run; the 1 CPU client was throttled in 32–46% of its CFS periods in every run; point reads are the most exposed; placement was verified by yb_table_properties and yb-admin leader listings before and after each distributed run

Source: studies/01-charity-tree/reports/20261005T-select-synthesis-evidence.md · run select-join-v4-topo-distributed-i1-20261005 · tag run/study-01/select-join-v4-topo-distributed-i1-20261005

v6-sel-17-placement-colocated-vs-distributed

normalized facts · repeated_controlled

On the same 3-node cluster, a colocated database, with all relations in one shared tablet led by one node, was compared with the pre-split distributed layout in every run. It took 0.71x (isolated) and 0.75x (blended) at a 20-row join, 0.78x and 0.82x at 2,000 rows, and 1.21x and 1.54x at 180,000 rows. Its forced-MATERIALIZED penalty at 2,000 rows was larger in every run: per-run medians 16.8x against 8.5x isolated, and 11.8x against 8.4x blended.

Trials: 12 whole-run replications · 1 fresh-load trial per cell · 5 inner iterations

Limits: the nodes are containers on one laptop, so this is data placement without a network; no direction was recorded before the runs; the pair is reported as measured; the colocated tablet's leader sat on one node per run (yb-n2 or yb-n3); the 1 CPU client was throttled in 32–46% of its CFS periods in every run

Source: studies/01-charity-tree/reports/20261005T-select-synthesis-evidence.md · run select-join-v4-topo-colocated-i1-20261005 · tag run/study-01/select-join-v4-topo-colocated-i1-20261005

v6-sel-gap-01-yugabytedb-scan-only-and-single-node-families

normalized facts · gap · coverage gap

Three SELECT families lack measurements that their YugabyteDB and cluster conclusions would need. The access-path and membership/absence runs used the default hash-sharded single-column indexes, so none of their 167 YugabyteDB plans used an index and their YugabyteDB margins describe scan plans. The access, aggregate and membership families ran on single nodes only, with no 3-node, equal-total or placement arm. Keyset pagination was never measured with a matching composite (amount, id) index.

Trials: 0 whole-run replications · 0 fresh-load trials per cell · 0 inner iterations

Limits: the join/CTE family closes the index arm (range-sharded rank index) and the placement arm for its own workload only; a range-index arm on YugabyteDB is a physical-design change and must be measured as its own isolate pair

Source: studies/01-charity-tree/reports/20261005T-select-synthesis-evidence.md · run select-order-access-yugabyte-1-20261004 · tag run/study-01/select-order-access-yugabyte-1-20261004

v6-sel-18-aggregate-preagg-derived-table

normalized facts · run_replicated

Pre-aggregating each parent's children in a derived table and then joining took 5.7–7.9x the join-then-GROUP-BY time for one parent, 4.3–8.9x at 10, 5.7–6.5x at 100 and 1.56–1.90x at 1,000 parents on PostgreSQL, in all three randomized-order runs; at 10,000 parents it took 0.77–0.80x, inside the family's 29% position drift. On YugabyteDB it was 2.7–3.3x at one and 10 parents, 1.8–2.7x at 100, mixed in sign at 1,000 and 1.10–1.23x at 10,000, so it never beat the post-join aggregate.

Trials: 6 whole-run replications · 1 fresh-load trial per cell · 5 inner iterations

Limits: the fixture joins one child relation, so pre-aggregation's expected advantage (avoiding a multiplier when two child collections are joined) is not exercised; the x05 control shows that multiplier as a wrong answer, not as a timing; position drift in the aggregate family is 29% on PostgreSQL and 74% on YugabyteDB; the PostgreSQL 10,000-parent advantage is inside it, so it is not a resolved win; one fixture of 10,000 parents with two children each; single-node engines, one sequential reader; the run environment page is xiles-podman-select-order-correction-20261004, which documents this same host and measurement slot

Source: studies/01-charity-tree/reports/20261005T-select-synthesis-evidence.md · run select-order-aggregate-postgres-1-20261004 · tag run/study-01/select-order-aggregate-postgres-1-20261004

rolldown / duplicated data

v2-05-d3-read-advantage-repeated

rolldown / duplicated data · run_replicated

D3's (d3_flattened_fk) read advantage over the normalized indexed design survives five fresh loads in one session and reproduces across two whole runs in reversed order, at 1.59x, 1.72x, 1.64x and 1.84x in its four further fresh loads against the primary's 2.47x; the recency SQL and actual heap avoidance explain the gain, while its insert cost remains imprecise.

Trials: 3 whole-run replications · 5 fresh-load trials per cell · 1 inner iteration

Limits: five fresh loads per configuration, all in one session; the insert-cost estimate is imprecise and must not be quoted as a ratio; the magnitude came down on replication: 1.59–1.84x against the primary's 2.47x, so the direction and the survival are what a reader should carry, and the ratio itself is a band that a future run could move again

Source: studies/01-charity-tree/reports/20260913T125342Z-v3.md · run 20260913T125342Z-v3 · tag run/01-charity-tree/20260913T125342Z-v3

v3-06-colocated-vs-noncolocated-locality

rolldown / duplicated data · single_run_directional

On YugabyteDB 3-node (RF=3) a pair differing in key locality, where one layout (y1-coloc) keeps a donor's donations in one tablet and the other (y2-noncol) hashes them across tablets, recorded the first faster on the warm read (882 against 673 ops/s) and on the cacheable read (852 against 520), with 0 wrong reads either way. Where the tables and leaders physically sat was not verified, so this is evidence about the engine's own locality on one host, not about verified colocation.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Limits: the logical partition mapping follows deterministically from the colocated table's PRIMARY KEY ((person_id) HASH, ...), so one donor's rows hash to one tablet by configuration, not by observed placement; physical tablet/leader placement was not observed: the runner could not produce a tablet/leader distribution (its placeholder held only the three server names), so this is engine locality on one host, not verified colocation; all containers run on one WSL2 host; one host is not colocation evidence; one run per layout, no cache in this scenario, no repeat; spreads are large (warm 40.0% y1, 22.2% y2); not re-measured on 2026–10–02: the pair needs yb-cluster3, which the fixed machine cannot carry, and the harness still has no tablet/leader placement record, so the placement mechanism remains an unsupported coverage gap rather than a completed comparison

Source: studies/05-cache-consistency/reports/20260923T1215Z-res2-placement-yb3.md · run 20260923T1215Z-res2-placement-yb3 · tag run/05-cache-consistency/20260923T1215Z-res2-placement-yb3

v4-gap-01-physical-placement-unverified

rolldown / duplicated data · gap · partially superseded by v6-sel-16-placement-splitting-effect, v6-sel-17-placement-colocated-vs-distributed · instrument gap + coverage gap

Moved to successor claims: v6-sel-16-placement-splitting-effect, v6-sel-17-placement-colocated-vs-distributed.

Still open in this claim: Study 05 key-locality pair (v3–06): placement labels are intent, not observed; Study 04 y1/y2 pair unexecuted; Study 01 D7/D24 and Study 03 L3 remain placement intent; every node runs as a container on one host.

Statement below is unchanged from its source registry; the moved dimensions are not restated as open here.

Physical tablet and leader placement is verified only for Study 01's SELECT join placement arms of 2026–10–05: every distributed run recorded three tablets per relation, with leaders on three nodes before and after the cells (v6-sel-16, v6-sel-17). Elsewhere it remains unverified. The Study 05 key-locality pair has been executed: v3–06 records the colocated layout ahead on both reads, but its runner could not produce a tablet/leader distribution, so the layout labels are intent rather than observed placement. Study 04's y1/y2 pair is still unexecuted, Study 01 D7/D24 and Study 03 L3 remain placement intent with measured effects, and every node still runs as a container on one host.

Trials: 0 whole-run replications · 0 fresh-load trials per cell · 0 inner iterations

Limits: containers on one laptop are not evidence of colocation; the runner could not produce a tablet/leader distribution, so the placement labels are intent, not observed placement; the locality pair is one run per layout with large spreads

Source: studies/05-cache-consistency/reports/20260923T1215Z-res2-placement-yb3.md · run 20260923T1215Z-res2-placement-yb3 · tag run/05-cache-consistency/20260923T1215Z-res2-placement-yb3

rollups / materialized aggregates

v2-02-rollup-family

rollups / materialized aggregates · run_replicated

Parent rollups win the aggregate read score - D4 rollup/trg about 61–85x and D5 rollup/app about 64–83x D1 minimal in two whole runs (73.8x and 69.9x in the first) - and pay for it on inserts: about a third to two-fifths of D1's insert rate in the same two runs (2.6k/s and 2.1k/s against 6.2k/s in the first).

Trials: 3 whole-run replications · 1 fresh-load trial per cell · 1 inner iteration

Limits: one run of the experiment, a single measurement per configuration; PostgreSQL 1 node numbers shown; YugabyteDB in the same report; the write cost is per-insert on a fresh load; the quoted rates (2.6k/s, 2.1k/s, 6.2k/s) are the primary run's levels: the 2026–10–01 replication measured D1 inserting at 6.1k/s and 4.9k/s with D4/D5 at 0.29–0.37x of it, so the read band and the write penalty ratio reproduce while the absolute rates do not

Source: studies/01-charity-tree/reports/20260912-small.md · run 20260912-small · tag none (legacy)

v2-04-recency-maintained-index

rollups / materialized aggregates · run_replicated

The recency flag works, but an index on the rollup column answers the window questions faster, and by how much depends on the question: across two whole runs on PostgreSQL (d20_recency_flag against d22_recency_rollup_idx) the index is 1.9–2.3x on q13 (the primary's quoted 2.9x is this query in its trailing regime), 2.6–3.1x on q15 trailing, level on q14 (0.98–1.24x) and slightly slower on q16 (0.85–0.94x), so the group-wide ratio is about 1.4x. The YugabyteDB half (quoted 7.9x) is unreplicated.

Trials: 3 whole-run replications · 2 fresh-load trials per cell · 1 inner iteration

Confounds: conf-02-study01-d20d22-write-axis

Limits: we cannot say how much of d22's write cost comes from the index, because d22 also maintains D4's multi-column rollup: the two designs differ by more than the index; three repeats of the measurement per configuration; claim-02's v1 trials field was recorded as 1 and is corrected here; the quoted 2.9x and 7.9x are single-query, single-engine figures (q13 trailing): the recency group geomean is about 1.4x here and 1.52x recomputed from the primary, and on q16 the flag is marginally FASTER than the index in both runs - quote the query, not one headline

Source: studies/01-charity-tree/reports/20260916T090036Z-v3.md · run 20260916T090036Z-v3 · tag run/01-charity-tree/20260916T090036Z-v3

v2-13-trigger-rollup-cost

rollups / materialized aggregates · run_replicated

Maintaining the portal overview with a per-row trigger (n3_rollup_trigger) costs 4.0–5.7x the application-maintained rollup (n4_rollup_app) on whole-configuration replacement across three runs in randomized order (median 4.94x: n4 518–635 ops/s against n3 91–159 ops/s), and the trigger's slowest 1% is 22–46x worse (p99 61–109 ms against 2.2–2.7 ms); repeating the measurement inside the trigger's own configuration disagreed by 35–115%, so this is a range rather than one number, and the 6.85x quoted earlier from a single run did not reproduce.

Trials: 3 whole-run replications · 3 fresh-load trials per cell · 1 inner iteration

Limits: PostgreSQL single node, small scale; three whole runs, each with three repeats of the measurement per configuration; the experiment was repeated as separate whole runs on one machine, so it is not independent of this hardware and is still exposed to this host's measurement hazards; repeating the measurement in that configuration disagreed by 35–115% across the three runs, more than the ~20% this project allows, so only the direction and an order of magnitude are supported, not a single ratio; the ~15x read gain belongs to the earlier single run and was not re-measured in the replication; the 6.85x figure from the earlier one-off run did not reproduce and must not be quoted; the trigger's product is a structural guarantee, not speed; the three runs realized only two distinct design orders (seeds 202 and 303 coincided, and in the sibling read tranche all three seeds coincided); randomized order describes the method, and the realized order is recorded in each manifest; the penalty is size-dependent, not a constant: crossing the same pair with entries per installed product gives 2.07x at 10 entries, 4.38x at 60 and 24.20x at 240, so this ratio holds at the small scale default of 60 entries and must not be quoted as a property of the design without the size (v2–18-maintenance-size-interaction)

Source: studies/04-configuration-portal/reports/20260925T190831Z.md · run 20260925T190831Z · tag run/04-configuration-portal/20260925T190831Z

v2-18-maintenance-size-interaction

rollups / materialized aggregates · run_replicated

Crossing who maintains the parent rollup (trigger n3_rollup_trigger against application n4_rollup_app) with the size of a replacement (10, 60 and 240 entries per installed product) shows the maintenance penalty is a function of size rather than a constant: across 7, 9 and 7 runs at the three sizes the application design delivers roughly 1.1–3.2x the trigger design on whole-configuration replacement at 10 entries, 2.5–9.0x at 60 and 10–24x at 240, the ordering by size holding in every arrangement measured, and no rollup mismatch is recorded at any size.

Trials: 7 whole-run replications · 1 fresh-load trial per cell · 1 inner iteration

Limits: PostgreSQL single node, small scale, one measurement per tier across 10, 60 and 240 entries per installed product, so the curve carries no spread at any level; one operation only: w05_replace_all replaces the whole configuration, and a workload that appends a single entry would sit at the low end of this curve at any size; the axis is capped by the small scale at 240 entries and the curve is still rising steeply there, so nothing here bounds where it flattens; no negative control in these three runs; control_coverage is recorded absent rather than implied; two designs cannot separate the two mechanisms that predict this shape: the trigger fires once per row written, and its per-row work may also be heavier than the application's single statement; at 60 entries per installation the three runs give a median of 3.42x, about 22% below the 4.38x the first run recorded and below the 3.99–5.69x v2–13 replicated for the same operation, so the corroboration of v2–13 here is approximate - same operation, same size, same direction - and not a match; the ratio at each size is a band describing what has been seen, not a range a future reading must fall inside: 7 runs at 10 entries span 1.06–3.15x, 9 at 60 span 2.51–8.95x and 7 at 240 span 10.20–24.20x, and the two lower bands touch when everything is pooled (3.15 against 2.51) even though every individual arrangement ordered them correctly; arrangement in time decides what a repeat can see: three runs of a size placed next to each other in the session agreed within 15–55%, while the same three runs spread across the session (a Latin square) spread by about 100% - the sequential layout understated the uncertainty because adjacent runs share the session's state; a reference cell measured immediately before and after this tranche moved 445 to 640 ops/s, +44%, so the session was not a stable instrument and the drift it produced is measured rather than inferred; the reference is a different operation from the one compared here and bounds the movement without being its exact size; a band built from six runs did not contain a seventh: tier 10 measured 1.06 against the 1.34–3.15 range the earlier tranches had established; the session does not drift in one direction, it wanders: three readings of one reference cell taken through a single tranche gave 8.64, 5.21 and 8.53, so reading only the start and the end of that tranche would have reported no movement while the middle sat 39% below both; the ratio is more portable than the absolute numbers: over those same three reference readings the ratio spread 40% against 49% for the application design's throughput and 114% for the trigger's, because a session term shared by both designs cancels in the ratio

Source: studies/04-configuration-portal/reports/20260926T152621Z.md · run 20260926T152621Z · tag run/04-configuration-portal/20260926T152621Z

embedded documents

v2-03-embedding-not-ahead

embedded documents · run_replicated

Embedding the donation child in the donor row (D6 embedded) never came out ahead of the normalized designs on reads: about 1.7–2.2x D1 minimal's read score in two whole runs (2.4x in the first), below both indexed designs, with about 0.57–0.60x D1's insert rate (0.57x in the first).

Trials: 3 whole-run replications · 1 fresh-load trial per cell · 1 inner iteration

Limits: one run, one trial; embedding here means a JSON/array child column, not a native document database; the band moved down on replication (1.7–2.2x against the quoted 2.4x) while staying below both indexed designs, so the finding is the ordering and the sub-multiple, not the 2.4x

Source: studies/01-charity-tree/reports/20260912-small.md · run 20260912-small · tag none (legacy)

v2-11-section-seat-map

embedded documents · run_replicated

On YugabyteDB the section document (L2 document) answers the 100k-seat map 4.5–11x faster than per-seat rows, while the section-sharded variant (L3 sharded) collapses point lookups to 26 ops/s; PostgreSQL shows no attributable difference. This is a family/package comparison, not a one-decision pair. The engine contrast now has a replication on both halves: the same pair of designs measures 24.4x and 27.8x on yb-single and 0.83x on pg-single, so the substitution is worth about 25x on one engine and nothing on the other.

Trials: 5 whole-run replications · 3 fresh-load trials per cell · 1 inner iteration

Confounds: conf-03-study03-layout-family-package

Limits: one read-only run, one trial, 5 s per measurement; quota-bound nodes; read-only phases; no control could fire; a difference below ~1.8x cannot be attributed to the design rather than to run-to-run scatter; a replication on yb-single on 2026–09–26, three repeats per cell, reproduced the direction strongly: l2_section_document read the 100k-seat map at 1138 ops/s against l1_claim_rows at 41, a 27.8x multiple against the 20.8x of 2026–09–21, while both absolute figures fell far below the original (l1 by 72 percent, l2 by 63 percent), so the multiple is the claim and the figures are one session's readings; l3_section_sharded has failed its load in all three passes, in three different positions and under two memory envelopes, and with two different symptoms (connections terminated mid-COPY with SQLSTATE 57P01, then connect: connection refused). The sharded collapse therefore cannot be tested on this host with this harness - which is different from having tested it and found nothing; the pair has since been measured in both orders: l1 then l2 gave a 27.8x multiple and l2 then l1 gave 24.4x, a 12 percent difference, so the claim's direction does not depend on which design runs first; the pass that supplied the reversal ran with the database container at 5 GB and the YugabyteDB memory limit at 3 GiB rather than 3 GB and 1.5 GiB, because the earlier passes lost cells to the out-of-memory killer; its absolute figures are therefore not comparable with the 3 GB runs, and only the ratio between the two designs inside a run is; the load failures are memory pressure rather than design behaviour: the database log records server processes terminated by signal 9 (killed), 122 such lines in one cell, on a host with 15 GB of memory - so the sharded half of this claim and its PostgreSQL half are blocked here by host capacity, and a lost cell of this kind never reached the correctness gate rather than failing it; the two passes that supply the AB/BA pair also differ in memory envelope, so the 12 percent agreement may hide an order effect and a memory effect that cancel; a pass at 5 GB in the first pass's order would separate them; the PostgreSQL half was replicated on 2026–09–27 and shows no attributable difference (l2_section_document 6565 ops/s against l1_claim_rows 7873 on the 100k-seat map, a 0.83x multiple), so the finding is the engine contrast - about 25x on YugabyteDB against none on PostgreSQL - rather than the design being faster in general; that PostgreSQL measurement is a single run in a single design order, unlike the YugabyteDB pair which has the AB/BA control; a reverse-order pass there is still owed; l3_section_sharded is declared yugabyte_only_designs in this study's runner and is therefore never attempted on PostgreSQL, so the claim's sharded element can only ever be engine-specific evidence; its 2026–09–21 figures (15.9 and 28.3 ops/s) have not been reproduced on any pass since because its load fails on yb-single; the PostgreSQL half has since been measured in both orders as well: l1 before l2 gave a 0.83x multiple and l2 before l1 gave 0.88x on the 100k-seat map, so the engine contrast (about 25x on YugabyteDB against none on PostgreSQL) now has the same order control on both engines and both halves; l3_section_sharded loads, verifies 65/65 and reproduces its collapse when it is the only cell in a session (2026–09–27), which shows its repeated load failures inside a matrix are a session effect on a machine whose declared container budgets cannot be enforced and not a property of the design; that dedicated run was pre-registered as a diagnostic that would not be folded into this claim, so it is recorded here as a route to measuring the sharded element rather than as the element's replication - accepting it as such is a decision, not a mechanical update

Source: studies/03-reserved-seating/reports/20260921T231328Z.md · run 20260921T231328Z · tag run/03-reserved-seating/20260921T231328Z

v2-gap-06-native-datastore-families

embedded documents · gap · coverage gap

No native document, key-value, graph or wide-column database was measured: JSONB/state is not evidence for a native document database, and Redis was measured only as a cache. Cross-family transfers to native datastores are analogy, not evidence.

Trials: 0 whole-run replications · 0 fresh-load trials per cell · 0 inner iterations

Limits: do not transfer the embedded-document or cache results to a native family

Source: studies/05-cache-consistency/reports/20260921T-survey3.md · run 20260921T-survey3 · tag run/05-cache-consistency/20260921T-survey3

append-only history / snapshots

v2-07-refund-answerability-and-storage

append-only history / snapshots · run_replicated

A ticketing database chosen only to prevent overbooking cannot report refunds at all: in 14 of 15 designs the refunds report (r05) is unanswerable and the sales/revenue (r01) and last-purchase (r03) reports are wrong across a refund, reproduced in two whole runs in both design orders on PostgreSQL at scale small, with the append-only ledger (X1 CAS+ledger) the only design that answers r05. The ledger's storage cost depends on which relations are counted and moves about a percentage point between runs: about 33–35% more than P3 by table size (33.4% and 34.8% in the two runs, 35.2% in the primary), about 35–36% counting every relation (35.0% and 35.9% in the two runs, 36.2% in the primary), and 37.4–37.9% by index size alone.

Trials: 5 whole-run replications · 1 fresh-load trial per cell · 1 inner iteration

Limits: answerability is a schema fact, not a timing; the +33–35% storage delta is a relation-size difference measured in the primary run and deterministic for the same dataset and design; it is not a repeated timed result; the P3 -> X1 read rows of this run are excluded, because the difference belongs to the configuration as a whole rather than to one decision inside it; the read phase of this claim's primary anchor 20260920T234953Z was client-CPU throttled by a design-dependent amount (2.3 s for p2_precreated_skip_locked to 58.1 s for r1_inventory_row, out of about 1085 CFS periods), so that report's read figures are not quotable: the hazard reproduces on the current harness (20260929T165533Z, 20260929T170353Z: r1_inventory_row 32.3 s of ~63 s of periods, both orders), and the 2026–09–30 budget calibration shows the client's quota is a symptom rather than the ceiling - the read level on this machine is set by the database's share of four CPUs, and adding a client core removes the client's throttling while making the reads slower. Answerability and the r05 storage/speed result are unaffected: the ledger still answers r05 at 30.3k and 33.0k ops/s against the primary's 32.2k, and at the new 2-worker read footprint the r05 result reproduces independently at 32,164 ops/s (20260930T164028Z), so that number has a footprint-2 footing; all fifteen designs now have one (the eleven-design tranche 20260930T201732Z/20260930T203913Z adds the rest, unthrottled), while this claim's primary anchor's own footprint-8 read figures remain not quotable (design-dependent inflation, 1.09x to 2.22x). **Correction, 2026–09–30: an earlier caveat here said the two many-row queries (q02_band_events, q05_band_tickets_sold) had client-bound levels; it is withdrawn.** A one-worker tranche (20260930T232926Z, 20260930T233744Z) shows that in-flight measured with the **mean** latency is exactly 1.000 in all 72 cells at one worker and 2.00 at two, so every read cell in the family is a store measurement at a stated footprint, and only its magnitude depends on the concurrency. Two errors produced the withdrawn caveat: q05 does not return 28 rows (all three variants return one - COUNT/SUM with no GROUP BY), and in-flight computed from the **median** understates the concurrency by the latency skew (mean/p50 up to 2.7x), which reads exactly like a worker idling between fetches. The primary anchor's **PostgreSQL read cells have since been re-stated** on the footprint-2 footing (analysis 20261001T-read-cells-restated: all fifteen designs, both orders, every cell store-bound at 2.00 operations in flight), so its `pg-single` read figures may be replaced rather than caveated. All fifteen designs have since been measured in a **single run**, twice and in reversed order (20261001T100948Z, 20261001T103931Z): 0 throttled periods in all 30 read phases and every one of the 270 cells store-bound at 2.00 operations in flight by the mean latency, so the family's levels are jointly comparable within that session. Two consequences for quoting: the top group (h0, h1, r1, r2, p4) differs by 4%, inside that session's own order term, so its internal ranking is not quotable - only the 1.5x gap between that group and the per-seat group is; and c1_count_naive carries a 40% position term (first of one order against last of the other), so its q01 level should not be quoted from these runs. Its two **YugabyteDB read arms have no replacement** and remain an explicit coverage gap; the storage percentage depends on the rule and on the page layout, and must not be quoted as one number: tables only 33.4–35.2%, every relation 35.0–36.2%, indexes alone 37.7%; the same pair's relation sizes moved 0.26% (p3) and 0.41% (x1) between two runs 29 minutes apart, so the delta moved 0.9 percentage points; the index-only figure spans 37.4–37.9% across the five runs, so even there the rule is a band and not a point, and the 2026–10–02 replication's M4 (reproducible within 1%) is falsified by that movement - a tighter figure would need a vacuumed comparison, which is a different measurement; the answerability statuses are the harness's own audit of its own schema, computed as a pure function of the design and emitted inside the read phase; the 2026–10–02 replication therefore tests reproducibility across two whole runs and not the independence of the audit's rule, and it re-measured no timing

Source: studies/02-ticket-booking/reports/20260920T234953Z.md · run 20260920T234953Z · tag run/02-ticket-booking/20260920T234953Z

v2-08-ledger-race-cost-128-buyers

append-only history / snapshots · run_replicated

At 128 buyers the append-only ledger's cost over compare-and-set rises with the size of the event rather than sitting in one band: on PostgreSQL it is about 1.1x at 10 seats (where the direction is not reliable), about 1.2–1.4x at 100–1000 seats, about 1.8x at 10000 seats and about 2.5x at 100000 seats, measured in twelve whole runs in both design orders, with wide spreads at the small tiers. The 1.8–2.2x on YugabyteDB, the 2.1–2.5x on the refund itself and the up-to-5x delay to the organiser's own edit are single-run figures that have not been re-measured, and no design ever under-booked a seat.

Trials: 12 whole-run replications · 5 fresh-load trials per cell · 1 inner iteration

Confounds: conf-05-study02-buyer-arms-disagree

Limits: one small run, closed loop, quota-bound nodes; scoped to 128 buyers; must not be extrapolated to 32 buyers; single run, three trials per tier; replicated on 2026–09–27 in both design orders at 128 buyers, three trials per tier: the p3/x1 ratio came out 1.53x, 1.71x and 2.00x at 100, 1000 and 10000 seats in the first order and 0.99x, 1.30x and 1.74x in the reversed order, so the ratios reproduce in size while the lowest of them falls below this claim's 1.1x floor; the ordering is not stable at the smallest tier: CAS read 4163 against the ledger's 2724 at 100 seats in the first order (1.53x) and 3588 against 3618 in the reversed order (0.99x), i.e. the ledger marginally faster - so the designs swap places at a fixed buyer count, which is what v2–09 records for changing buyer counts; the former 'tight spreads' wording did not hold at that tier: the 100-seat cell spread 57% for CAS and 37% for the ledger in the first order, and 47% and 51% in the reversed order, while every other tier stayed within 4–17% - and because the wide spread appeared at the same tier in both orders it belongs to the cell rather than to a design's position in the session; re-scoped on 2026–09–27 (HIGH, task repo/v2–08-rescope): the flat 1.1–2.3x band and the words 'tight spreads' were removed from the statement because the cost measured across the tier axis is monotone (about 1.1x at 10 seats to about 2.5x at 100000) and the small-tier cells spread 47–77%; the superseded wording, the evidence behind each change, and the earlier post-v6 re-scopings of v2–06 and v2–11, are recorded in book/evidence/v6/SUPERSESSION_LEDGER.md; the earlier 'single run, three trials per tier' limit describes the primary run only; the claim now rests on 8 whole runs (2 at 3 trials across 3 tiers, then 6 at 5 trials across the 100- and 1000-seat tiers); settled on 2026–09–27 by a third tranche of six runs (5 fresh-load trials per cell, three in each design order): the 100-seat p3/x1 run median was 1.279, 1.333, 1.257, 1.712, 1.328 and 1.476, so the single 0.99x reading of the reversed replication does not reproduce and the former band is not widened by this evidence; the ledger was slower in 29 of the 30 individual trial pairs (pooled per-trial median ratio 1.391x, range 0.940–1.920); the former 'tight spreads' wording still failed at the 100-seat tier across all eight runs: the cell's spread was 47–77% for CAS and 26–54% for the ledger in the six settle runs, and the run-to-run spread of the tier-100 ratio is 34% over those six and 54% over all eight (0.992–1.712, median 1.331), so that tier supports a direction and not a point ratio; the 1000-seat positive control read 1.304–1.712 in all six settle runs, so the harness detected the ledger's cost in every run; correctness was green throughout (46/46 P3, 48/48 X1 verifications, 0 over-booked, 0 under-booked, 0 timed-out), and no settle cell is void; the tier axis was then measured on 2026–09–27 in four more whole runs (5 trials per cell, two runs in each design order, all five tiers measured in every run): the pooled run-median p3/x1 rose 1.075, 1.237, 1.409, 1.845 and 2.505 at 10, 100, 1000, 10000 and 100000 seats, monotone in the pooled medians and in every individual run, so the ledger's cost is a function of tier and the claim's former 1.1–2.3x band described its middle three tiers rather than the whole axis; the statement's YugabyteDB, refund and organiser-edit figures come from the primary run alone and are labelled as not re-measured; the replication programme covered only the PostgreSQL race, so nothing here re-measures them, and v2–09's finding that the design order moves the ratio at the small sizes still applies; both endpoint tiers lie outside the former band and had never been repeated before this tranche: at 10 seats the run medians were 1.023–1.180 (pooled 1.075, below its 1.1x floor) while the per-trial median was 1.131 and the ledger was slower in only 12 of 20 trial pairs, so the cost there sits at the floor and its direction flips in 40% of trials; at 100000 seats the ledger was slower in 20 of 20 pairs (pooled per-trial median 2.456, run medians 2.249–2.637), above the 2.3x top; the 100000-seat tier is a single event per trial, so its 20 trial pairs are 20 readings of one event and an event-specific effect would be indistinguishable from a tier effect; the 10-seat cell is near-degenerate (100 events of 10 seats against 128 buyers) and its cells spread 16–71%; correctness was green in all four tier-axis runs (0 over-booked, 0 under-booked, 0 timed-out, 0 errors); with the two earlier tranches the tier-100 reading now rests on twelve runs whose run medians span 0.992–1.712 (median 1.331, spread 54%), so that tier still offers a direction rather than a point ratio

Source: studies/02-ticket-booking/reports/20260921T205212Z.md · run 20260921T205212Z · tag run/02-ticket-booking/20260921T205212Z

v2-09-postgres-ledger-ratio-unsettled

append-only history / snapshots · run_replicated

On PostgreSQL the ledger is slower than compare-and-set at 1,000 seats at every buyer count tested, 8 to 128, and in both design orders (55 of 55 trial pairs over eleven runs), so the two designs do not change places with the load: the primary run's apparent swap came from its own confound, buyer count and design order having moved together. The size of the cost does vary without a simple trend (about 1.2–1.3x near 16–32 buyers and 1.4–1.9x at 64–128; one 8-buyer cell changed state mid-run), and the design order moves absolute throughput several-fold without changing which design is faster. On YugabyteDB the primary run's own range is 1.7–5.7x, wider than a headline would quote.

Trials: 11 whole-run replications · 5 fresh-load trials per cell · 1 inner iteration

Confounds: conf-05-study02-buyer-arms-disagree

Limits: the stable direction is measured at 1,000 seats across 8–128 buyers and at 32 buyers across all five tiers; the buyer-count axis is not swept at the other four tiers, so the full buyer x tier surface is open; one primary run per (buyers, order) cell, with a reference cell that moved 8.6% between the two ends of the tranche, so a level whose two orders differ by more than that is placed by the ratio and not by the absolutes; the direction is stable but the ratio is not: the eleven run medians span 1.06–5.10 (1.22–1.86 with the collapsed 8-buyer cell set aside), so the cost is a range and not a point; the 8-buyer level's pooled median is not interpreted: a recurrence test of six runs and twelve cells (2026–09–29, tag study-02/v2–09–8buyer) found four low-throughput events in 60 trials, readings of 619–655 ops/s against a session normal of about 2500 (a 4x drop), and they hit both designs (three of the four in the CAS arm) and both orders, so the low mode is a property of this buyer count's very short trials and not of the ledger; the original p3-first cell read as a state change only because its slow trials were its last three, which set the cell median; the primary evidence is from a different harness state (phases verify,write,race,churn on a five-day-old image), so comparing it with this tranche mixes the confound with a period effect and the re-scope is stated as a direction, not a ratio; one side of the primary comparison is a partial configuration on yb-cluster3 (medians of two measurements); at 32 buyers the ledger was slower in all ten tier x order cells of two 2026–09–29 runs (ratios 1.021–1.606) and below the 128-buyer pooled ratio at every tier (1.06 vs 1.08, 1.06 vs 1.24, 1.23 vs 1.41, 1.56 vs 1.85, 1.56 vs 2.51), so the primary's tier-wide reversal does not reproduce anywhere on the axis; those two runs are anchors but are not counted in the eleven-run replication record, which is the buyer-count sweep at one tier; at 32 buyers and 10–100 seats the direction sits at the measurement's floor: only 3 of 10 trial pairs at 10 seats and 6 of 10 at 100 favour the CAS design, with cell spreads of 86–174%, so those two tiers support the median and not a per-pair claim, while the 1,000-seat cell reproduced the sweep's ratio a day later within 1% (1.245 and 1.219 against 1.233 and 1.219); the 8-buyer low-throughput mode is not a quota effect on either side: the twelve cells' per-trial client_cpu entries (present since the study's first commit, and corrects an earlier limit here that said the client was not recorded) show zero client throttling and *more* client CPU in the slow trials (2.6–2.8 s against about 1.9 s), so the client waits rather than starves, while the database container's cpu.stat throttled about 4 ms across the collapsed cell; the cause is narrowed to a latency excursion on the response path and is not identified further, since no per-trial plan or server-side clock is recorded; three of the four 8-buyer low events were single trials that recovered on the next trial and one spanned consecutive trials, so 'persistent collapse' and 'transient spike' are names for where the event fell in the cell (a five-trial median cannot distinguish them at a ~7% event rate); no 8-buyer magnitude is quoted from either tranche

Source: studies/02-ticket-booking/reports/20260921T183722Z.md · run 20260921T183722Z · tag run/02-ticket-booking/20260921T183722Z

v2-gap-01-ledger-load-multiplier-unstable

append-only history / snapshots · gap · unstable measurement

The append-only ledger's load-time multiplier is not published: the same pair and loader measured X1's pg-single load at 7.8x P3's in one run and 1.2x in the two sibling runs, with an empty harness diff.

Trials: 3 whole-run replications · 1 fresh-load trial per cell · 1 inner iteration

Limits: a single one-off load of one configuration is not a cost of the design; needs its own repeated load-only measurement before any multiplier can be quoted; the read phase of this claim's primary anchor 20260920T234953Z was client-CPU throttled by a design-dependent amount (2.3 s to 58.1 s across designs, out of about 1085 CFS periods) and the hazard reproduces on the current harness (20260929T165533Z, 20260929T170353Z), so that report's read figures are not quotable; the load-phase figures this claim rests on are a separate phase and were not throttled. The read family has since been re-measured at the new 2-worker read footprint for four designs (20260930T164028Z, 20260930T164834Z): their read levels are quotable with the footprint attached, and all fifteen designs now have a footprint-2 footing (the eleven-design tranche 20260930T201732Z/20260930T203913Z adds the rest, unthrottled, and 20261001T100948Z/20261001T103931Z measures all fifteen together in one session); this claim's primary anchor's own footprint-8 read figures remain not quotable

Source: studies/02-ticket-booking/reports/20260920T234953Z.md · run 20260920T234953Z · tag run/02-ticket-booking/20260920T234953Z

v2-gap-02-hold-funnel-unanswerable

append-only history / snapshots · gap · schema limitation

No Study 03 design can answer the hold funnel report that would size the 40-minute hold window: releasing an expired hold clears the row, so an expired hold is indistinguishable from a seat never held.

Trials: 0 whole-run replications · 0 fresh-load trials per cell · 0 inner iterations

Limits: a schema limitation, not a missing measurement

Source: studies/03-reserved-seating/reports/20260921T231328Z.md · run 20260921T231328Z · tag run/03-reserved-seating/20260921T231328Z

external read copies / caches

v2-14-cache-throughput-gain

external read copies / caches · run_replicated

Adding a cache to warm cacheable reads buys 2.2–2.5x in throughput: two whole pg-single + Redis runs on 2026–10–02 measured the cached warm read at 2.54x and 2.24x the no-cache owned baseline, inside the primary's 2.2–3.5x band. The primary's p99 clause did not reproduce: the cached tail here was 8.5–10.1 ms against the baseline's 26.9–27.6 ms, a cut of about 3x rather than the primary's order of magnitude (0.8–2.0 ms against 15.8–27.1 ms), so the claim carries the throughput band and records the tail as not reproduced under this client. The cache's own CPU and memory are not counted against the database side (db-only framing), and Redis here is a shared cache, never an authoritative key-value store.

Trials: 3 whole-run replications · 3 fresh-load trials per cell · 1 inner iteration

Confounds: conf-04-study05-db-only-cache

Limits: one machine, closed loop, small scale (800 donors), PostgreSQL + Redis cache only; the p99 clause did not reproduce: cached p99 8.5–10.1 ms against the baseline's 26.9–27.6 ms (about 3x), where the primary recorded 0.8–2.0 ms against 15.8–27.1 ms; the tail is suspected client-bound at this client budget and was not isolated with a plan or a footprint cell; warm cacheable reads only; the Redis container's budget is outside the db-only comparison (the equal-total framing is a separate claim, v3–03); absolute throughput is session-specific; only the within-run ratio travels; the 2.2–3.5x range remains supported only by the PostgreSQL single-node + Redis cells; do not transfer it to YugabyteDB, a cluster, or open-loop demand

Source: studies/05-cache-consistency/reports/20260921T-survey3.md · run 20260921T-survey3 · tag run/05-cache-consistency/20260921T-survey3

v2-15-three-instance-staleness

external read copies / caches · mechanism_supported

Three logical application instances sharing one Redis cache on write-through with relaxed freshness record about 87% wrong reads, while every relaxed configuration on a single instance records none; a private in-memory lease cannot coordinate separate instances, and a database version token that is never checked does not remove the problem. The 87.81% belongs to the owned model (o:opt:red:wth:rel) and 87.24% to the legacy model.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Confounds: conf-04-study05-db-only-cache

Limits: single trial, one machine, small scale; generality unproven; the 87%-vs-0% separation is far outside the ~20% noise floor, but the exact rate is one trial; the signed all-green analysis swapped the two models' rows; this claim uses the report and JSON attribution

Source: studies/05-cache-consistency/reports/20260921T-survey3.md · run 20260921T-survey3 · tag run/05-cache-consistency/20260921T-survey3

v2-16-strict-freshness-read-cost

external read copies / caches · run_replicated

The strict-freshness contract cost nothing measurable in READ throughput: the controlled strict/relaxed pair differed by 7.3% in one whole run and 5.7% in the other with the sign flipping, inside the cells' 8.7%-22.8% trial spread, so the null reproduced in both design orders. This claim measures read throughput only: it does not quantify the write-path cost of a strict protocol, and it must never be restated as 'strict freshness is free'.

Trials: 3 whole-run replications · 3 fresh-load trials per cell · 1 inner iteration

Confounds: conf-06-claim12-full-noise-range, conf-04-study05-db-only-cache

Limits: read throughput only; the two fresh runs' trial spreads reached 22.8%, so only an effect larger than that could have been seen; small scale, closed loop, one machine; the strict/relaxed pair here is the owned write-through pair; a strict protocol's write-path cost is not measured

Source: studies/05-cache-consistency/reports/20260921T-survey3.md · run 20260921T-survey3 · tag run/05-cache-consistency/20260921T-survey3

v3-01-churn-crosses-real-ttl

external read copies / caches · single_run_directional

A sustained 1801 s churn phase against the fixed 300 s TTL crossed 6.0 real TTL boundaries with 0 wrong reads in 2,879,077 blended reads and 380,824 writes; the hard-expiry counter is 0 and only probabilistic early expiry fired (37,063), so an entry was never observed to be removed by its hard TTL. This is scoped mechanism and duration evidence, not a general cache result.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Limits: one PostgreSQL + Redis configuration at small scale; one sustained measurement, never repeated; hard expiry fired zero times; that clause is an open gap, not a confirmation; closed loop, so read volumes are a floor; the 5x-for-120 s burst has no write-rate knob and was not run; the 1800 s horizon was not re-run on 2026–10–02: the 600 s pair recorded for v3–02 re-tests the same mechanism at a shorter horizon, and this claim's 1801 s / 6.0-boundary figure remains the primary's single run

Source: studies/05-cache-consistency/reports/20260923T1100Z-churn1800-through.md · run 20260923T1100Z-churn1800-through · tag run/05-cache-consistency/20260923T1100Z-churn1800-through

v3-02-churn-600-through-vs-aside

external read copies / caches · run_replicated

At 601 s the write-through and write-aside churn cells each crossed 2.0 real 300 s TTL boundaries with 0 wrong reads, in two whole runs in reversed order (through 2.004/2.003 boundaries with 1,326,612 and 885,864 reads; aside 2.003/2.004 with 1,173,729 and 1,156,637), and neither strategy is claimed to win. One aside cell recorded 1 impossible cache value in 1,173,729 reads while its gate passed, so the study's floor that an impossible value must always be zero did not hold cleanly and the gate does not classify impossible values during churn.

Trials: 4 whole-run replications · 3 fresh-load trials per cell · 1 inner iteration

Limits: the cache container's own resources are not counted in this comparison; hard expiry fired 0 times in every cell; only probabilistic early expiry fired, and the 5x burst was not run; closed loop, so read volumes are a floor; one cell (20261002T191609Z, aside-strict) recorded 1 impossible cache value that the gate did not catch; it is not repeated in the other three cells and has no identified cause, and the gate's coverage of impossible values during churn is a recorded gap

Source: studies/05-cache-consistency/reports/20260923T1020Z-churn600-through.md · run 20260923T1020Z-churn600-through · tag run/05-cache-consistency/20260923T1020Z-churn600-through

v3-03-equal-total-framing-labelled

external read copies / caches · run_replicated

Cache resource accounting carries a second, separately labelled comparison in which both sides are given the same total resources: two whole runs on 2026–10–02 recorded the db-only framing at 8,199 warm ops/s (cache budget none) and the equal-total framing at 7,931 (cache 1 CPU / 768 MiB), 3.3% apart inside the 17.7%/18.4% trial spread, with resource_framing and both budgets recorded in every result. What the claim asserts is that the two ways of counting resources are labelled and never pooled; the primary's 11k/8.0k point figures did not reproduce and are not quoted.

Trials: 4 whole-run replications · 3 fresh-load trials per cell · 1 inner iteration

Limits: what matters is that the two framings are labelled and kept apart, not the ratio between them; the fresh runs are 3.3% apart, inside the 17.7%/18.4% spread; the primary's 11k/8.0k figures did not reproduce; one host, small scale, each framing measured once per session

Source: studies/05-cache-consistency/reports/20260923T1150Z-res2-equaltotal-pg.md · run 20260923T1150Z-res2-equaltotal-pg · tag run/05-cache-consistency/20260923T1150Z-res2-equaltotal-pg

v3-04-yb-single-strict-relaxed

external read copies / caches · run_replicated

On YugabyteDB 1-node (RF=1) the strict and relaxed write-through reads differ by 1.043x and 1.105x, both inside the cells' 21.7%-74.7% trial spread, so the strict/relaxed read difference is a null on one node too. The primary's direction (relaxed 596 above strict 514) did not reproduce: both fresh runs lean strict-slightly-ahead, also inside spread, so no design is named the more consistent one.

Trials: 3 whole-run replications · 3 fresh-load trials per cell · 1 inner iteration

Limits: single endpoint (RF=1); this is not a cluster result; the trial spreads are large (up to 74.7%), so the null is weak evidence; the primary's direction (relaxed ahead) did not reproduce; the fresh runs lean strict-ahead, still inside spread

Source: studies/05-cache-consistency/reports/20260923T1152Z-res2-yb1.md · run 20260923T1152Z-res2-yb1 · tag run/05-cache-consistency/20260923T1152Z-res2-yb1

v3-05-yb-cluster3-strict-relaxed

external read copies / caches · single_run_directional

On YugabyteDB 3-node (RF=3), where the client spreads operations over all three endpoints, one run recorded the strict write-through read at 942 ops/s against the relaxed configuration's 846, the opposite direction from the 1-node pair, with each configuration measured once and no engine ranked.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Limits: one run per freshness policy; direction is not repeated and differs from the 1-node pair; three containers on one host, no real network; endpoint spread is round-robin by construction; app-99 phases have large spread (50.4% relaxed, 49.4% strict); not re-measured on 2026–10–02: yb-cluster3 RF=3 does not fit the fixed 2 GiB podman machine, so the opposite-direction result remains the primary's single run and this target is an explicit coverage gap

Source: studies/05-cache-consistency/reports/20260923T1157Z-res2-yb3.md · run 20260923T1157Z-res2-yb3 · tag run/05-cache-consistency/20260923T1157Z-res2-yb3

v3-gap-01-study05-remaining-regimes

external read copies / caches · gap · partially superseded by v4-gap-01-physical-placement-unverified, v4-gap-02-endpoint-distribution-partial · coverage gap + instrument gap

Moved to successor claims: v4-gap-01-physical-placement-unverified, v4-gap-02-endpoint-distribution-partial.

Still open in this claim: hard-TTL expiry never observed removing an entry; medium scale beyond the harness maximum of 3,000 people; open-loop arrival demand.

Statement below is unchanged from its source registry; the moved dimensions are not restated as open here.

Four things about the cache study remain unmeasured, and they are absences of evidence rather than results: no entry was ever seen to reach its hard expiry (0 hard expiries in all three churn configurations), nothing beyond the harness maximum of 3,000 people was run, the distribution of tables and leaders across the cluster is unrecorded, and no open-loop arrival pattern was run.

Trials: 0 whole-run replications · 0 fresh-load trials per cell · 0 inner iterations

Limits: these are coverage gaps, not null results; medium scale is an order of magnitude beyond the current generator; placement verification needs an instrument the runner does not have; open-loop demand is owned by task-20260922T200831Z-study05-v2-open-loop

Source: studies/05-cache-consistency/reports/20260923T1100Z-churn1800-through.md · run 20260923T1100Z-churn1800-through · tag run/05-cache-consistency/20260923T1100Z-churn1800-through

Retired predecessor claims

- `claim-gap-04-study02-03-validation-open` — Retired in v1->v2; the validation it described completed 2026-09-22. Kept in the v3 chain so the retirement history stays readable.
- `v2-gap-04-study05-unmeasured-regimes` — Superseded by v3-01..v3-06 plus v3-gap-01: real-TTL churn, repeated trials, equal-total accounting and YugabyteDB/cluster cells are now measured; the still-open regimes are retained explicitly in v3-gap-01.
- `v2-gap-05-colocation-unverified` — Retired: its clause 'Study 05's placement pair was never executed' is contradicted by v3-06, which executed the key-locality pair. The physical-placement clause is carried forward, narrowed, in v4-gap-01. *Closed:* Study 05 key-locality pair executed (v3-06).
- `v2-gap-07-real-network-balanced-endpoints` — Retired: its clause 'balanced client access across cluster endpoints is not proven' is contradicted for the 2026-09-23 Study 05 cells by v3-05, whose client spreads over all three endpoints. The remaining endpoint-distribution and no-real-network clauses are carried forward, narrowed, in v4-gap-02. *Closed:* Study 05 three-node endpoint spread (round-robin over three endpoints).